SWE5006 PRACTICE PROJECT · TEAM 8

BLOCBUY

ONE FOR ALL • ALL FOR ONE

A community group-buying platform with pay-first teams

Two prices: solo & team • *Pay first to team up* • *Full refund if the team size not met*

1 Introduction 2 Analysis, design, patterns 3 Testing 4 CI/CD demo 5 System demo

Lim Jiechao Liu Jin Panpakorn Siripanich Wang Ziang Yu Qianshuo

Graduate Certificate in Designing Modern Software Systems, 26 October 2026

Group buying fails when buyers cannot commit together

> **Small merchants can discount only against committed volume, but no single buyer can reach it: demand exists, but no buyer commits first.**

Today

- Merchants discount only when volume is predictable
- One buyer can never reach a bulk price
- Advertising is costly and not tied to real purchases

BlocBuy: pay-first teams

- Two prices: solo and team
- Pay upfront to start a team, then share the invite link
- Friends join by paying upfront too
- Fills in time: team price. Expires: everyone refunded automatically

Stripe test mode only

Scope: nine use cases, six quality attributes with targets

Report [FILL: § (#463)] · scope, quality attributes

IN SCOPE · PROPOSAL §6.1

UC1 Browse and view listings
UC2 Register / authenticate
UC3 Initiate a team (pay first)
UC4 Join a team
UC5 Team lifecycle resolution
UC6 Refund on team expiry
UC7 Share attribution
UC8 View my orders and teams
UC9 Admin listing management

+ Receive notifications (Sprint 4, SWE5001; off on dev)

OUT OF SCOPE · PROPOSAL §6.2

Merchant self-service	Analytics
Recommendations	Automatic team matching
Email, SMS, push	Native apps
Live payments	

QUALITY ATTRIBUTES AND TARGETS · PROPOSAL §6.3

Attribute	Target	Shown in
Correctness under concurrency	Team capacity never exceeded No payment double-charged or double-refunded	**§5 demo** **§3**
Performance	Interactive operations ≤ 2 s at 100 concurrent users Expiry resolved ≤ 1 min after the window	**§5 k6 run** **§3**
Security	Google sign-in Role-based admin Signed Stripe webhooks No credentials in code	**§5 demo** **§4**
Maintainability, extensibility	New pricing, payment provider or team consequence without editing domain logic	§2 patterns §4 §5
Code quality	SonarCloud gate on every PR Unit tests on the complex classes	§3 §4
Auditability	Every payment, refund and transition traceable Every artefact traceable to a commit	§4

JC Lim

Promised, delivered, changed, dropped: status on 21 October

Report [FILL: § (#463)] · status, promised vs delivered

DELIVERED **UC1–UC9**
All nine closed as completed

OPEN **Paid start, join, refund on dev**
Checkout returns; the webhook never confirms

DROPPED **Gated prod, ECS circuit breaker**
For SWE5006; built for SWE5001

PARTIAL **E2E, burndown, saved plans**
Stubs; daily data, no chart; manual

OUTSTANDING

Paid flow on dev: go/no-go 23 Oct
SSH port 22 open on the dev host
CodeQL off
CloudFront → ALB over HTTP until 16 Nov
Dev volume unencrypted

Changed: promised	Built
UC9 admin API only, no GUI	Admin console, built in Sprint 3
Adapter (UC6), Observer (UC7)	Gateway ports; Observer on UC5 → UC6
State for the team lifecycle	Transition table; State on listings
Google sign-in with JWT	Sa-Token session, HttpOnly cookie
Spring Data JPA	MyBatis behind repository ports
Dev on ECS Fargate and RDS	One EC2 host, Compose, database on the host
Secrets Manager, no static keys	SSM and a root-only env file; 2 IAM users, 1 key
Blocking OWASP and IaC scans	Trivy, not required; 3 per-job roles

Six of these eight changes have no ADR yet.

Milestones achieved and effort per member

Report [FILL: § (#463)] · milestones, effort

MILESTONES ACHIEVED

Date	Milestone
27 Aug	Proposal submitted
5 Sep	Backend pipeline live on dev
14 Sep	Sprint 1 tag
20 Sep	Stripe Sandbox acceptance (local)
22 Sep	Web app live on dev
27 Sep	Sprint 2 tag
30 Sep	Admin console merged
11 Oct	Sprint 3 tag
23 Oct	Sprint 4 tag [FILL: date]
—	Paid start, join, refund on dev: not yet

EFFORT PER MEMBER · PLANNED HOURS [VERIFY: S4 split, PR #538]

Member	S1	S2	S3	S4	Planned	Actual
Liu Jin	—	30	30	51	**111**	[CONFIRM: agreed]
Yu Qianshuo	—	30	30	48	**108**	[CONFIRM: agreed]
Wang Ziang	—	30	30	46	**106**	[CONFIRM: agreed]
Panpakorn	—	30	30	41.5	**101.5**	[CONFIRM: agreed]
Lim Jiechao	—	30	30	38	**98**	[CONFIRM: agreed]

Sprint 1 hours were not recorded

Team done: Sprint 2 134 of 150 h; Sprint 3 117 of 150 h

Proposal budget: 10 man-days each

Planned, not actual: no member logged actual hours, and the progress reports carry none. [CONFIRM: actual effort agreed by all five]

Ownership: one use case and one design problem each

Report [FILL: § (#463)] · ownership

EVERY USE CASE HAS AN OWNER · * PROPOSED, NOT YET AGREED

UC1 Lim* **UC2** Pan* **UC3** Eric **UC4** Eric **UC5** Eric* **UC6** Yorik **UC7** Lim* **UC8** Lim **UC9** Liu Jin

Notifications Pan [CONFIRM: owners marked * (open decision 4)]

Member	Use case	Design problem → GoF pattern
Liu Jin	UC9 admin listings and team terms	Listing lifecycle → **State** (full form)
Wang Ziang (Eric)	UC3 start, UC4 join	Team creation → **Factory Method**
Yu Qianshuo (Yorik)	UC6 refund on expiry	Team outcome consequences → **Observer**
Panpakorn (Pan)	Receive notifications	Outbox transports → **Template Method**
Lim Jiechao	UC8 my orders and teams	Pricing → **Strategy**

Each member presents their own design problem in section 2; the team statechart (UC4–UC6, UC8) goes with Eric's UC4 design.

Report: use case in §3.4, design problem in §3.5 [FILL: section numbers (#463)] [VERIFY: UC9 owner]

JC Lim

Four sprints: the Definition of Done reached, and what each retro changed

Report [FILL: § (#463)] · sprints, DoD, retrospectives

Definition of Done ladder: coded → tested → integrated → releasable → released

Sprint	Goal	Definition of Done reached by the goal increment
1 · 1–12 Sep	Walking skeleton: sign-in, browse, admin on dev	**integrated**: backend on dev 5 Sep; no frontend, so not releasable
2 · 15–26 Sep	Pay, join a team, see the order, replay-safe	**integrated**: merged, tested, Checkout off on dev; 134 of 150 h
3 · 28 Sep–9 Oct	Invite → pay → refund on deployed dev	**integrated**; goal not met under our "on dev" rule; 117 of 150 h
4 · 12–23 Oct	Complete system on dev; three services on ECS (SWE5001)	[FILL: step recorded at the 23 Oct review]

Same basis every sprint: integrated = merged to dev with every check green; releasable = shown working on deployed dev, our "on dev" rule

Retro improvement → change in the next sprint

- **1 → 2** No burndown, no hours → Daily board snapshots, dated day logs
- **2 → 3** A 10,000-line PR; cliff burndown → PRs under 1,500 lines, mid-sprint check, "on dev" rule in the DoD
- **3 → 4** [FILL: retro #451] → Owners keep board estimates current, checked mid-sprint

Agile artefacts: a story card, a burndown, a practice

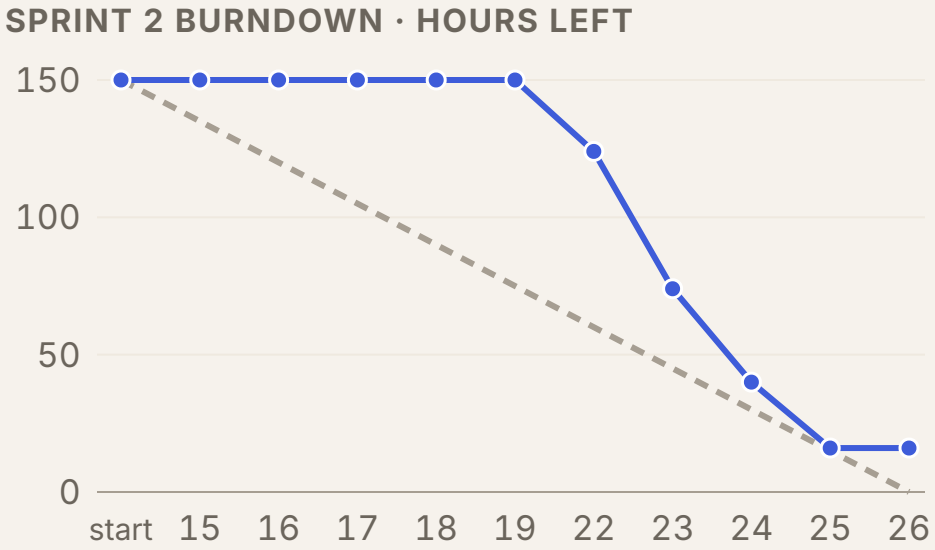
STORY CARD — Liu Jin

As an admin, I can set a listing's team size, lifetime and team price, so that buyers can start blocs on real listings.

- [x] Admin-only API: 403 for non-admins, 401 anonymous
- [x] Team size ≥ 2, lifetime in range, team price below individual; 400 with field errors
- [x] Stale version → 412; every write audited
- [x] A configured listing can start a team
- [x] Contract changes additive, merged before the controller
- [x] Unit, PostgreSQL and authorization tests; ArchUnit green

Design problem field: State, Optimistic Offline Lock

Report [FILL: § (#463)] · appendix: agile artefacts



Flat for a week, then a cliff: work merged late in large PRs

Reconstructed from merge dates

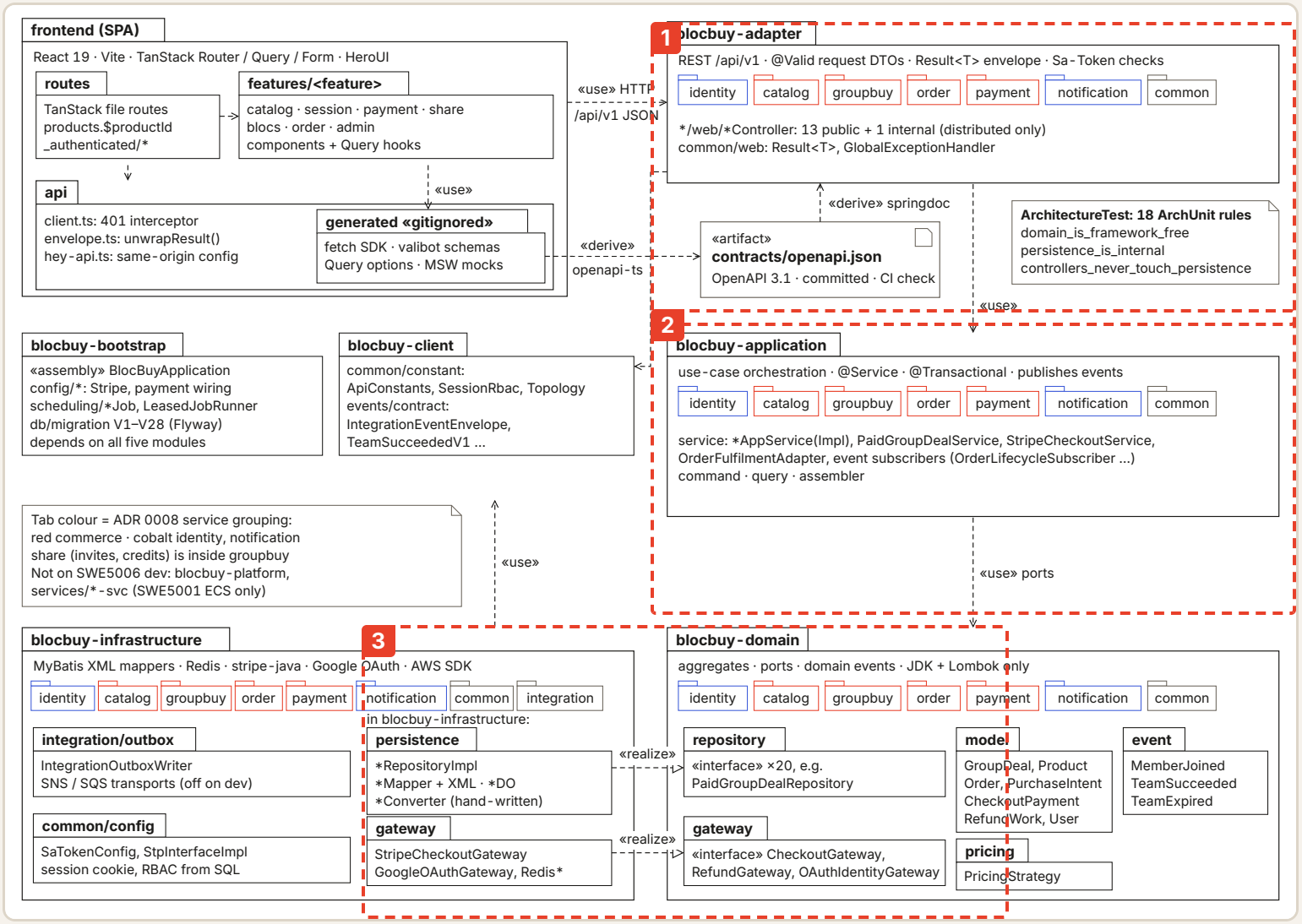
Test first, then refactor: two failing tests reproduced a price-quote bug before the fix; then team admission was refactored into shared checks.

WZ Eric

Logical architecture: COLA layers across six bounded contexts

Detail pages D9-1 to D9-3 (backups)

Report §3.1 · Liu Jin, presented by Eric



DECISIONS THAT SHAPE IT

- Modular monolith: COLA layers, six contexts
  skeleton plan D1, D4
- Framework-free domain; 18 ArchUnit rules
  skeleton plan D1 · no ADR
- Consequences run before the commit
  ADR 0002
- Cross-service events through an outbox
  ADR 0007
- One codebase, two topologies
  ADR 0008
- Router owns URL state; generated client
  frontend ADR 0001, 0003

PLATFORM

Java 21, Spring Boot 4, MyBatis, Sa-Token
PostgreSQL 18, Redis 8, Stripe Checkout
React 19, TanStack, TypeScript
AWS EC2, S3, CloudFront

WZ Eric

Transition strategies from analysis to design

Report §3.2 · Yorik, Eric

Strategy · concern	Object types · use cases	Become in design
Language, framework maintainability	«control», «entity» · all	Spring `@Service` by constructor injection; plain Java domain
Presentation usability	GUI «boundary» · UC1–UC9	TanStack route + component + Query hook
Distribution interoperability	«control» · all	`@RestController` returning `Result`; generated client as stub
Persistence integrity	«entity» · all	Aggregate + repository port, implemented with MyBatis
External interface provider isolation	system «boundary» · UC2–UC6	Gateway port + Stripe / Google adapter; signed webhook
Security security	every «control» · all	Login interceptor; `checkRole(ADMIN)`; CSRF header; rate limits
Scheduling expiry ≤ 1 min	Scheduler actor · UC5, UC6	`@Scheduled` job under a database lease, every minute
Concurrency no oversell	«entity» with capacity · UC4	Row locks in a fixed order + guarded UPDATE + unique indexes
Domain events decoupling	«control» · UC5–UC7	Event record + transactional listener; outbox across services

WZ Eric

Typical changes: class and sequence diagrams

Report §3.2 · transition strategies

CLASS DIAGRAMS: EACH ANALYSIS TYPE BECOMES DESIGN CLASSES

GUI «boundary»	→	Route, component and Query hook
«control»	→	REST controller + application service, called through the generated client
«entity»	→	Aggregate behind a repository port, implemented with MyBatis
External «boundary»	→	Gateway port + adapter

SEQUENCE DIAGRAMS: A PLAIN CALL BECOMES A GUARDED TRANSACTION

Analysis	`control → entity.update()`
Design	`service → repository.lockDeal()` (SELECT … FOR UPDATE) `→ aggregate.joinPaid() → guarded UPDATE` all in one transaction

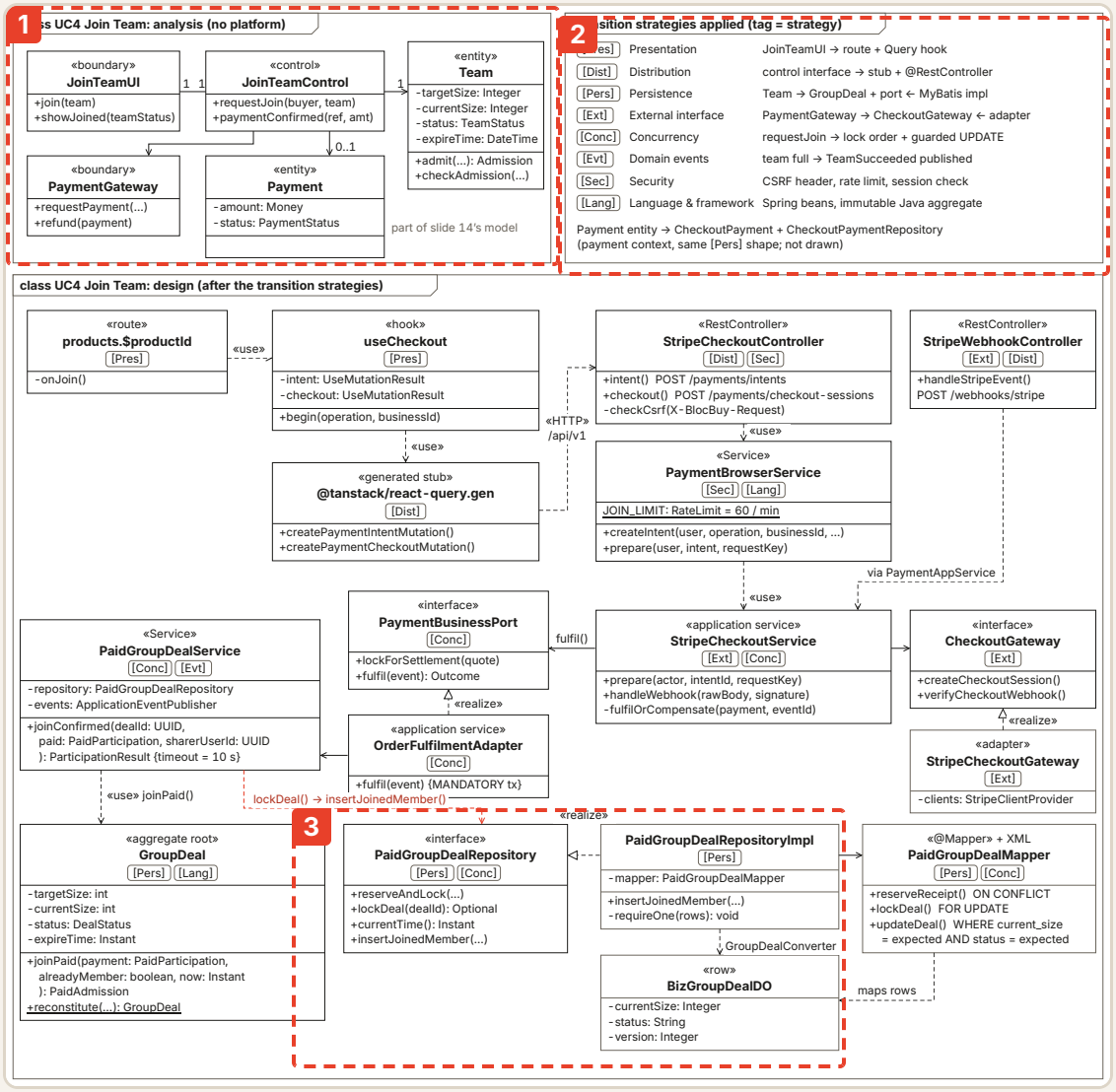
Why: the domain names its needs as interfaces and infrastructure supplies them, so persistence and providers can change without touching the rules.

WZ Eric

One strategy pair: analysis objects become design classes

Detail pages D12-1 to D12-3 (backups)

Report §3.2 · transition strategies

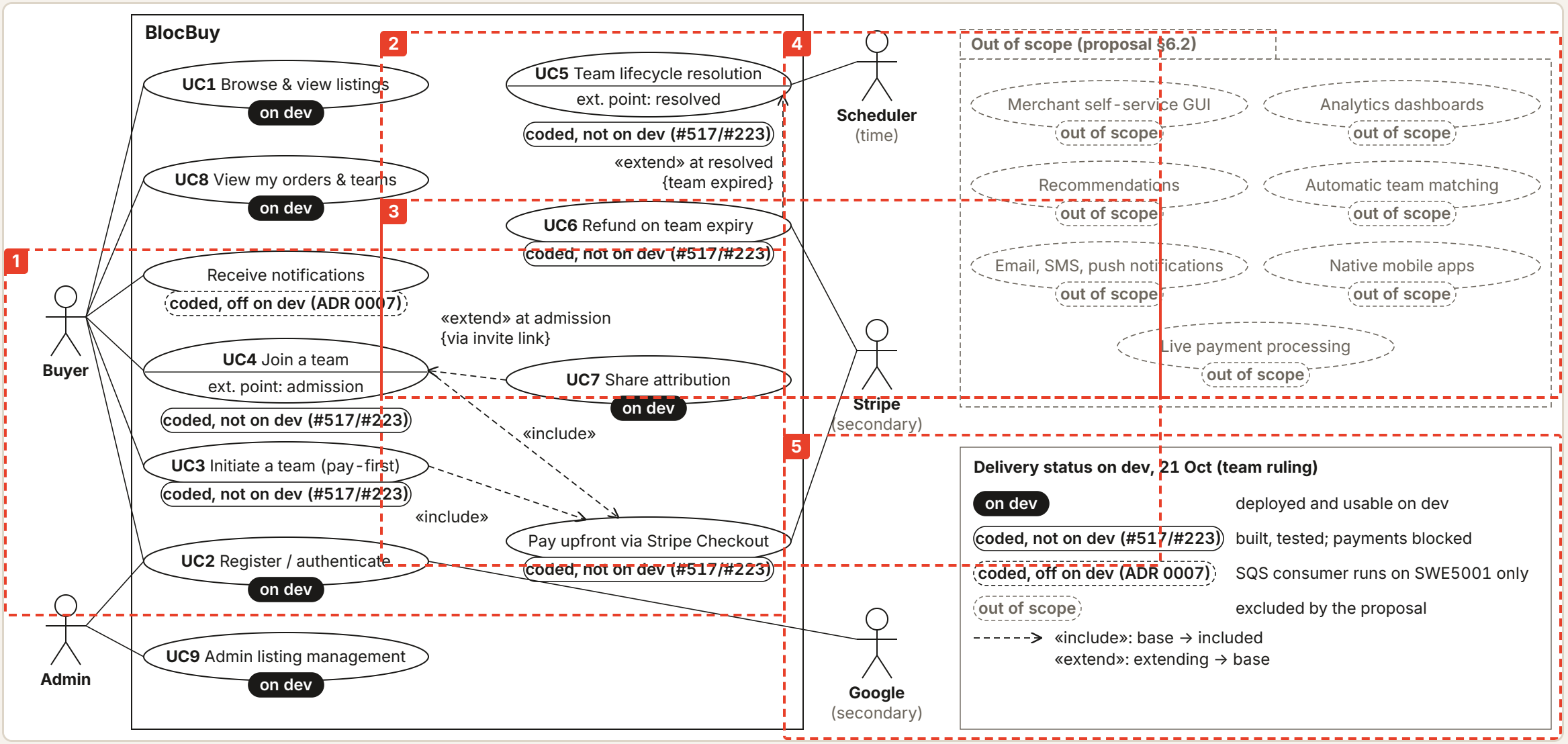


WZ Eric

Use case diagram: in scope, out of scope and status

Detail pages D13-1 to D13-5 (backups)

Report §3.3 · use case diagram and specifications

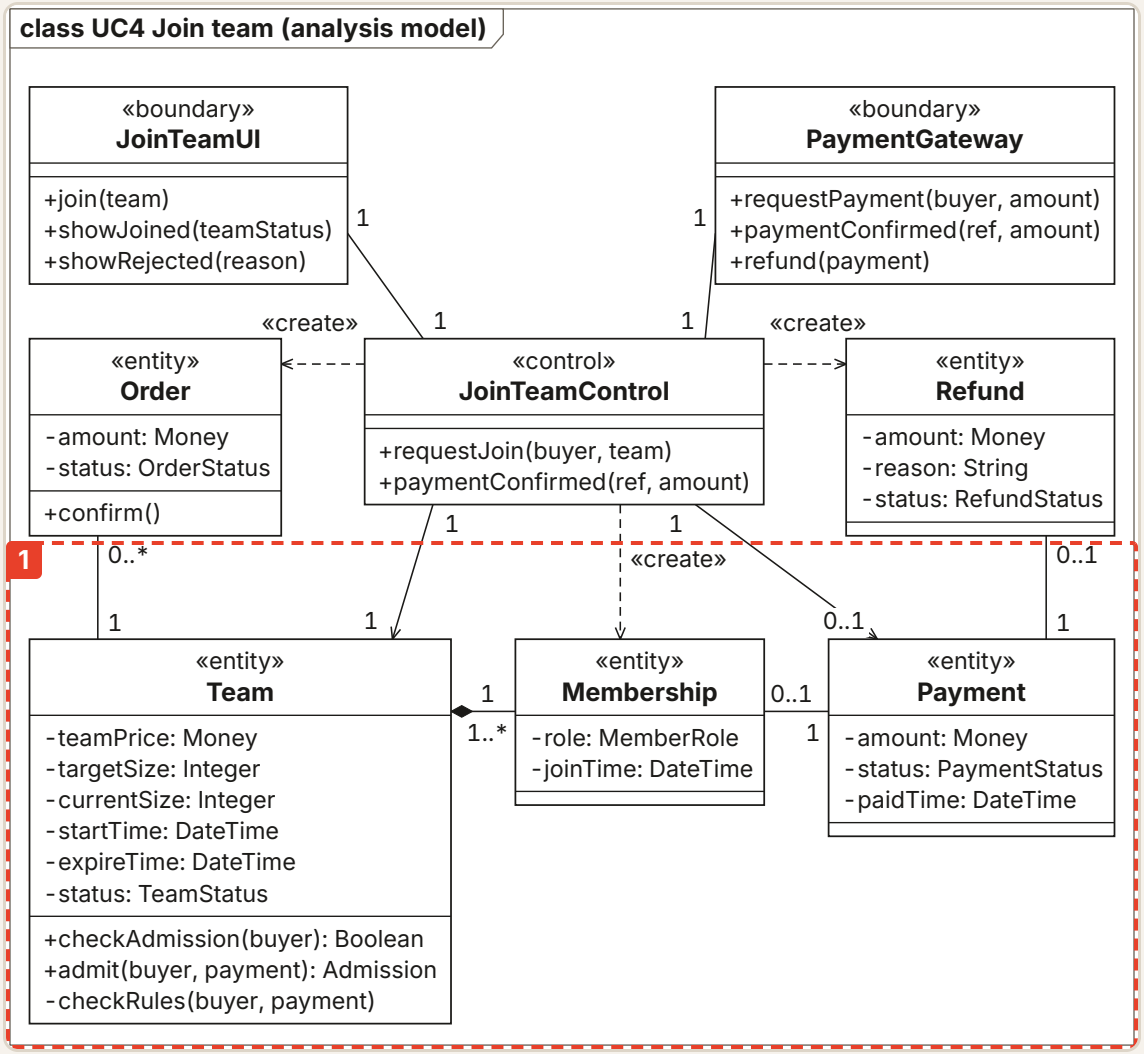


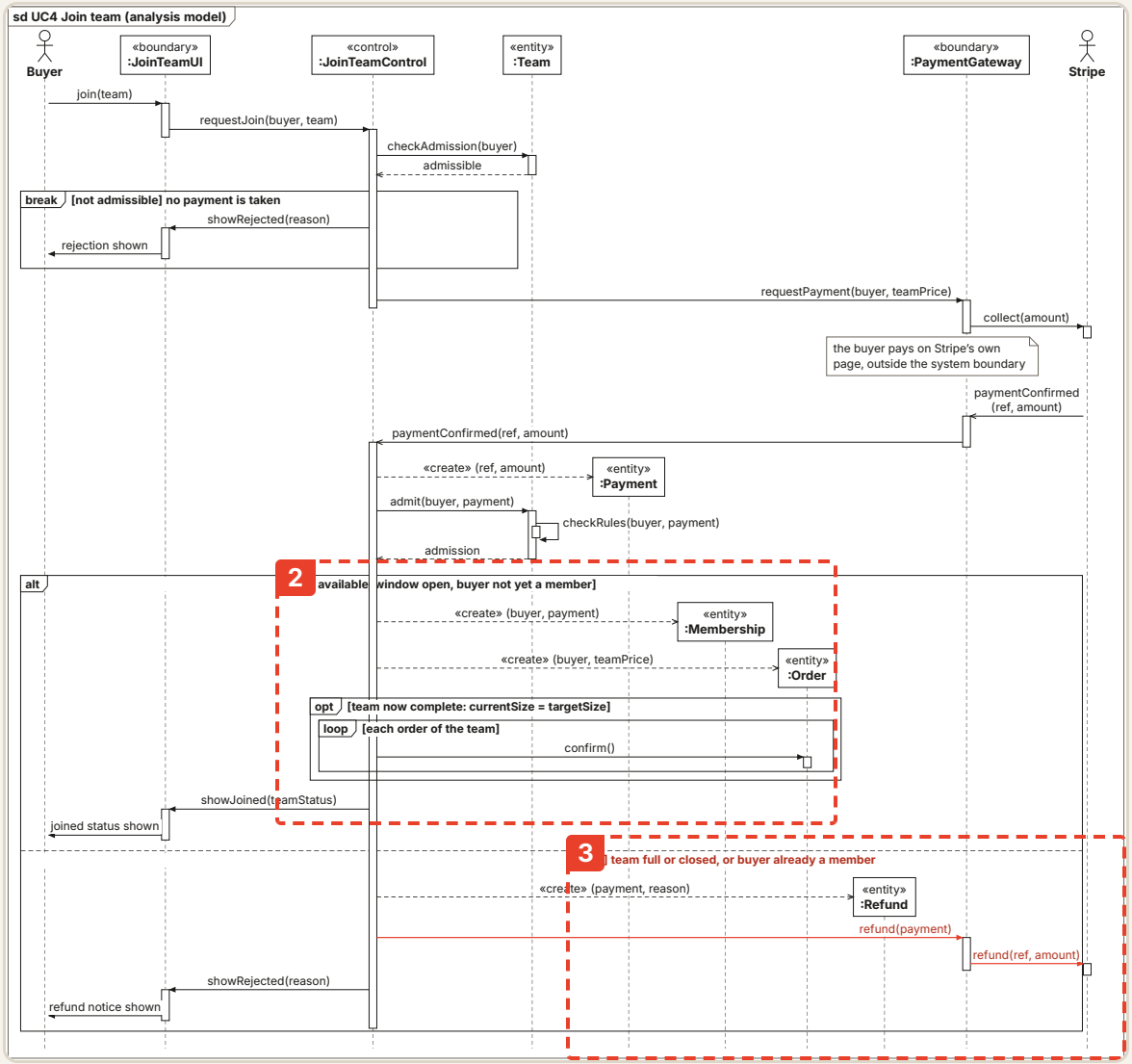
WZ Eric

UC4 join: the analysis model

Detail pages D14-1 to D14-3 (backups)

Report §3.4 · Eric (UC4 analysis model)

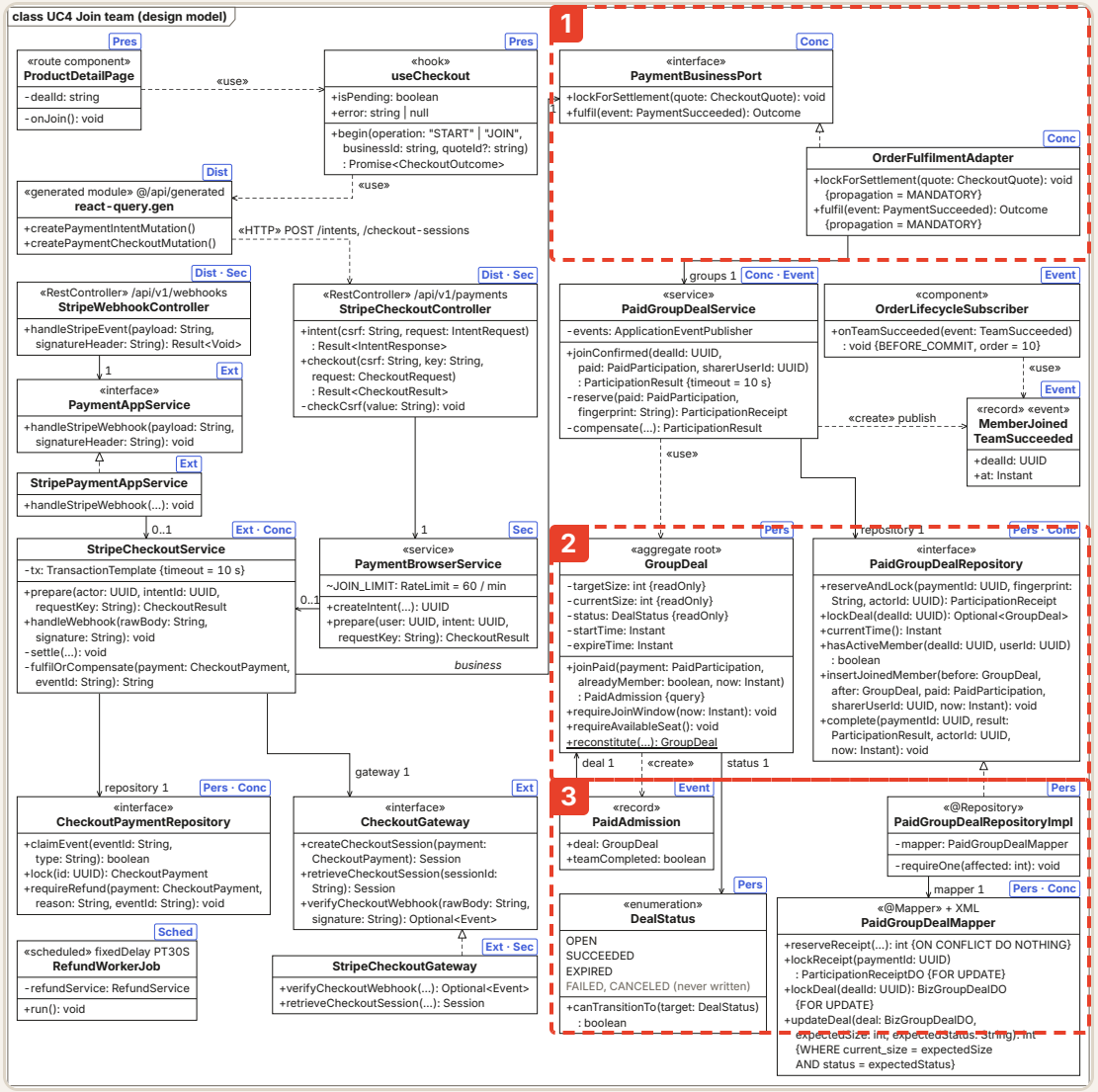




WZ Eric

UC4 join: the design class diagram

Detail pages D15-1 to D15-3 (backups)



Report §3.4 · Eric

STRATEGY TAGS

Pres presentation
Dist distribution
Pers persistence
Ext external interface
Sec security
Conc concurrency
Sched scheduling
Evt domain events
Lang language, framework

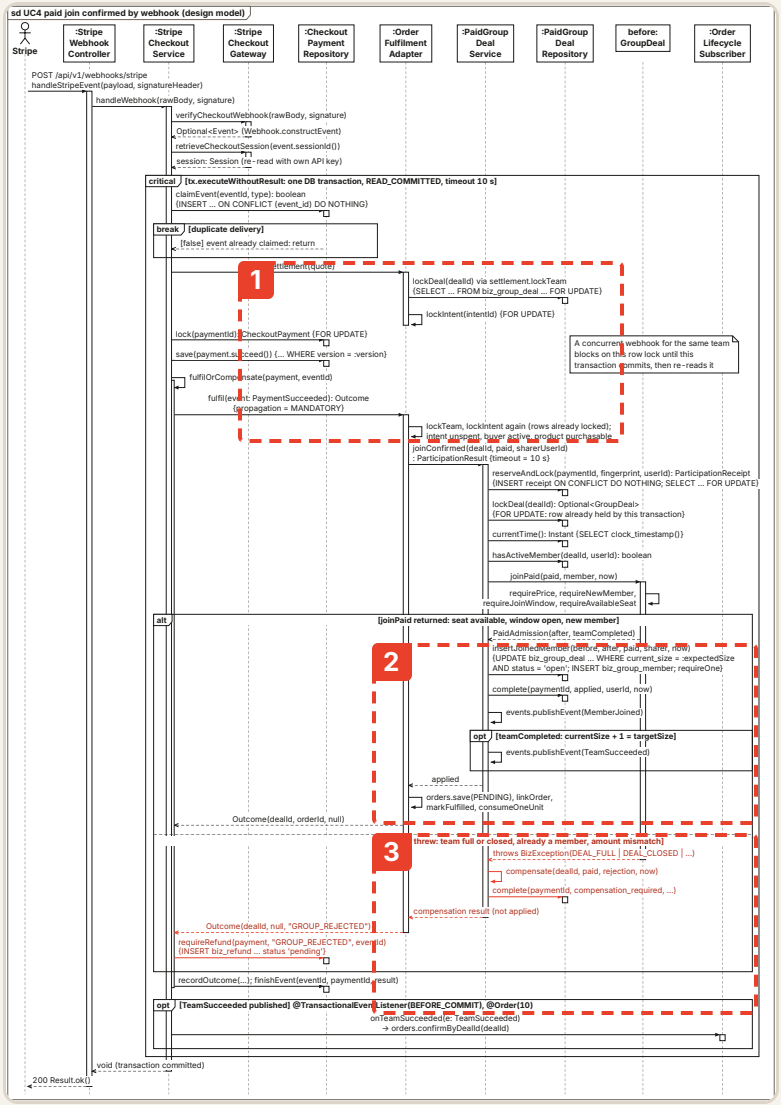
Typed signatures, visibility and multiplicities

WZ Eric

UC4 join: the design sequence, paid join by webhook

Detail pages D16-1 to D16-3 (backups)

Report §3.4 · Eric

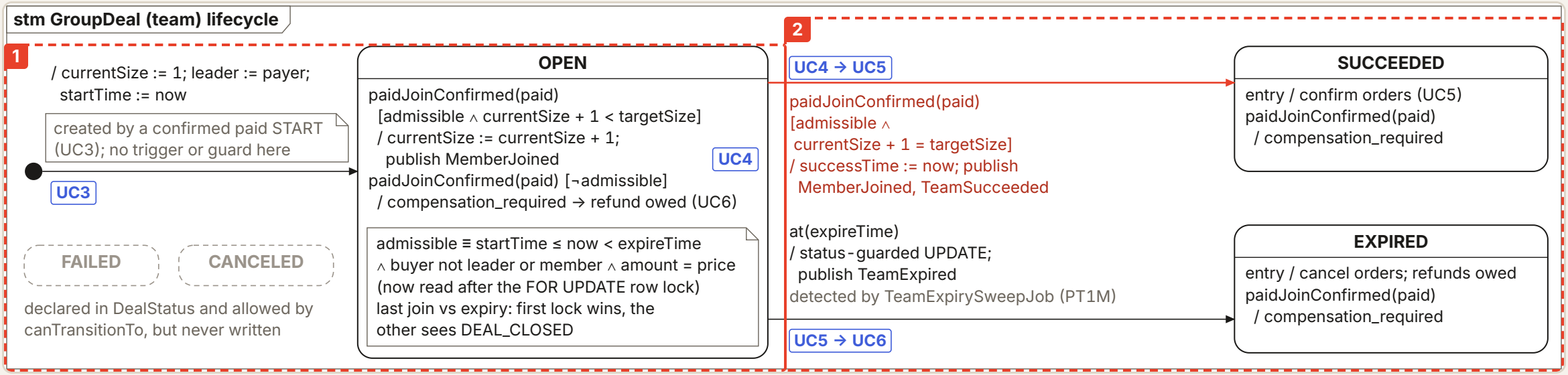


WZ Eric

Team-formation statechart

Detail pages D17-1 to D17-2 (backups)

Report §3.4 · Eric (team statechart)



WHY A STATECHART

Team state gates four use cases: UC4 join, UC5 resolution, UC6 refund and UC8 my teams.

IN CODE

`DealStatus` and the guards of `GroupDeal`

WZ Eric

UC3 start a team: Factory Method

Detail pages D18-1 to D18-3 (backups)

Report §3.5 · Eric (UC3)

WHERE IT SHOWS

The «create» of a team: the service once named the concrete creator itself

CAUSE FOR REDESIGN

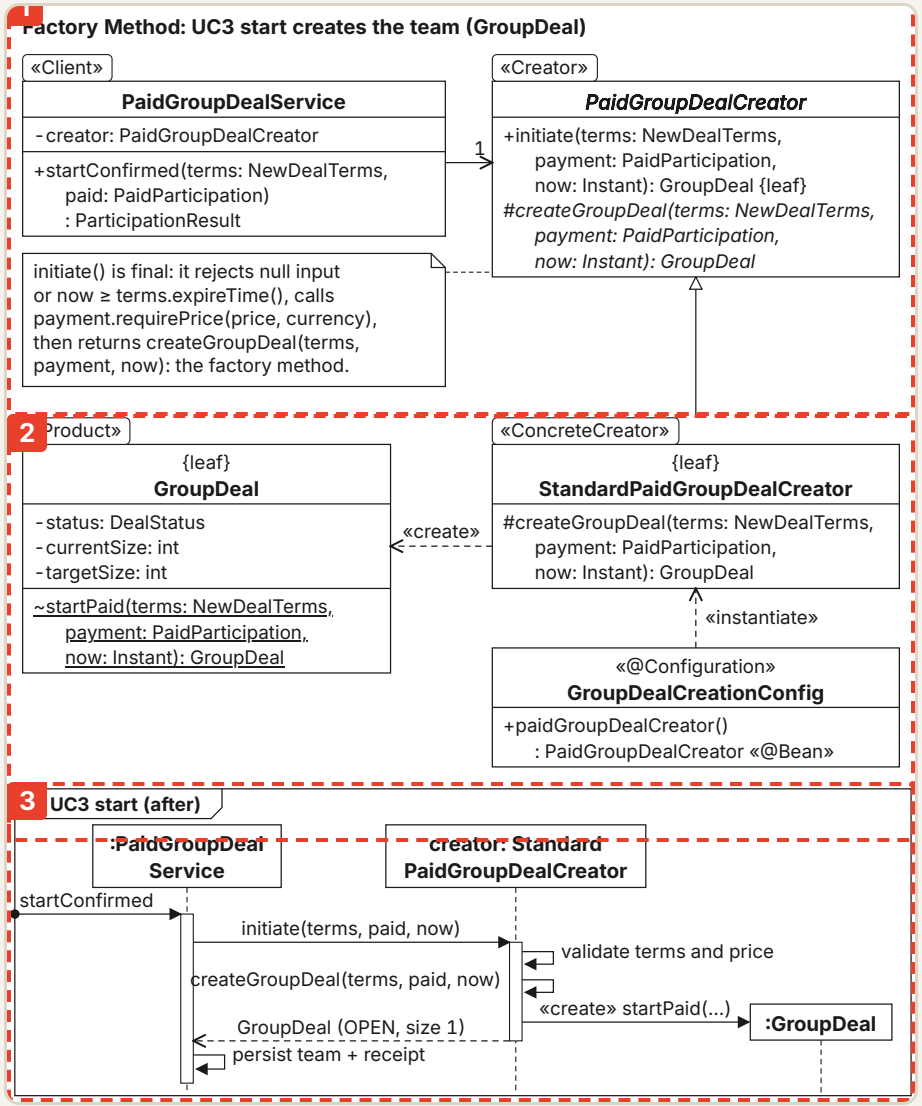
Creating an object by specifying its class explicitly

WHAT VARIES

The class of the team that is created

PATTERN

Factory Method



YQ Yorik

UC5 to UC6 team outcome: Observer

Detail pages D19-1 to D19-2 (backups)

Report §3.5 · Yorik (UC6)

WHERE IT SHOWS

In the expiry sequence: it called the order and refund code inline

CAUSE FOR REDESIGN

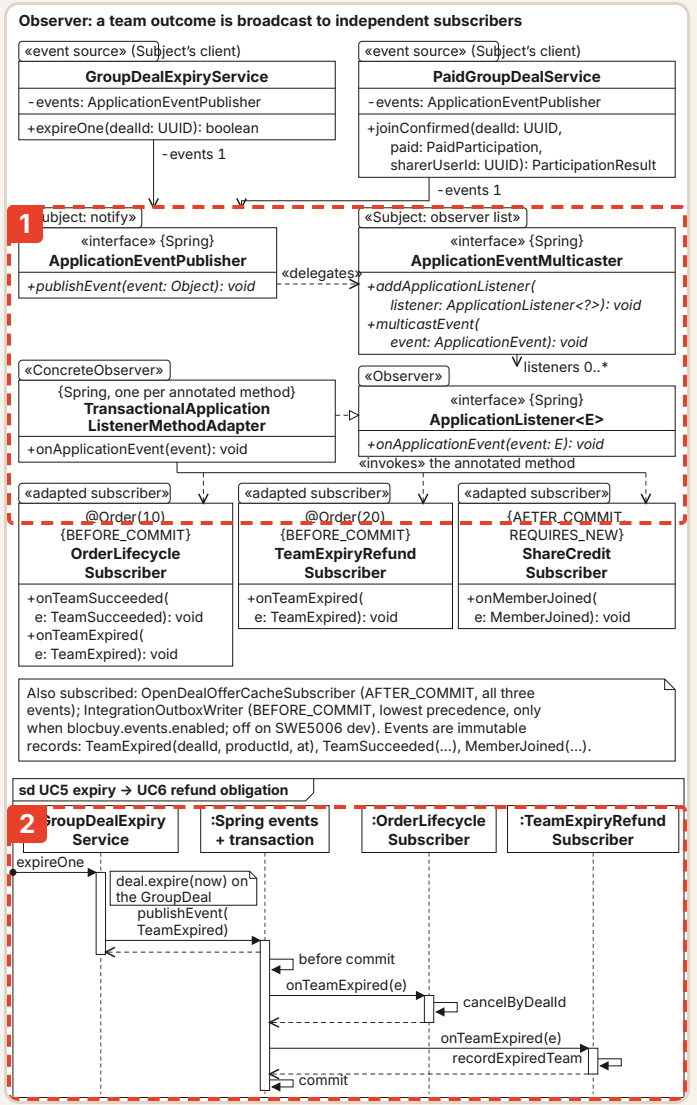
Tight coupling: more and more objects depend on a team's outcome

WHAT VARIES

The objects that depend on a team's outcome

PATTERN

Observer



Notifications: Template Method for outbox transports

Detail pages D20-1 to D20-2 (backups)

Report §3.5 · Pan (notifications)

WHERE IT SHOWS

In the class diagram: the SQS and SNS transports would each copy the batch loop

CAUSE FOR REDESIGN

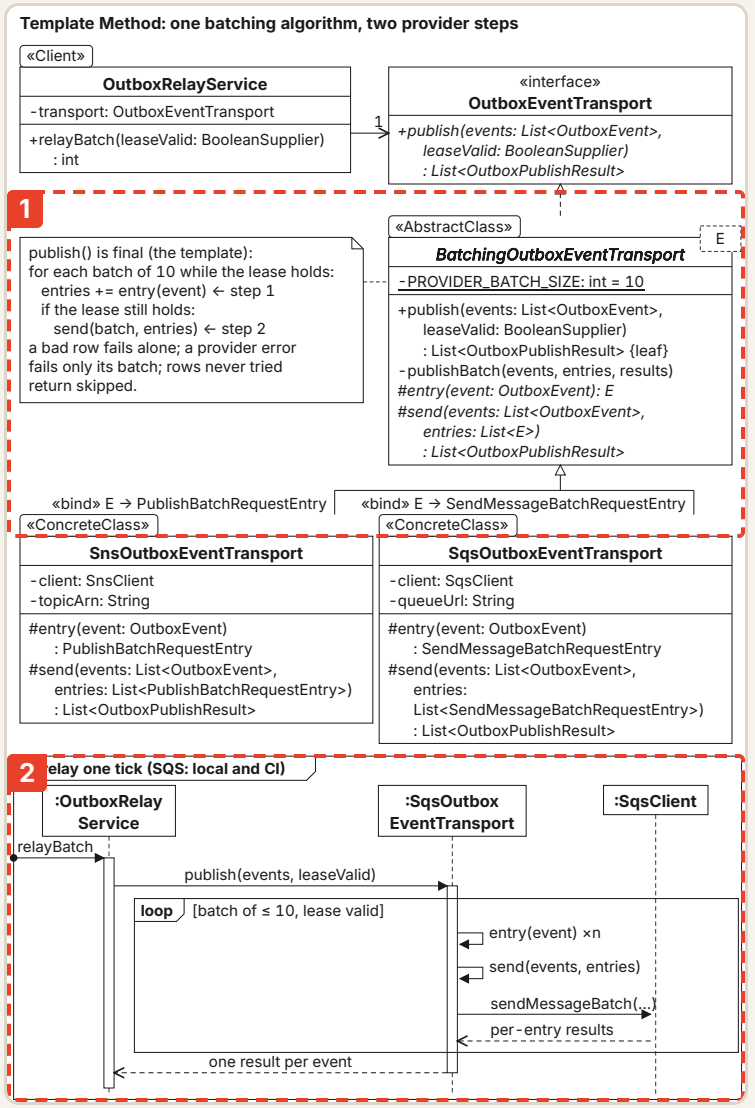
Algorithmic dependencies: the same steps, two providers

WHAT VARIES

Steps of an algorithm

PATTERN

Template Method



UC1 pricing: Strategy

Detail pages D21-1 to D21-2 (backups)

Report §3.5 · Lim (UC1)

WHERE IT SHOWS

In the class diagram: browse and the team quote each switch on the pricing mode

CAUSE FOR REDESIGN

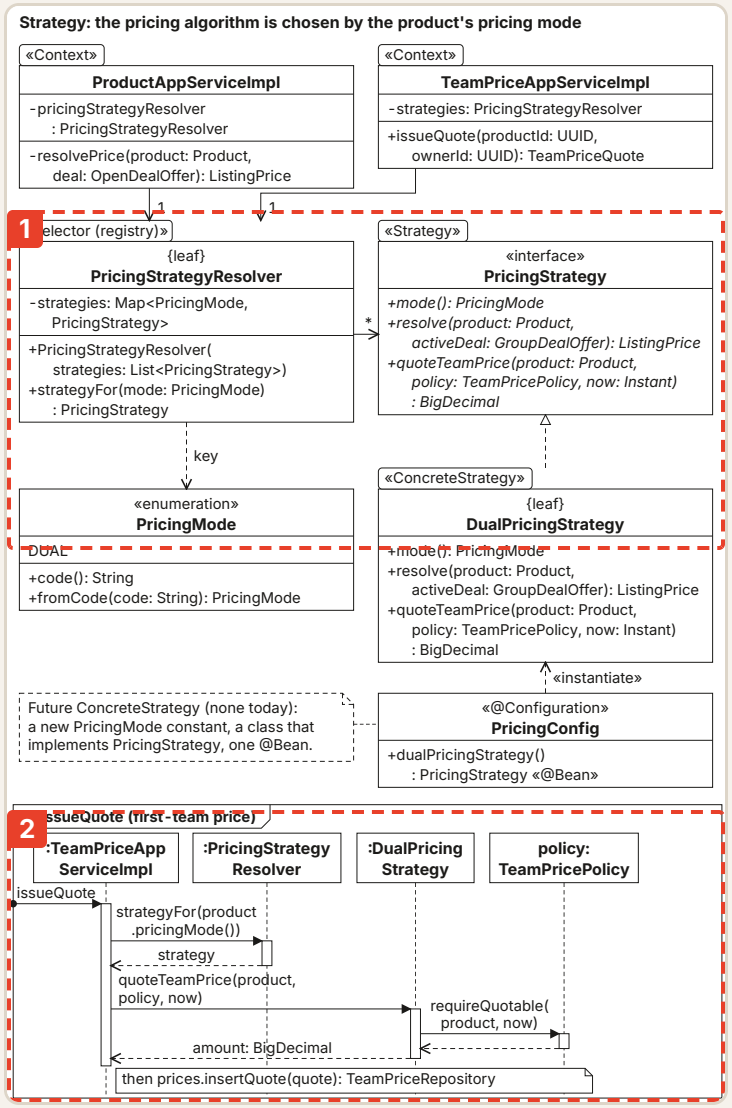
Algorithmic dependencies: the arithmetic sits inline in both

WHAT VARIES

An algorithm: how a price is computed

PATTERN

Strategy



LJ Liu Jin

Design problem: listing rules that depend on its state

Detail pages D22-1 to D22-2 (backups)

Report §3.5 · Liu Jin (UC9)

PROBLEM (UC9)

Publish, delist, revise and delete are legal in different states of a listing

WHY REDESIGN

Hard to maintain: one rule in four places

Hard to extend: a new state edits every switch

Hard to reuse: the rule cannot be tested apart from Product

WHAT VARIES

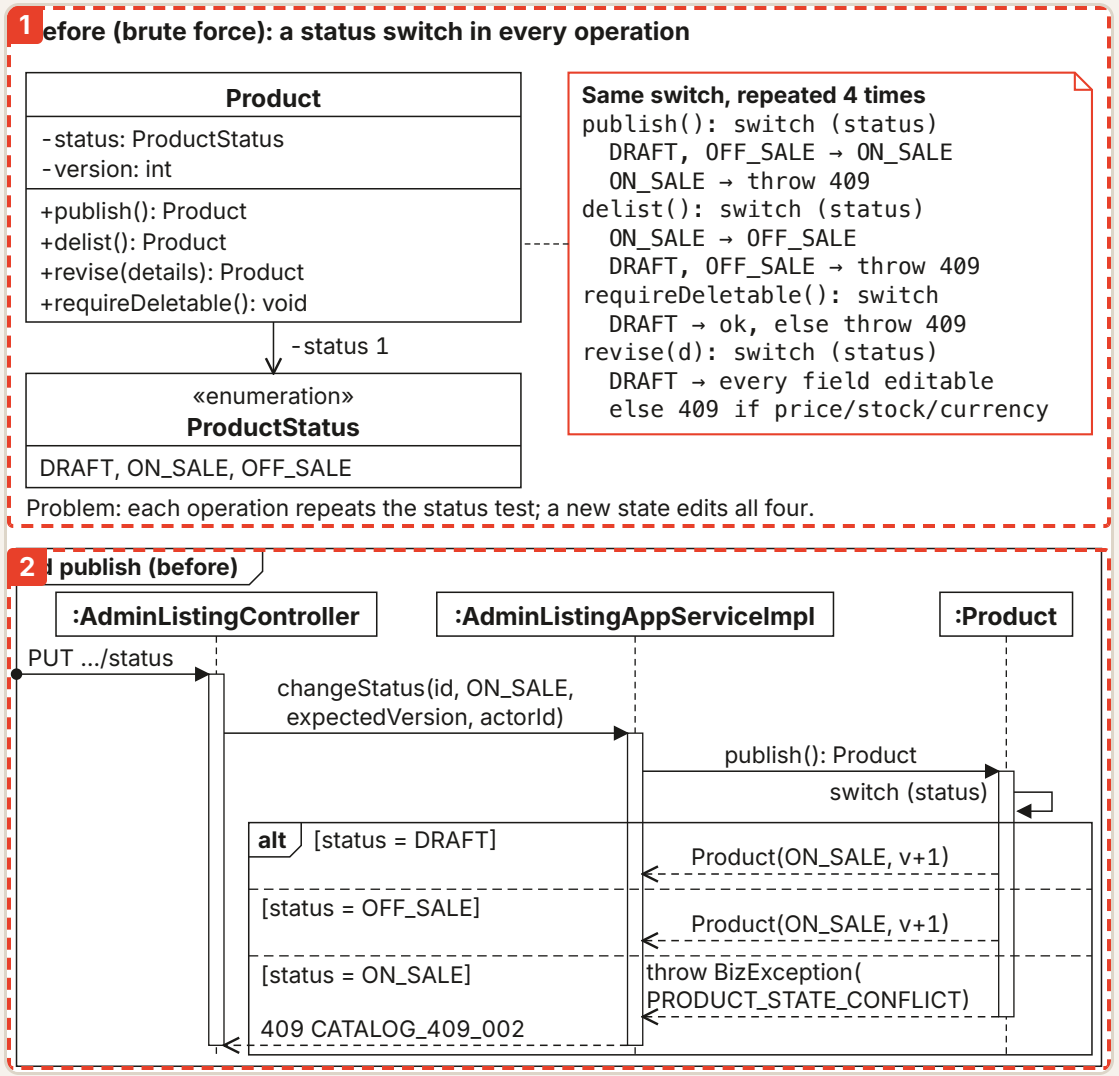
The states of an object

CANDIDATES

State

Strategy: nobody chooses; the state decides

Transition table, as on the team



LJ Liu Jin

After: the State pattern, and why the team uses a table

Detail pages D23-1 to D23-2 (backups)

Report §3.5 · Liu Jin (UC9)

PATTERN

State: the listing delegates to its state

IMPLEMENTATION DECISIONS

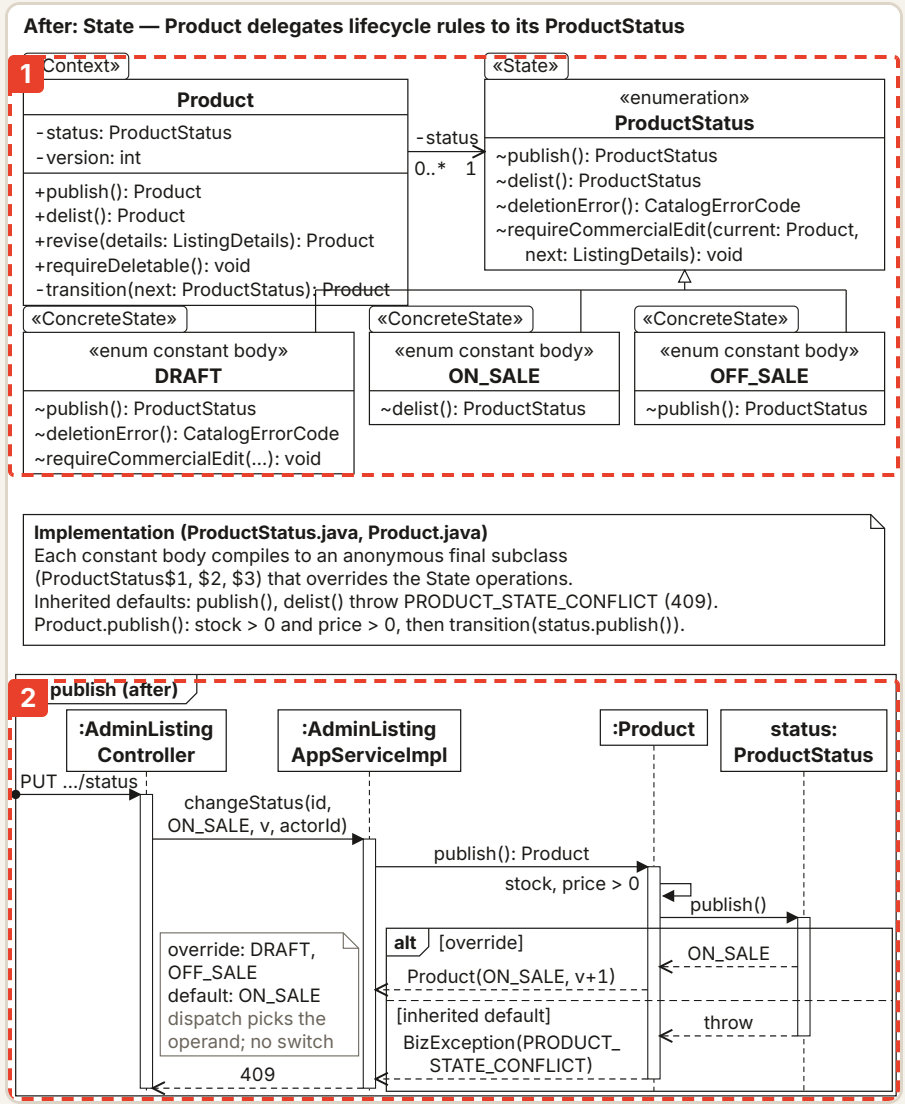
The state returns its successor; the listing applies it

Moves a state does not allow refuse with 409

Each move makes a new revision; a stale write gets 412

WHY THE TEAM USES A TABLE INSTEAD

Only OPEN has behaviour; SUCCEEDED and EXPIRED are final, so a transition table suffices

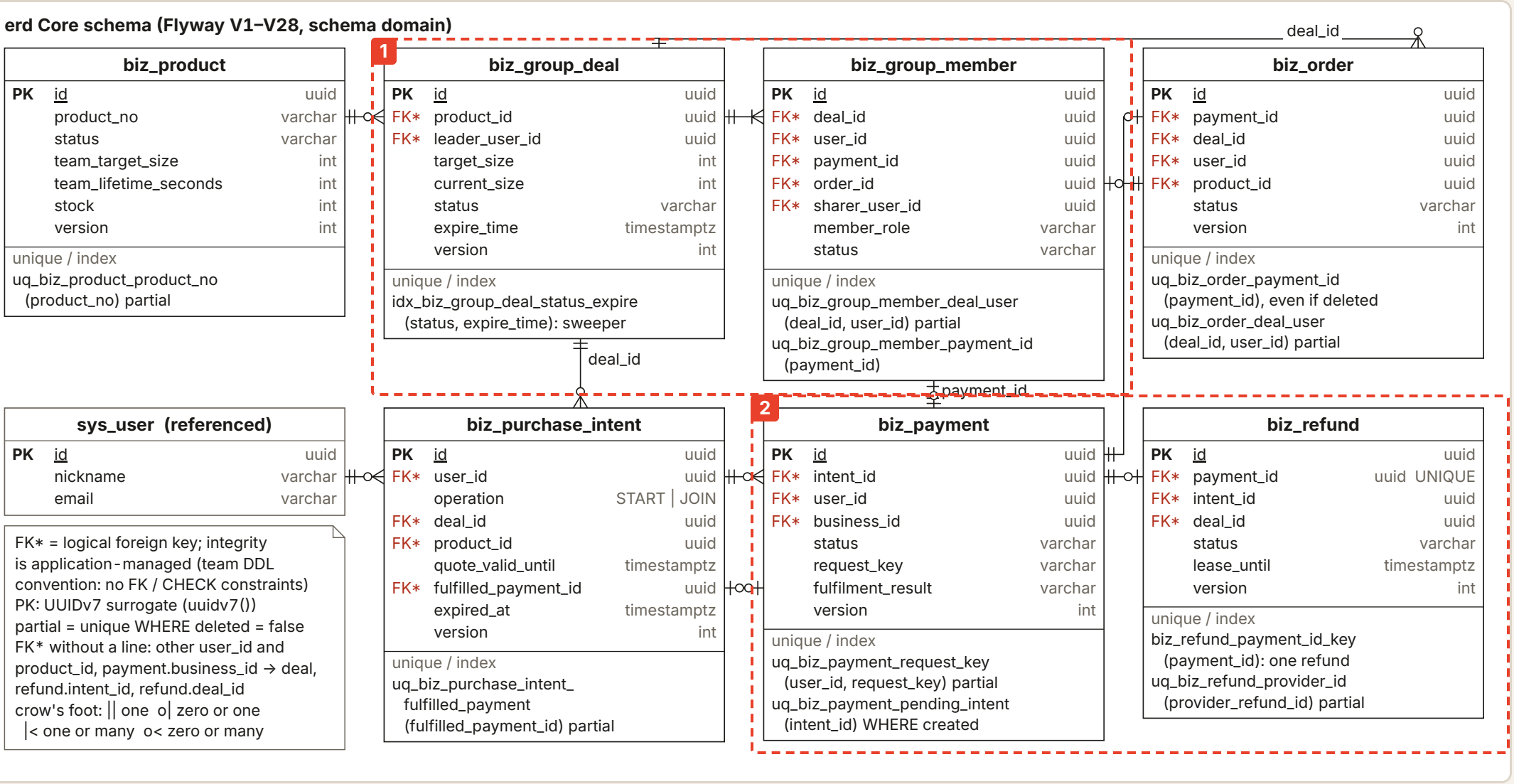


LJ Liu Jin

Database schema: core entities and mapping decisions

Detail pages D24-1 to D24-2 (backups)

Report §3.6 · Liu Jin



LJ Liu Jin

Testing on the Agile Testing Quadrants

Detail pages D25-1 to D25-4 (backups)

Report [FILL: § (#463)] · types of testing

Q2 business-facing · supporting the team — automated

Story acceptance system

End-to-end system

Q3 business-facing · critiquing the product — manual

Exploratory, acceptance system

Not done: outside users

Q1 technology-facing · supporting the team — automated

Unit, component unit

Integration integration

Contract, ArchUnit integration

Q4 technology-facing · critiquing the product — tools

Performance, load system

Security scanning types

LJ Liu Jin

Scripted performance test: tool, scenario, target

Report [FILL: § (#463)] · performance test

THE SCRIPTED TEST

Tool	k6 1.6.1, image pinned by digest
Target	**SWE5001 ECS environment, same code**: CloudFront → ALB → Fargate Never the shared SWE5006 dev host, which also runs its database
Load	Open arrival rate 10 → 200 requests/s Ten 60 s steps; 100 VUs preallocated, 400 max
Mix	7 listing pages : 3 listing details, anonymous No think time; 10 s timeout
Pass, per step	p95 < 500 ms Errors < 1% Zero dropped requests
Result	In the system demo (slide 36) [FILL: 5.3 pre-run label and date]

PROPOSAL TARGET (§6.3)

Interactive operations ≤ 2 s with up to 100 concurrent users

Little's law $N = X \cdot (R + Z)$: 100 users, 0.5 s think, ~30 ms response → **X ≈ 190 requests/s**

Inside the 200 driven, with a p95 gate four times stricter than 2 s

The 0.5 s think time is our assumption

EXPIRY ≤ 1 MIN AFTER THE WINDOW

Measured **locally** under 100-user load (30 Sep): 30 teams, worst lag 58.2 s

On ECS: [FILL: V4 expiry burst, or "local only"]

PS Pan

Source code: one monorepo, Branch by Feature, reviewed changes

Report [FILL: § (#463)] · source control strategy

ONE MONOREPO

`backend/`	Maven COLA modules; SWE5001 services/
`frontend/`	React SPA
`contracts/`	openapi.json, the API contract
`deployment/`	Compose, SSM bundle, deploy.sh
`infra/`	Terraform: dev frontend, SWE5001
`perf/`	k6 load tests
`docs/`	ADRs, sprint logs
`.github/`	workflows, CODEOWNERS, Dependabot

BRANCHING: BRANCH BY FEATURE

One short branch per story

A pull request, then a squash merge onto the trunk, **dev**

Linear history since 12 Sep

Up to date with dev before merging, instead of merging the trunk daily

WHO MAY CHANGE IT

GitHub: a **code owner** approves; six checks green

Owners: backend Liu Jin, frontend Lim, pipelines Pan

AWS: IAM Identity Center, one-hour sessions

[VERIFY: MFA enforced]

CI: one OIDC role per job, no AWS keys in GitHub

Exception: two IAM users for SSM port forwarding, one with an access key

PS Pan

Artefacts: each one traceable to a commit

Report [FILL: § (#463)] · artefacts

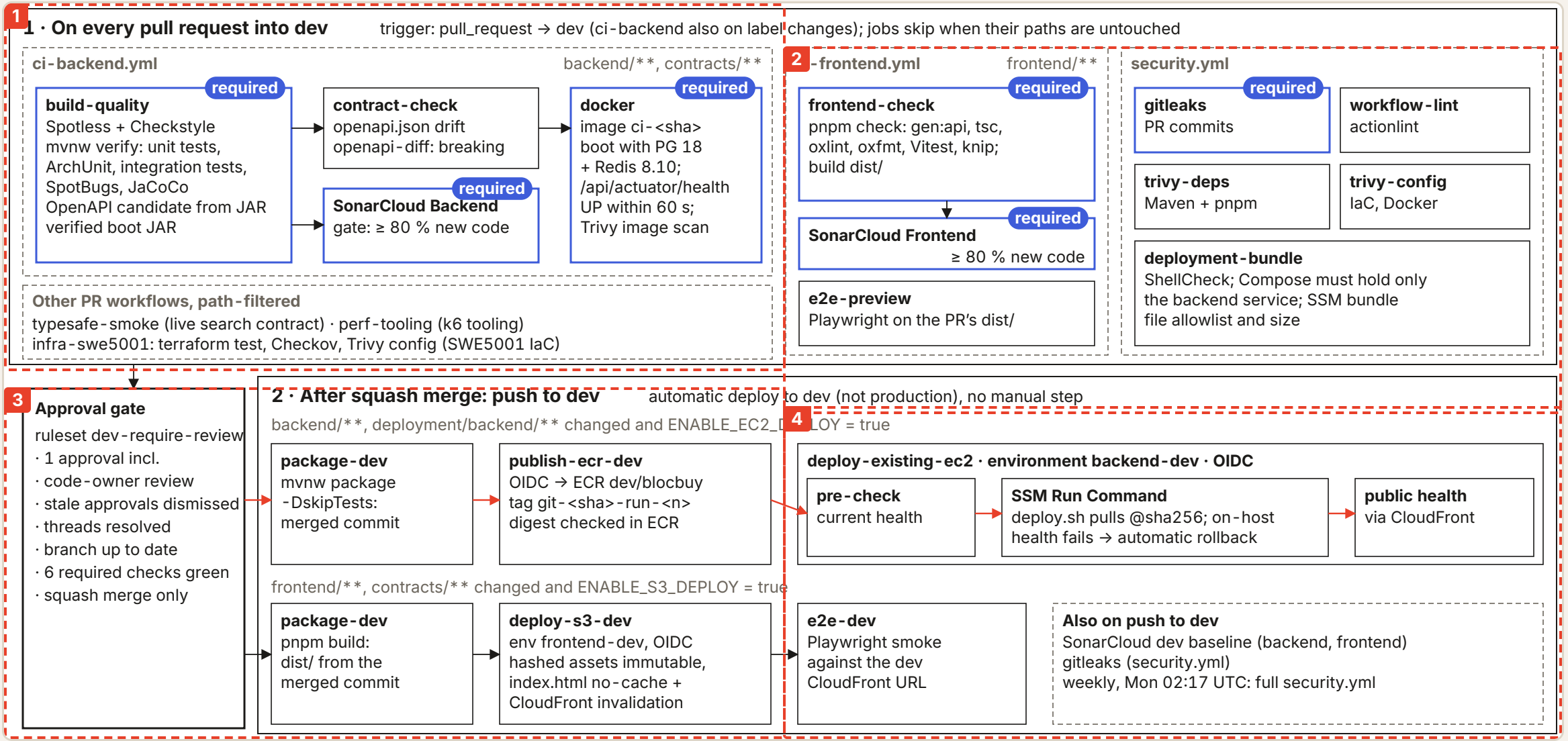
Artefact	Identity	Stored in
Backend image	`git-<sha>-run-<n>`, deployed by its `@sha256` digest	ECR, immutable tags, newest 30 kept
API contract	`openapi.json` committed; drift fails CI	Repository
TypeScript client	Generated from the contract, gitignored	Build only
Frontend bundle	Content-hashed files, immutable cache	S3 behind CloudFront
Database schema	Flyway migrations V1–V28	Repository
Infrastructure	Terraform plan saved, applied by an operator; CI never applies	S3 state, DynamoDB lock
SBOM	Not produced	—

PS Pan

Continuous integration: pull-request jobs and post-merge jobs

Detail pages D29-1 to D29-4 (backups)

Report [FILL: § (#463)] · continuous integration



PS Pan

Security controls by pipeline stage, and what each one catches

Report [FILL: § (#463)] · pipeline security

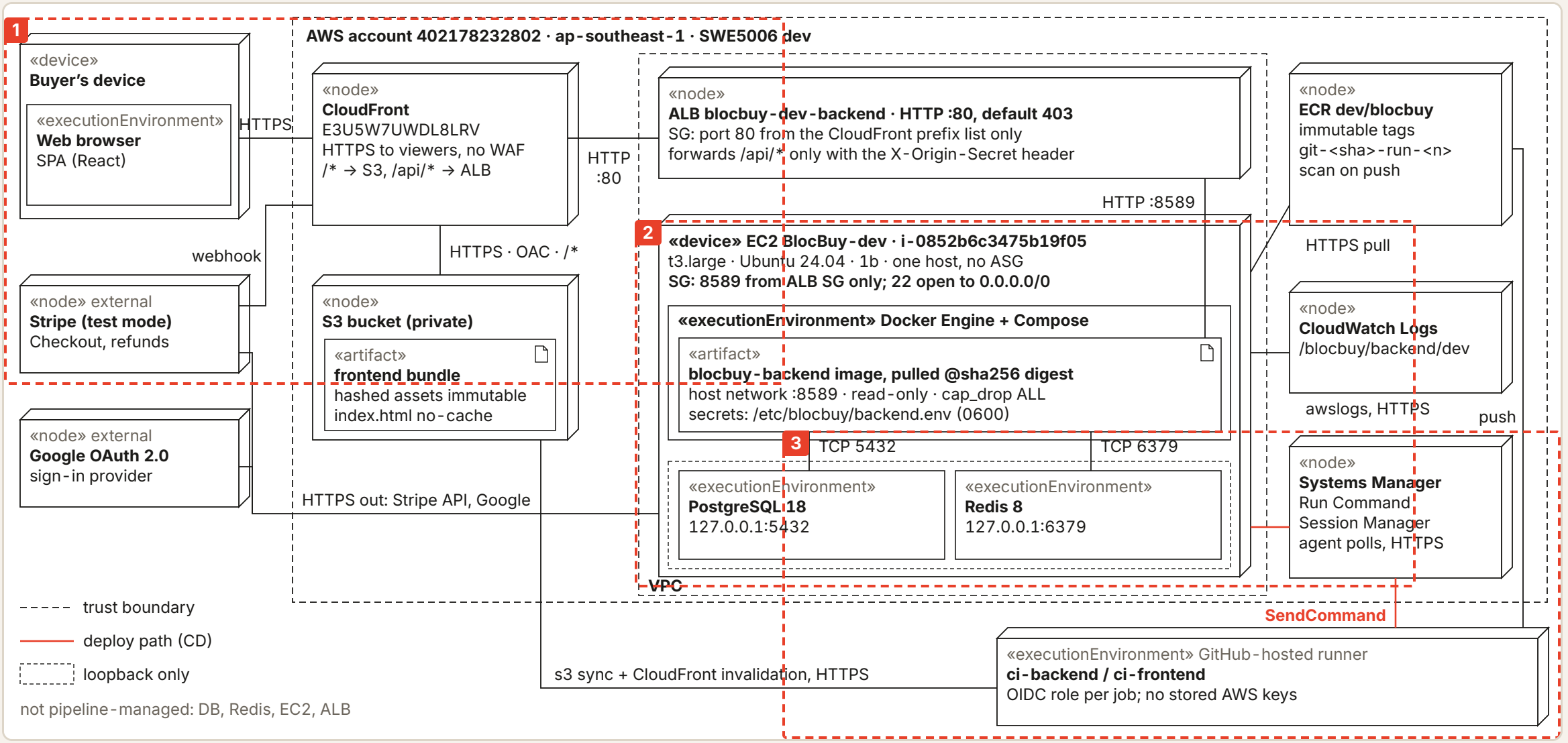
Stage	Control	Catches	Gap
Pre-commit	Lefthook: format, lint, type-check (frontend)	Style and type errors	—
Commit · PR	**gitleaks** required: the PR's commits, full history weekly Code-owner review actionlint	Secrets, unsafe workflows	—
Build	SAST: **SonarCloud** required, SpotBugs SCA: Trivy deps, Dependabot daily	Code flaws, vulnerable libraries	CodeQL off: code scanning unavailable
Test	**Trivy image** required (HIGH, CRITICAL) Trivy config, deployment-bundle rules	Image CVEs, IaC misconfiguration	No DAST against the running site
Deploy	OIDC role per job Image by digest Read-only container, all capabilities dropped	Stolen keys, swapped images	Rebuilt after merge, not rescanned

Types tested (slide 25, Q4): static analysis, dependencies, image and IaC configuration, secrets
Every third-party action is pinned to a commit SHA

Delivery: one environment, the code owner approves, the deploy verifies itself

Detail pages D31-1 to D31-3 (backups)

Report [FILL: § (#463)] · continuous delivery



PS Pan

Technical findings: three issues, their root causes and fixes

Report [FILL: § (#463)] · technical findings

Finding	Root cause	Fix
First dev deploys failed (14–17 Sep)	The hand-built environment drifted from its design Old OIDC subject format, a placeholder instance ID, port 8589 open wide	Fixed on dev by hand The prod stack renders each value in Terraform, so a plan shows it No deploy failed after 17 Sep
build-quality red for two days (19–20 Oct)	Dependabot upgraded the SpotBugs engine without its Maven plugin	Both now upgrade together A POM policy test guards it
Weekly full-history gitleaks failing (since 21 Sep)	20 matches were file digests in a contract baseline, not secrets	Each digest re-hashed, then fingerprinted exactly 4 badge-token findings still open

Still open: the Stripe webhook never confirms a payment on dev, so a green pipeline is not yet a working paid flow.

PS Pan

Live: one change from pull request to dev

Step	What you see	Time
P1	The pull request: one frontend change, six required checks green	60 s
P2	Lim approves as the frontend code owner	30 s
P3	Squash merge: one commit on dev	20 s
P4	Delivery runs; the backend PR and deploy, triggered beforehand	90 s
P5	Verified: e2e-dev green, the change live on dev	90 s
P6	Merge to live in minutes; hand-over	40 s

WHY THE BACKEND IS PRE-RUN

Backend PR checks take about 16 min, longer than the segment, so that run was triggered before the talk and we walk through it finished.

BACKUP VIDEO

Recorded on dev by 24 Oct, one clip per step

In the cloud and on a USB drive

`[FILL: link]`

YQ Yorik

System demo: two buyers, one bloc, and the qualities it shows

Step	What you see	Use cases	Quality attribute (slide 3)	Runs on
D1	Leader signs in, starts a bloc, pays in Stripe Sandbox, copies the invite	UC2 UC1 UC3 UC7	Security: only a signed webhook confirms payment	dev, if GO
D2	Joiner opens the invite, joins and pays: 2 of 2, the team succeeds	UC4 UC5 UC7	Correctness: guards re-checked after payment	dev, if GO
D3	My blocs and orders, filtered by the URL	UC8	Functionality	dev
D4	The bloc started at 16:05 expired and refunded itself	UC5 UC6	Unattended expiry and refund; ≤ 1 min measured locally (slide 36)	dev, if GO
D5	No session, a buyer on admin, a write without the marker	—	Security: 401, 403, 400, HttpOnly	dev
D6	Twenty paid buyers race for one seat	UC4	Correctness under concurrency	local test
36	Scripted k6 load, pre-run	UC1	Performance: ≤ 2 s at 100 users	SWE5001 ECS
38	Back to the §4 pipeline run	—	Maintainability	dev

“dev, if GO”: live on dev after the 23 Oct go/no-go, otherwise its 24 Oct recording; a local result is never shown as dev

YQ Yorik

Live: two buyers, one bloc

Step	What you see	Time
D1	Start a bloc and share the invite	120 s
D2	A friend joins and pays: the team succeeds	120 s
D3	My blocs and my orders	75 s
D4	The 16:05 bloc: expired, order cancelled, refunded	90 s
D5	Security: 401, 403, 400, HttpOnly cookie	60 s
D6	Last-seat race: the test, in the IDE	45 s

TWO BUYERS, ONE SCREEN

Two Chrome profiles, Leader and Joiner, both signed in with Google

Stripe Sandbox: a test card, no money moves

STARTED BEFORE THE TALK

A team of two on `US34-04` at 16:05: its 40-minute window, the shortest allowed, ends ≈ 16:45.

BACKUP VIDEO

Recorded on dev by 24 Oct, one clip per step

In the cloud and on a USB drive

`[FILL: link]`

YQ Yorik

Performance against the target: the scripted k6 run

K6 PRE-RUN · SWE5001 ECS ENVIRONMENT, SAME CODE · NOT THE DEV HOST

Requests/s	p95 (ms)	Errors	Dropped	Step
10	—	—	—	—
20	—	—	—	—
40	—	—	—	—
60	—	—	—	—
80	—	—	—	—
100	—	—	—	—
125	—	—	—	—
150	—	—	—	—
175	—	—	—	—
200 ●	—	—	—	—

● at or above ≈ 190 requests/s: the proposal's 100 users, 0.5 s think time assumed (slide 26) [FILL: results from the 24 Oct pre-run]

TARGET (PROPOSAL §6.3)

≤ 2 s with up to 100 concurrent users

Our gate per step:
p95 < 500 ms
Errors < 1 %
No dropped requests

Verdict: [FILL: pass or fail per gate]

EXPIRY ≤ 1 MIN

Worst lag 58.2 s for 30 teams under 100-user load (**local**, 30 Sep)

NOT CLAIMED

Authenticated paths under load
Anything about the dev host
Scaling (one task)

YQ Yorik

Scalability: a stated trade-off, not a claim

DEV, THE SYSTEM YOU SAW

Compute	One EC2 t3.large, no Auto Scaling group
Data	PostgreSQL 18 and Redis 8 on the same host
Effect	One point of failure; a deploy restarts the API

WHY, DELIBERATELY

- One complete system, always on, at low cost
- The proposal lists no scalability target
- One codebase, two topologies

Scale path, not claimed here: the same code on ECS in our SWE5001 topology, commerce 2–6 tasks on requests per target, notification 1–4 on queue backlog.

YQ Yorik

Maintainability: the pipeline you just saw, and the seams in the code

SHOWN IN §4 (SLIDE 33)

One change, live on dev

Six required checks, a code-owner review, an automatic deploy and a smoke test

18 ArchUnit rules hold the module boundaries on every build

PRIMARY DESIGN DRIVER: EXTENSIBILITY

Change	Added as	Pattern
New pricing mode	One more `PricingStrategy` and its `PricingMode` constant	Strategy slide 21
New consequence of a team's outcome	One more subscriber to `TeamSucceeded` / `TeamExpired`	Observer slide 19
New payment provider	One more adapter behind `CheckoutGateway`, `RefundGateway`	Gateway port (external-interface strategy) slide 12

Existing rules stay unedited: each change adds classes; a pricing mode also adds one enum constant.

Thank you

Questions?

github.com/PPsyrius/iss-groupbuy-team8 d1b3np88z657cp.cloudfront.net

Backup slides: index

B1 **Agile artefacts appendix**
Where are the backlog, stories, sprint plans, reviews and retros?

B2 **Sprint 2 and 3 burndowns**
How did earlier sprints burn down?

B3 **ADR index**
Which decisions are recorded, and where?

B4 **Database schema, all tables**
Which tables does each context own?

B5 **Expiry and refund sequence**
How does an expired team become refunds?

B6 **UC6 refund recovery: candidates and decisions**
How do you never refund twice?

B7 **Test inventory**
How many tests of each kind, and where do they run?

B8 **Workflows, required checks and timings**
What runs when, how long, and how often is it green?

B9 **Known gaps and limitations**
What is not done yet in the pipeline and on dev?

B10 **SWE5001 production topology**
What would scaling out look like?

B11 **Stripe Sandbox refund, local evidence (20 Sep)**
Is the refund real, and where has it been proven?

B12 **Scale-out and load results (SWE5001 ECS + local baseline)**
How does it scale, and how fast is it?

B13 **UC9 listing statechart**
Which listing moves are legal in each state?

B14 **UC4 design diagrams, complete**
Can we see every class and message of the join?

B15 **Every member's use case and design problem**
Where are the other members' models?

B16 **Entity-relationship diagram, all tables**
What does the whole schema look like?

D9-1 **Diagram details, 1:1 (D slide-step, e.g. D15-2)**
Every crop the talk steps through, readable in the PDF

Click a row (or type #B6 or #D15-2 in the URL). Slide 39 stays up during questions.

Agile artefacts appendix

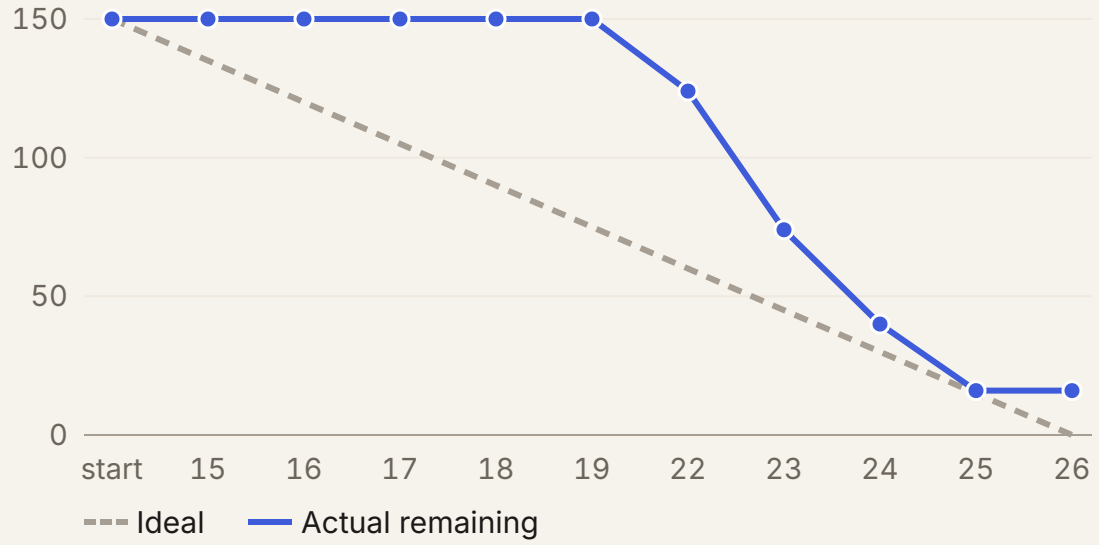
Artefact	Where it lives	State on 21 Oct
Product backlog	GitHub Issues and Project board 2 (GroupBuy Kanban Board); issue templates in .github/ISSUE_TEMPLATE	170 items: 137 Done, 15 Ready, 15 Backlog, 3 In progress
User stories with acceptance criteria	UC issues #24–#32; story issues (e.g. #221, #223, #180); AC per story in docs/agile/sprint-03/plan.md §5	template: complexity, accountable members, design problem, AC
Sprint plans	S1 #49; S2 Sprint 2 plan (docx) and #93; S3 docs/agile/sprint-03/plan.md and #232; S4 docs/agile/sprint-04/plan.md and #465	all four sprints
Burndown charts	S2 chart in docs/sprint2-progress-report.typ; daily data docs/agile/metrics/*.csv (board-snapshot.yml, since 17 Sep)	S2, S3 reconstructed (B2); S4 from snapshots
Sprint reviews	S1, S2 in the progress reports; S3 docs/agile/sprint-03/review.md	S4 due 23 Oct `[FILL: S4 review]`
Retrospectives	S1 report; S2 report §Sprint Retrospective	S3 missing (#451) · S4 due 23 Oct `[FILL: S3, S4 retros]`
Day logs, DoD	docs/agile/sprint-0N/log/ (S2–S4); DoD in the S2 report, sprint-03/plan.md §8, sprint-04/plan.md §8	dated logs per working day

Source: board and issues read with gh on 21 Oct; docs/agile/README.md. The report appendix holds the full set (#463).

Sprint 2 and 3 burndowns

Sprint 2 · remaining hours, 15–26 Sep

150 h planned · 16 h left · reconstructed from merge dates

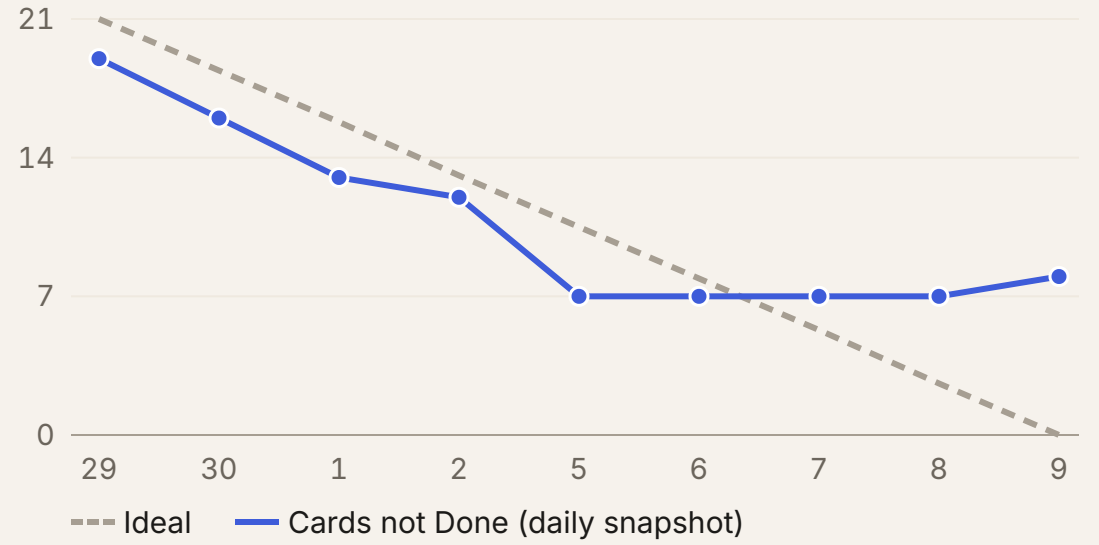


Flat for a week, then a cliff: work merged late in very large pull requests. Retro answer: PRs under 1,500 lines and hours on the board daily.

Source: docs/sprint2-progress-report.typ, "Daily burndown data"

Sprint 3 · open sprint cards, 29 Sep–9 Oct

21 cards · 150 h planned · 33 h left at freeze (review)



Steadier burn, then stuck at 7 from 5 Oct: #223, #224 and #229 waited for Checkout on dev, #226 for the NFR-37 decision. Hours come from the review.

Source: docs/agile/metrics/.csv, sprint-03/review.md*

WZ Eric

ADR index

Backend · backend/docs/adr

#	Decision
0001	Purchase intent lives in order; group terms on the product
0002	Team resolution is published; consequences run before commit
0003	Referral credit is a post-commit subscriber
0004	Retire the unused legacy PaymentIntent path
0005	Gate Stripe Checkout with a deployment property
0006	Payment owns the admin attention read model
0007	Outbox, one SNS topic, one SQS queue per consumer (Proposed)
0008	One codebase, two topologies: dev monolith, three services on ECS

Frontend · frontend/docs/adr

#	Decision
0001	React SPA, pure CSR; Router owns URL state, Query server state
0002	Same-origin /api with an HttpOnly cookie session
0003	Generated API client, not committed
0004	One build artifact promoted dev to prod
0005	SPA fallback by a CloudFront Function
0006	oxlint and oxfmt toolchain
0007	Hand-managed dev CloudFront, then Terraform import
0008	Documentation and evidence conventions
0009	Result envelope: one unwrapResult as select
0010	React Compiler via the Babel preset
0011	Testing strategy: Vitest, MSW, Playwright smoke
0012	Agent configuration in AGENTS.md
0013	pnpm and Node pinned, exact versions
0014	Stripe Hosted Checkout with a server-owned intent
0015	Admin console shell, roles guard, versioned writes

LJ Liu Jin

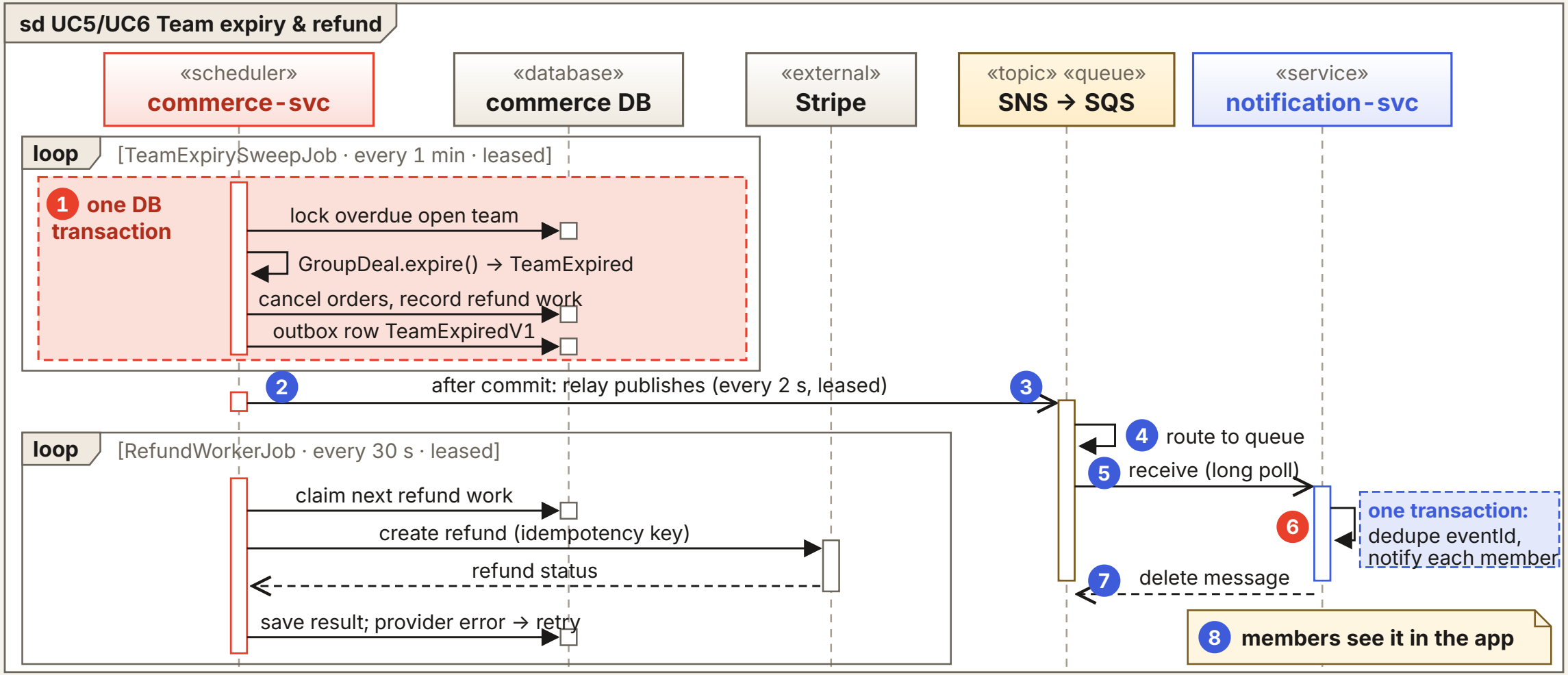
Database schema, all tables

Context	Tables	Migrations	Keys that make “once” hold
identity	`sys_user, sys_user_oauth, sys_role, sys_permission, rel_user_role, rel_role_permission`	V3, V24	one binding per Google subject
catalog	`biz_product, biz_team_price_policy, biz_team_price_quote, log_product_team_terms`	V4, V6, V14, V20, V21	biz_product.version (Optimistic Offline Lock)
groupbuy	`biz_group_deal, biz_group_member, biz_group_participation_receipt, biz_share_invite, biz_share_credit`	V5, V7, V18	one member per user and team; one receipt per payment
order	`biz_order, biz_purchase_intent`	V12, V15, V19, V22	intent fulfilled by one payment
payment	`biz_payment, log_payment_webhook_event, biz_refund, log_payment_review, biz_payment_review_case, log_refund_attempt`	V2, V9–V11, V16	webhook event_id; one refund per payment
notification	`biz_notification, sys_consumed_message`	V27, V28	PK (event_id, consumer); (event_id, user_id)
jobs, outbox	`sys_outbox, sys_scheduler_lock, sys_scheduled_item_retry, sys_checkout_expiry_cursor`	V13, V17, V25, V26	one lease per job; outbox id = eventId

28 Flyway files, 29 tables in schema `domain`; V1 sets the rules: snake_case, timestamptz, numeric money, invariants in the database. On ECS (ADR 0008): one database per service on one RDS instance; identity-svc and notification-svc own their rows above, commerce-svc the rest.

YQ Yorik

Expiry and refund sequence



As built for ECS. The SNS → SQS hop and notification-svc (steps 2–8) are the ECS topology; on dev the monolith runs no broker.

YQ Yorik

UC6 refund recovery: candidates and implementation decisions

Candidate	Decision	Why
Mark refunded, then call Stripe	**Rejected**	A crash would report money returned that never left
Hold a DB transaction across the call	**Rejected**	A rollback cannot undo a refund; locks held over network I/O
Idempotency Key per obligation	**Selected**	Every retry reuses one request identity
Retry after a delay	**Selected**	Fixed 30 s in code, not exponential backoff
Human-in-the-loop review	**Selected**	Mismatched or ambiguous work leaves automatic claims
Distributed transaction with Stripe	**Rejected**	No shared protocol across PostgreSQL and the provider

Implementation decisions

1. Obligation commits with the expiry, before any I/O
2. Key from the obligation ID: `"blocbuy-refund-" + id`
3. Short claim and finish transactions; lease and version fence workers
4. Verify payment facts before, provider snapshot after
5. After 23 h: search Stripe by metadata, never create
6. Reconcile only retrieves an existing refund; no admin endpoint exists

Source: docs/swe5006/report-drafts/yorick-yu-contribution.md. Evidence is local with a provider substitute.

LJ Liu Jin

Test inventory

Kind · level	Tools	Count by module	Example classes	Runs
Unit · unit	JUnit Jupiter, Mockito	185 *Test classes (162 unit; 23 Spring/Testcontainers), 1,329 annotations: domain 42, application 41, infrastructure 47, bootstrap 46, adapter 4, platform 2, commerce-svc 2, client 1 (some, like StripeCheckoutFlowTest, use real PostgreSQL)	`RefundServiceTest`, `PaidGroupDealTest`, `DealStatusTest`, `ProductStatusTest`	every PR (build-quality, required)
Integration · integration	Testcontainers: PostgreSQL 18, Redis 8.10, ElasticMQ	49 *IT classes (45 start containers), 242 annotations: bootstrap 24, infrastructure 15, services 9, platform 1	`OrderRepositoryIT`, `TeamExpirySchedulingIT`, `StripeRolloutAcceptanceIT` (acceptance)	every PR (build-quality, required)
Architecture · unit	ArchUnit	18 rules in 1 class	`ArchitectureTest`	every PR (build-quality, required)
API contract · integration	springdoc export, diff of contracts/, openapi-diff	1 contract file	`contracts/openapi.json`	backend PRs (contract-check, not required)
Component · unit	Vitest, Testing Library, MSW	77 files: features 58, routes 10, components 4, test 3, api 2	`account.test.tsx`, `admin-listing-edit.test.tsx`	every PR (frontend-check, required)
End-to-end · system	Playwright, Chromium	8 specs, 24 test declarations	`buyer-journey.spec.ts`, `join-refusal.spec.ts`, `smoke.spec.ts`	PR (e2e-preview); after deploy (e2e-dev)
Performance · system	k6 1.6.1 (SWE5001 ECS); Node workload (local)	k6 browse V2/V3; 3 local scenarios	`perf/k6/browse.js`, `backend/perf/workload.mjs`	by hand (k6 tooling tested in CI)

Counted 22 Oct at a1c6ac6e: 186 `*Test.java` (185 + ArchitectureTest), 49 `*IT.java` (45 start containers). Latest full runs: backend 2,048 passed, 1 opt-in skip (21 Oct log, local); frontend 799 (19 Oct log).

PS Pan

Workflows, required checks and timings

Workflow · trigger	Jobs (required in blue)	Median	Green
ci-backend · PR	**required**: build-quality (13.1 min), docker (2.8 min), SonarCloud Backend; contract-check (12 s)	16.1 min to docker end (18 runs, 16–19 Oct)	18 of 20 finished runs; 16 cancelled by a newer push
ci-backend · push to dev	package-dev 46 s → publish-ecr-dev 73 s → deploy-existing-ec2 40 s; SonarCloud Backend refreshes the dashboard	3.1 min to deployed (88 runs)	deploy 88 of 97; all 9 failures 14–17 Sep (SSM: instance not in a valid state)
ci-frontend · PR	**required**: frontend-check (3.4 min), SonarCloud Frontend; e2e-preview (69 s), intent-stale	4.8 min to e2e-preview end (19 runs)	19 of 20 finished runs
ci-frontend · push to dev	package-dev 35 s → deploy-s3-dev 21 s → e2e-dev 82 s; SonarCloud Frontend	2.7 min to e2e-dev green (94 runs)	deploy 97 of 97; e2e-dev 94 of 97 (3–4 Oct)
security · PR, dev, weekly	**required**: gitleaks (10 s); trivy-deps (72 s), trivy-config, workflow-lint, deployment-bundle when their paths change	12 s on most PRs	PR 97 of 98 finished; dev push 100 of 100
rollback-backend · manual	validate, then SSM rollback to the recorded previous digest or a given one	—	never run
SWE5001 · prod branch	swe5001-release-gate, swe5001-deploy, infra-swe5001 (promotion PR; see the SWE5001 deck)	—	—

Last 100 runs per workflow and event, read with `gh run list` and the jobs API on 19 Oct. PR durations count only runs where the gated jobs ran (docs-only PRs skip them in about 10 s). Push medians run from the push run start, not the merge click.

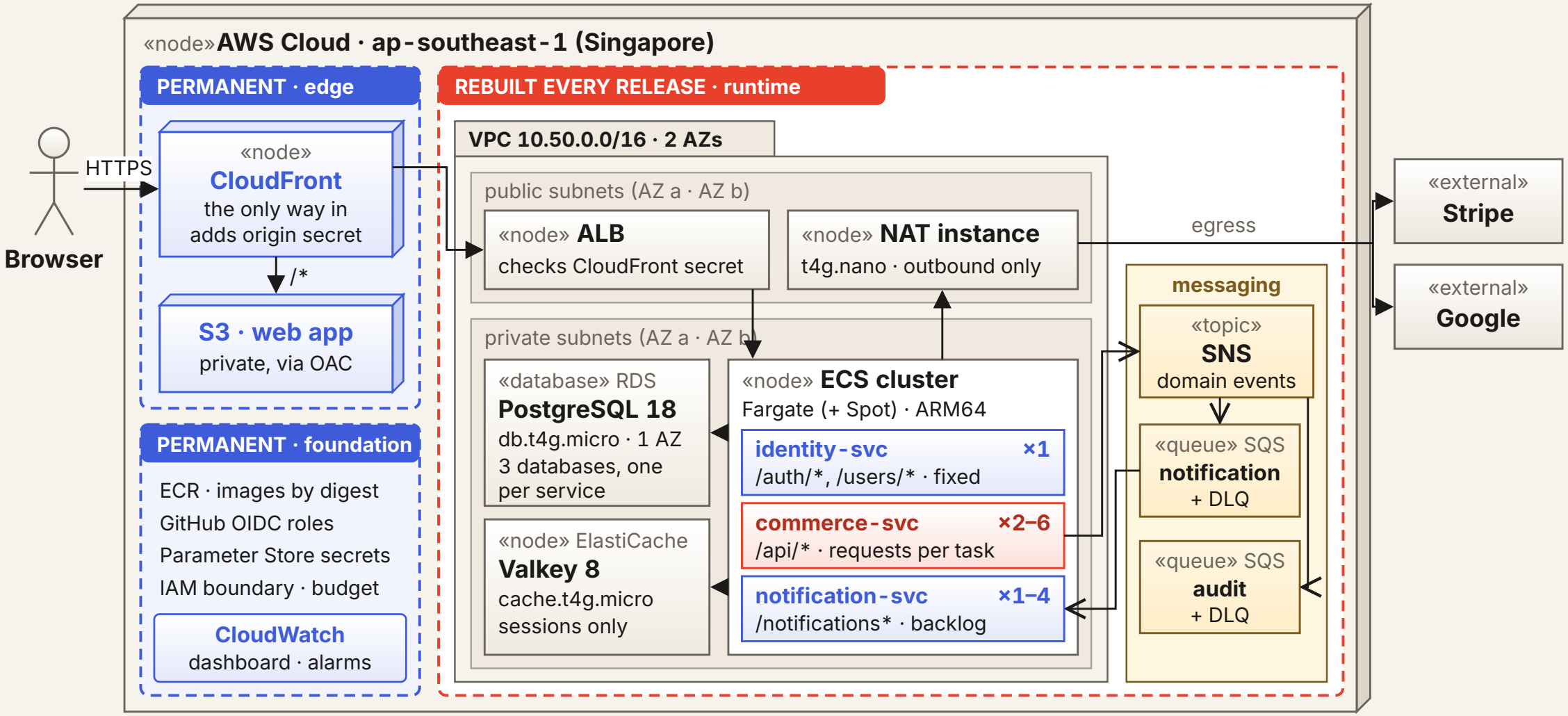
PS Pan

Known gaps and limitations

Gap	Effect	Status
CodeQL off	no CodeQL SAST; SonarCloud and SpotBugs remain	ENABLE_CODEQL=false; code scanning not enabled on the repository
CloudFront → ALB over HTTP	/api payloads and the origin header cross the AWS network unencrypted	NFR-37 TLS still unmet; accepted risk on dev until 16 Nov (#227, NFR register); fix needs an origin hostname and ACM
SSH open to the internet	port 22 from 0.0.0.0/0 on the dev host, which has a public IP; SSM sessions make it unnecessary	AWS read-only check, 21 Oct; open item 9
Dev EBS volume not encrypted	database and host data at rest unencrypted	AWS read-only check, 19 Oct; not raised as an issue yet
Rollback never exercised	recovery path untested on the real host	rollback-backend.yml: 0 runs; no automatic rollback has fired
No dev alarms	failures are noticed by e2e-dev or by people	all six CloudWatch alarms belong to SWE5001
Dev not zero-downtime	a deploy restarts the only API container (502 on 3–4 Oct)	accepted for dev
Secrets in a host file	manual creation and rotation, root-only 0600	accepted for dev; prod uses Parameter Store
Rebuilt after merge	the deployed image and dist/ are not the PR-scanned artefacts; Trivy not re-run	strict up-to-date checks; ECR scans on push; ADR 0004 text to reconcile
Weekly full-history gitleaks red	20 digest false positives fingerprinted; 4 badge-token findings still fail the scan	PR #580, ci-failure-audit-20261021.md
No DAST, no SBOM	no scan of the running site; no component inventory per image	not planned before 26 Oct

PS Pan

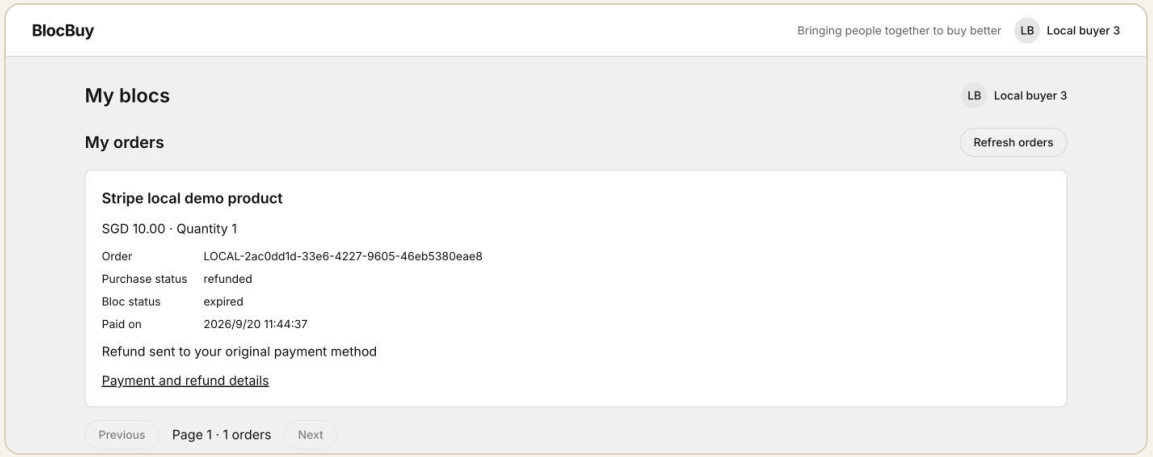
SWE5001 production topology



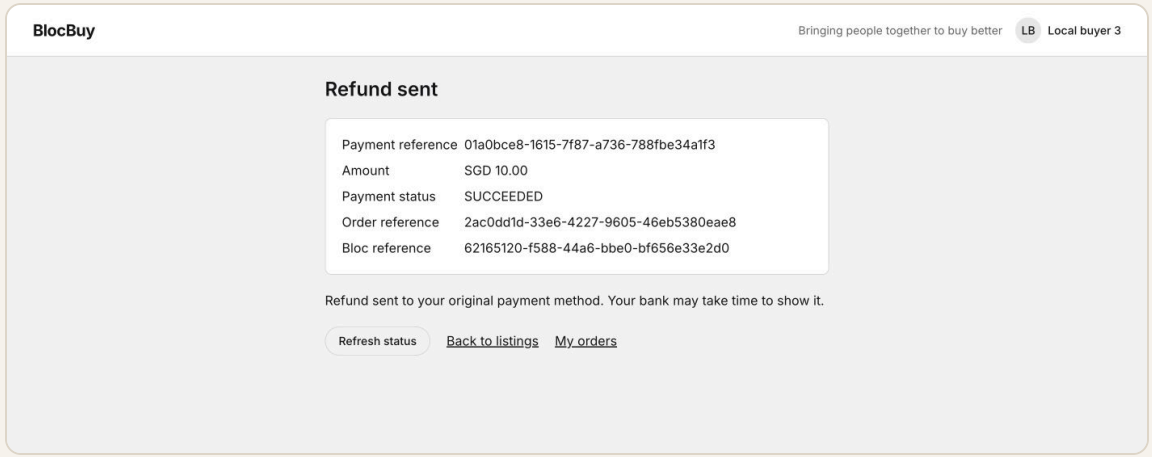
SWE5001 prod: the same code on ECS, rebuilt from scratch on every release and destroyed after it. SWE5006 dev is slide 31.

YQ Yorik

Stripe Sandbox refund, local evidence (20 Sep)



My orders: the expired bloc's order is refunded (UI of 20 Sep, since redesigned)



Receipt: "Refund sent", payment 01a0bce8..., SGD 10.00

Scenario	Result on 20 Sep (Team8 Stripe Sandbox, livemode=false)
START and JOIN	two SGD 10 payments → one member and one order each; the bloc reached succeeded, 2 of 2
Webhook replay	replayed valid events changed nothing; a modified payload with the old signature got 400
Expiry	an under-filled bloc expired → one SGD 10 refund obligation; its unpaid Checkout session expired
Refund	one full Sandbox refund `re_3UHbeV5iEVLN2uQJ1V71krwQ`, succeeded; a recheck created no duplicate

Local, not dev: real Stripe Sandbox calls from a local backend and PostgreSQL; no Google sign-in, no deployed environment. On dev the refund waits for #223 and #517. Source: `backend/docs/payment/stripe-sandbox-acceptance-20260920.md`

YQ Yorik

Scale-out and load results (SWE5001 ECS + local baseline)

SWE5001 PROD · ECS FARGATE

Scale-out

- Target tracking on ALB requests per target: 12,000 a minute, scale-out cooldown 60 s
- commerce 2–6 tasks in two AZs, notification 1–4 (read back 16 Oct)
- V3, 19 Oct: commerce **1 → 3 tasks** in about four minutes, no manual scaling; at 3 tasks ≈ 455 req/s, server p95 13–50 ms, 2 errors in 570,445
- Not V3 acceptance: the workstation generator dropped iterations, so the analyzer rejects the run
- V4 queue drain: `[FILL: backlog peak, notification tasks, drain time (19 Oct)]`

SWE5001 V2 · ECS, 16 OCT

Load, one task

- k6, ten one-minute plateaus from 10 to 200 requests a second, through CloudFront
- 57,600 requests, **0 errors**, run twice
- p95 between 23 and 57 ms on every plateau (target under 500 ms)
- No saturation reached: 200 a second is a tested lower bound, not a limit

DEV CODE · LOCAL RUN, 30 SEP

Workload baseline

100 users each	p95	Errors
Browse (list + detail)	24 ms	0
JOIN intent, unpaid	57 ms	0
My Orders	18 ms	0

30 paid teams expired within 58 s of their deadlines under that load. One laptop with real HTTP, PostgreSQL and Redis; **not a deployed result**.

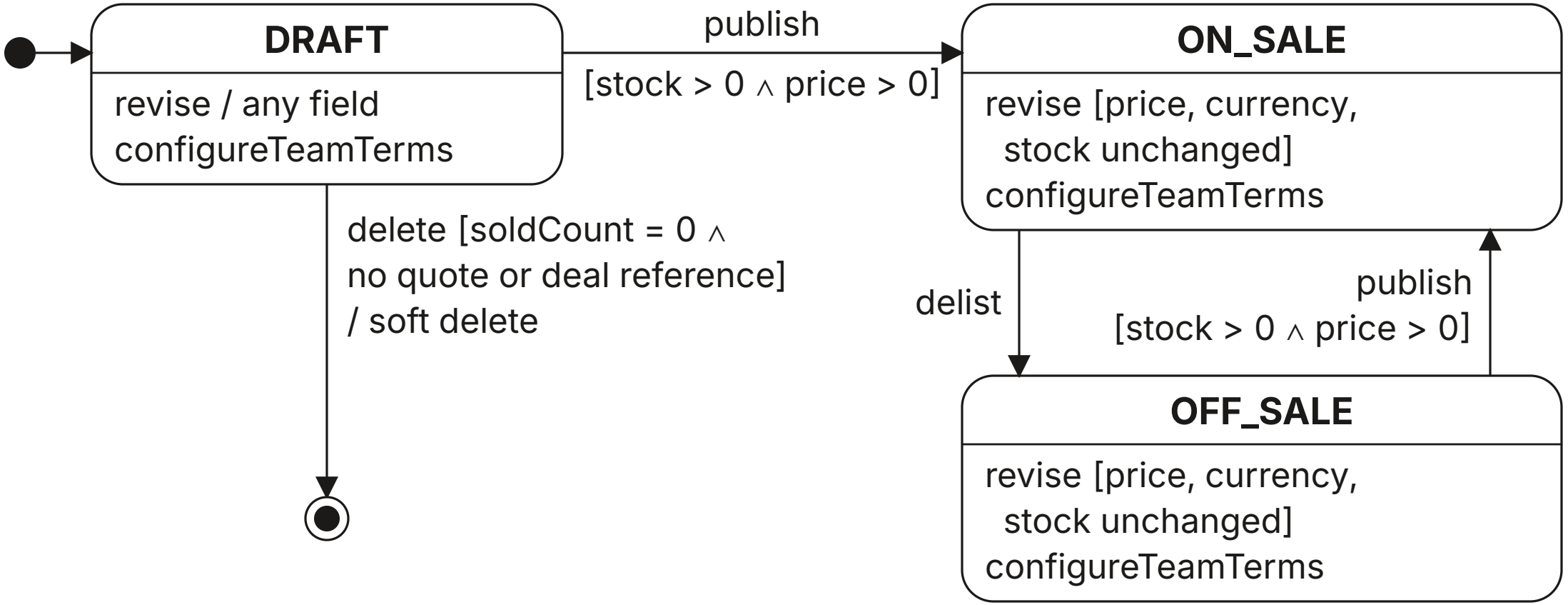
Dev, where the demo runs: one EC2 host by design (backend ADR 0008), so no auto-scaling there. Same code, two topologies: scale-out exists only in the SWE5001 topology and is not claimed for SWE5006 (slide 37).

Sources: docs/swe5001/evidence/20261016-http-baseline-dashboard/README.md (V2, scaling policy); docs/swe5001/evidence/20261019-v3-autoscaling/README.md (V3); docs/perf/sprint3-baseline.md (local workload).

LJ Liu Jin

UC9 listing statechart

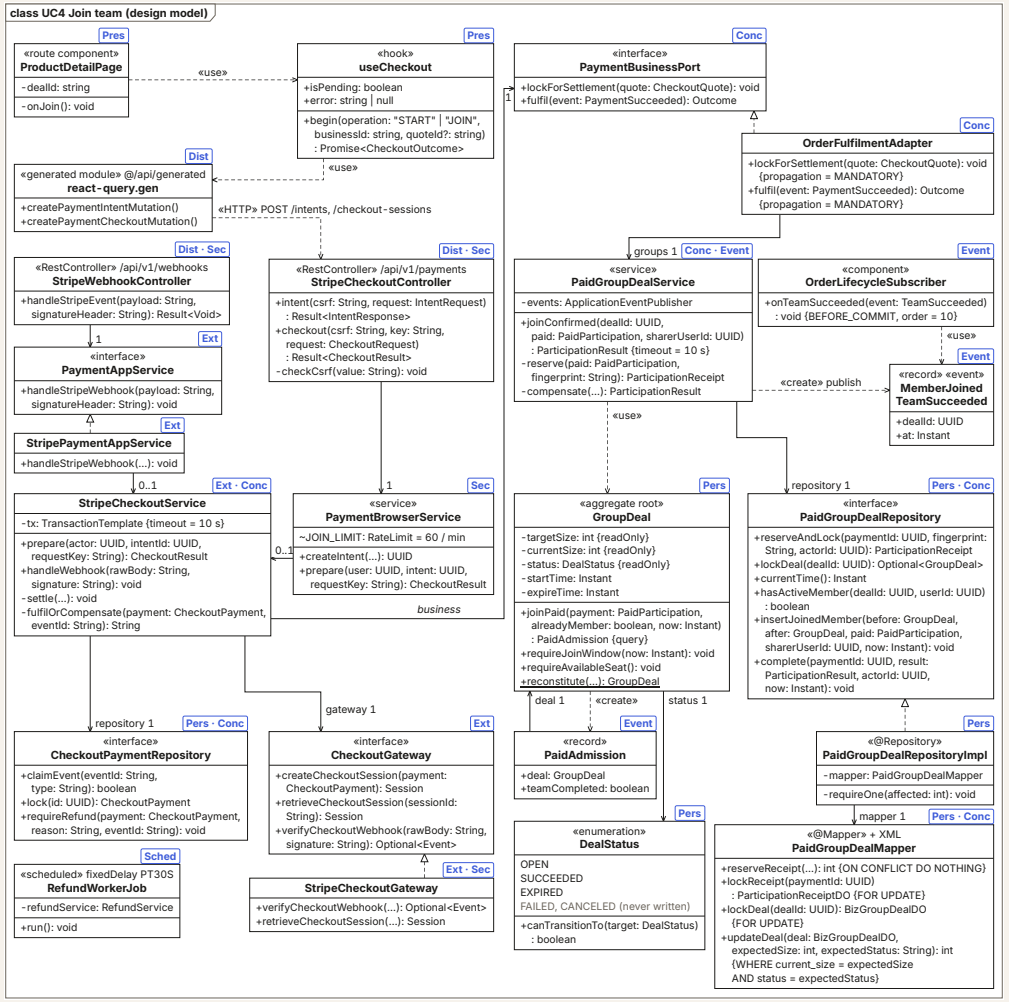
stm Listing lifecycle (Product.status)



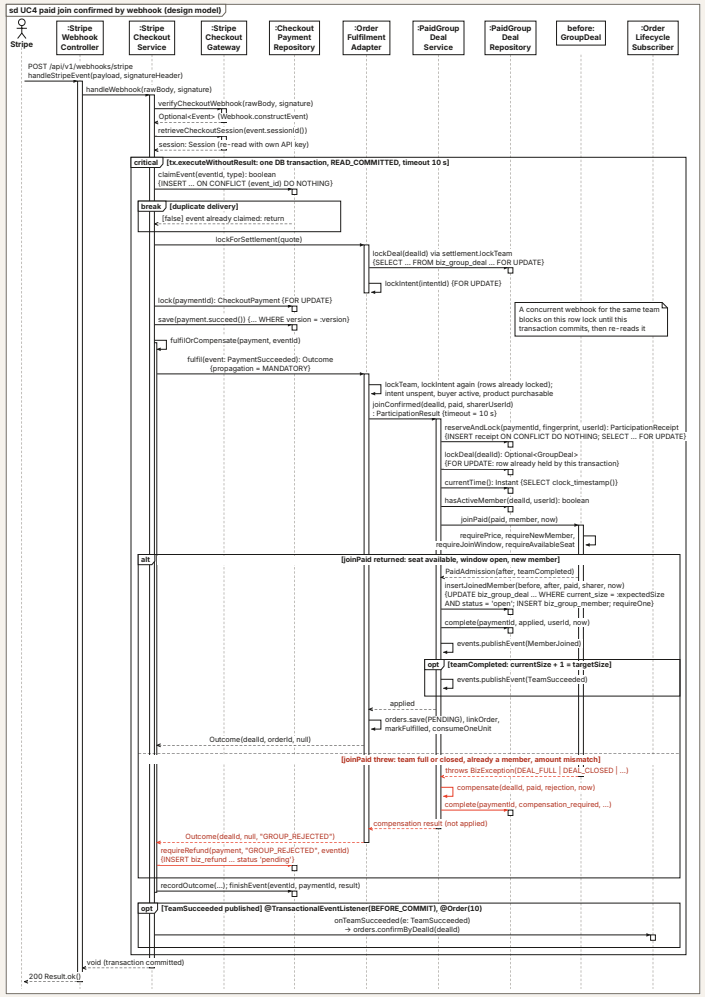
Nothing returns to DRAFT; any other event → 409, state unchanged.

WZ Eric

UC4 design class and sequence diagrams, complete



Design class diagram (2320 × 2314)



Design sequence (2320 × 3306)

Both are vector: open `src/diagrams/uc4-design-*.svg` in a browser tab and zoom. Every message carries its source line in the diagram spec. Report §3.4 (Eric).

Every member’s use case and design problem

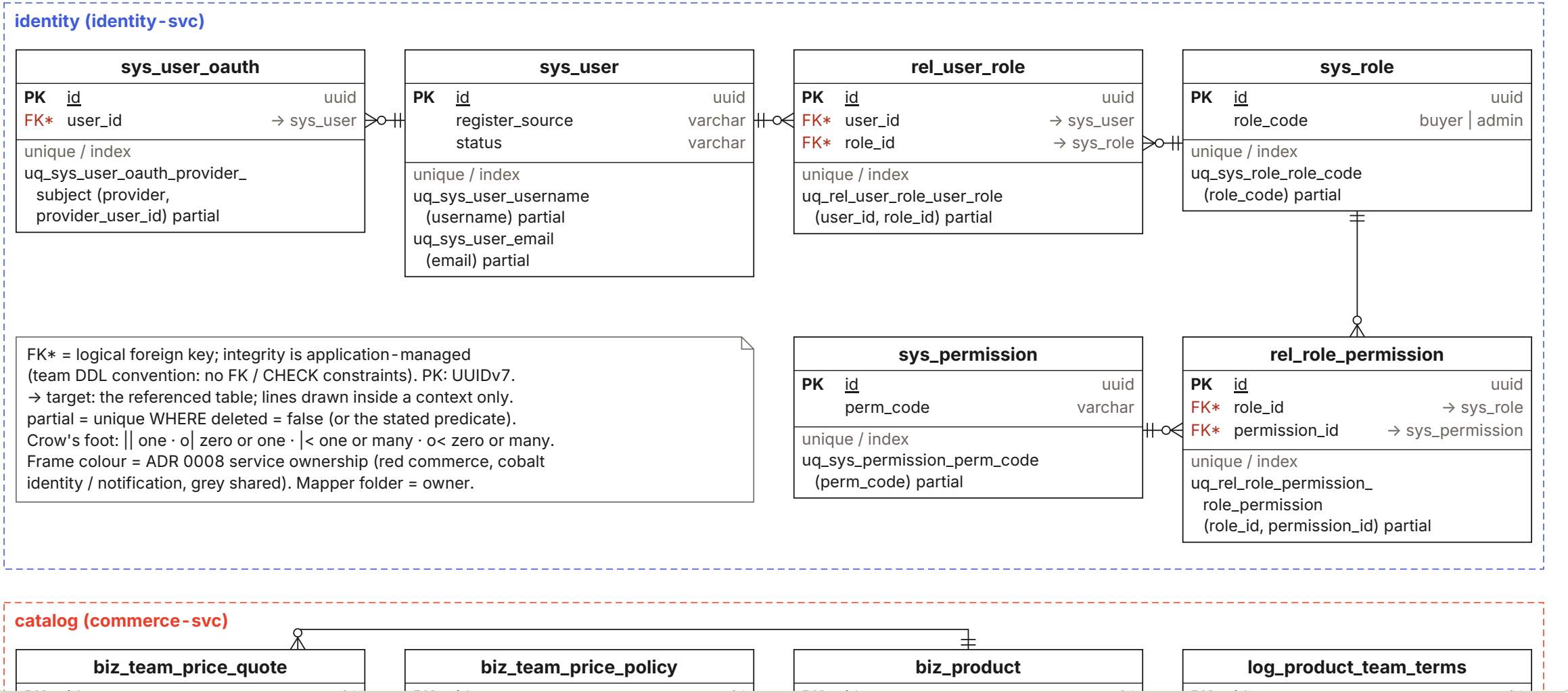
Member	Use case (report §3.4)	Design problem (report §3.5)	On a slide
Eric	UC3, UC4 start and join a team	Factory Method: team creation	UC4 models 13–16; pattern 17
Yorik	UC6 refund on team expiry	Observer: team outcome; refund idempotency as a mechanism (B6)	pattern 18; sequence B5
Pan	Receive notifications	Template Method: outbox transports	pattern 19
Lim	UC8 my orders and my teams (with UC7 frontend)	Strategy: UC1 pricing	pattern 20
Liu Jin	UC9 listing administration and team terms	State: listing lifecycle (full form)	patterns 21–22; statechart B13

Each member’s analysis class, analysis sequence, design class and design sequence diagrams are in their report section (template RT 3.4.1); the deck shows UC4 only (rubric G8). Owners: #463. `[VERIFY: UC9 owner: #32 names Eric, #463 names Liu Jin (open item 12)]`

LJ Liu Jin

Entity-relationship diagram, all 29 tables (scroll; 1:1 pages B16-1 to B16-21 follow)

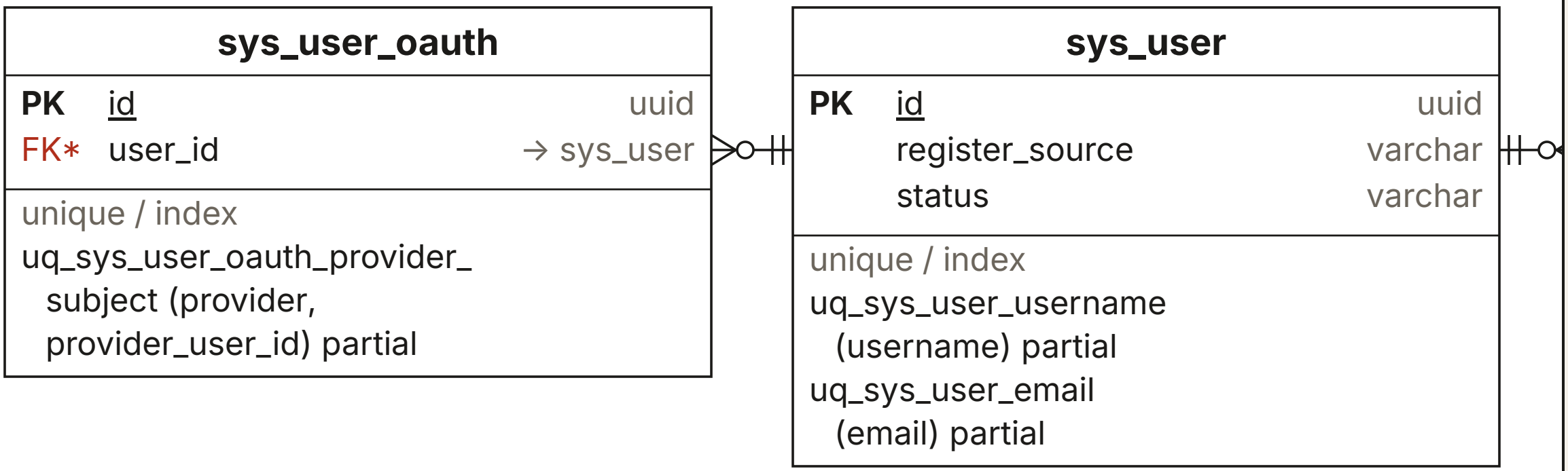
erd Full schema of the SWE5006 monolith (Flyway V1–V28, schema domain), by owning context



LJ Liu Jin

ERD 1:1 · identity (left, part 1/2) · 1/21

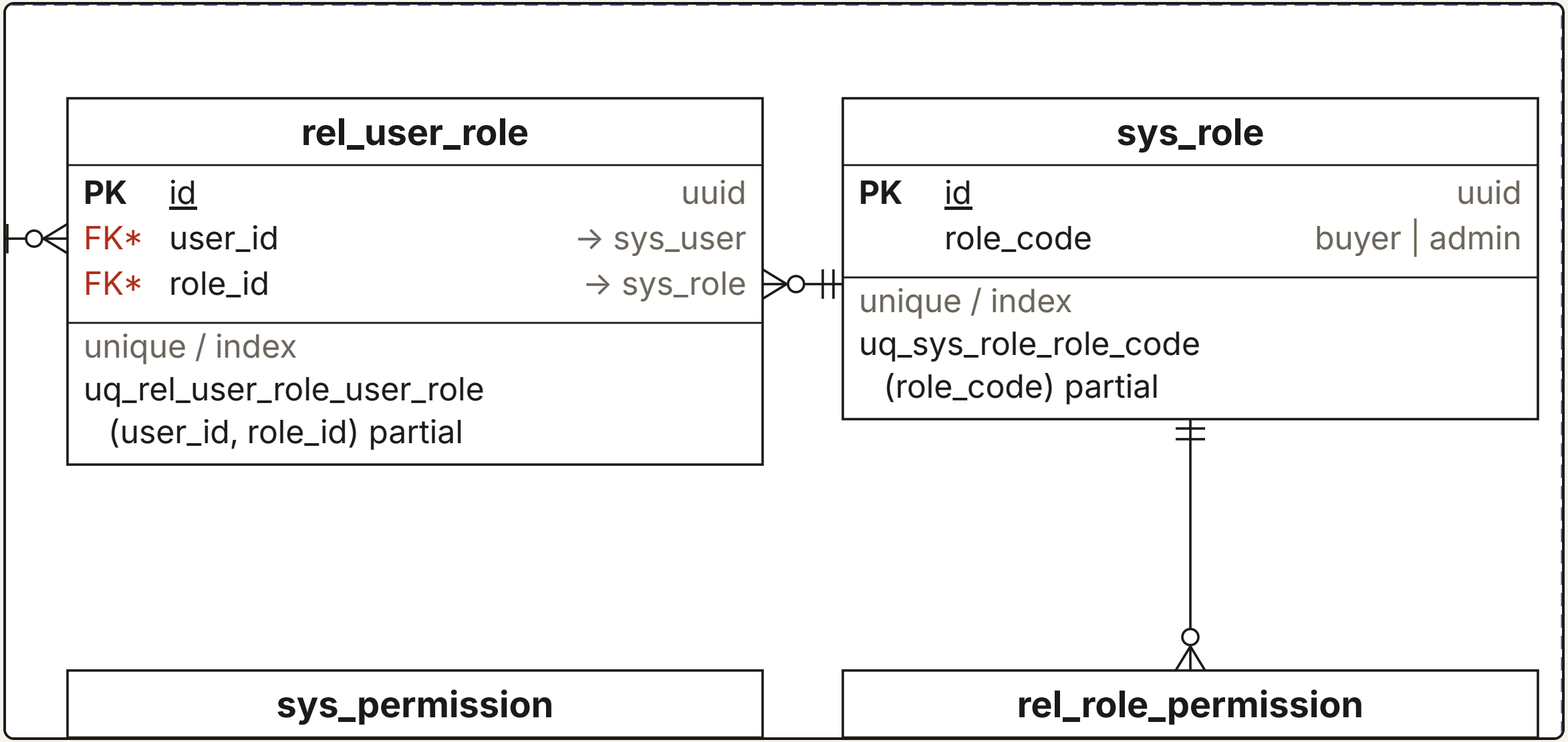
identity (identity-svc)



FK* = logical foreign key; integrity is application-managed

LJ Liu Jin

ERD 1:1 · identity (right, part 1/2) · 2/21



LJ Liu Jin

ERD 1:1 · identity (left, part 2/2) · 3/21

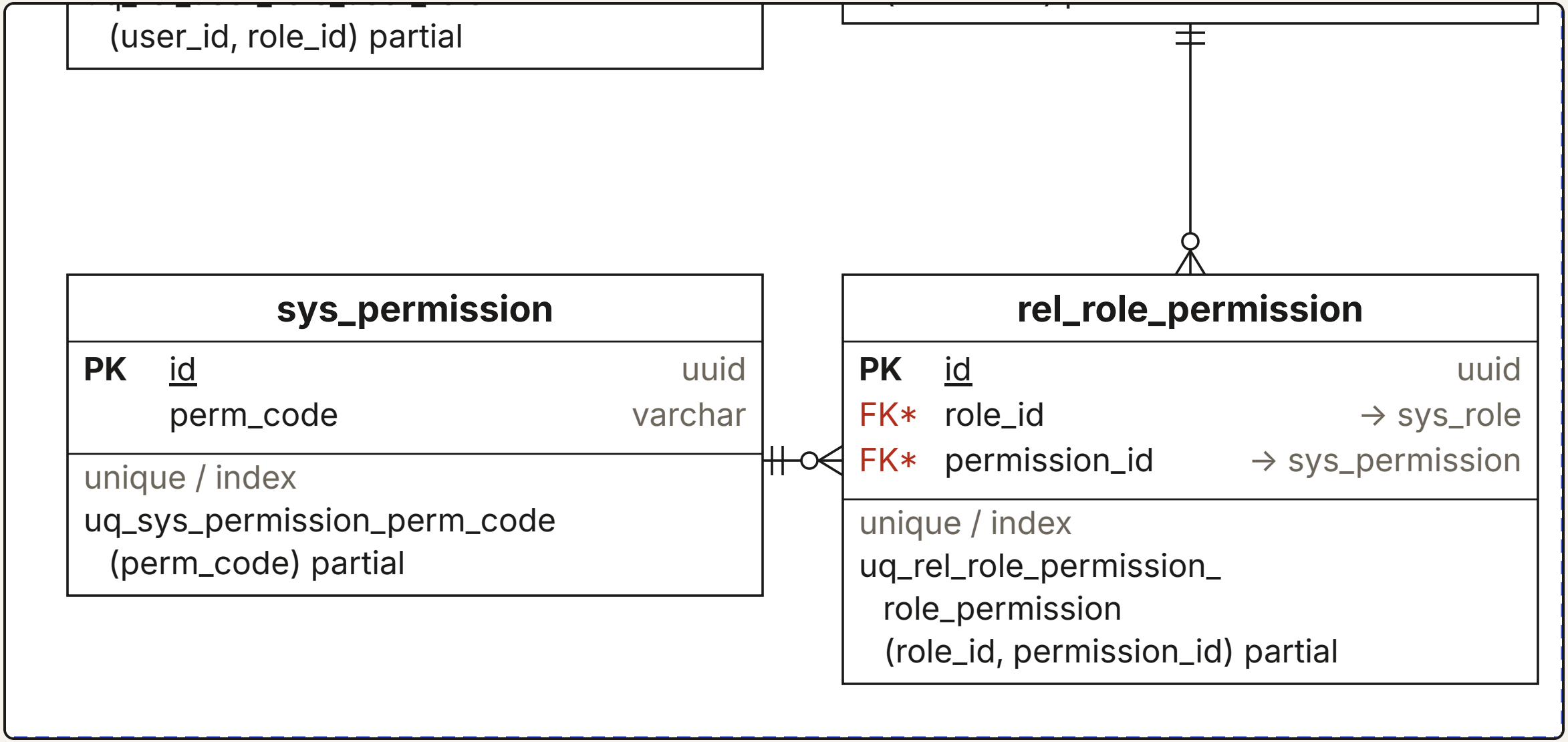
provider_user_id) partial

(username) partial
uq_sys_user_email
(email) partial

FK* = logical foreign key; integrity is application-managed
(team DDL convention: no FK / CHECK constraints). PK: UUIDv7.
→ target: the referenced table; lines drawn inside a context only.
partial = unique WHERE deleted = false (or the stated predicate).
Crow's foot: || one · o| zero or one · |< one or many · o< zero or many.
Frame colour = ADR 0008 service ownership (red commerce, cobalt identity / notification, grey shared). Mapper folder = owner.

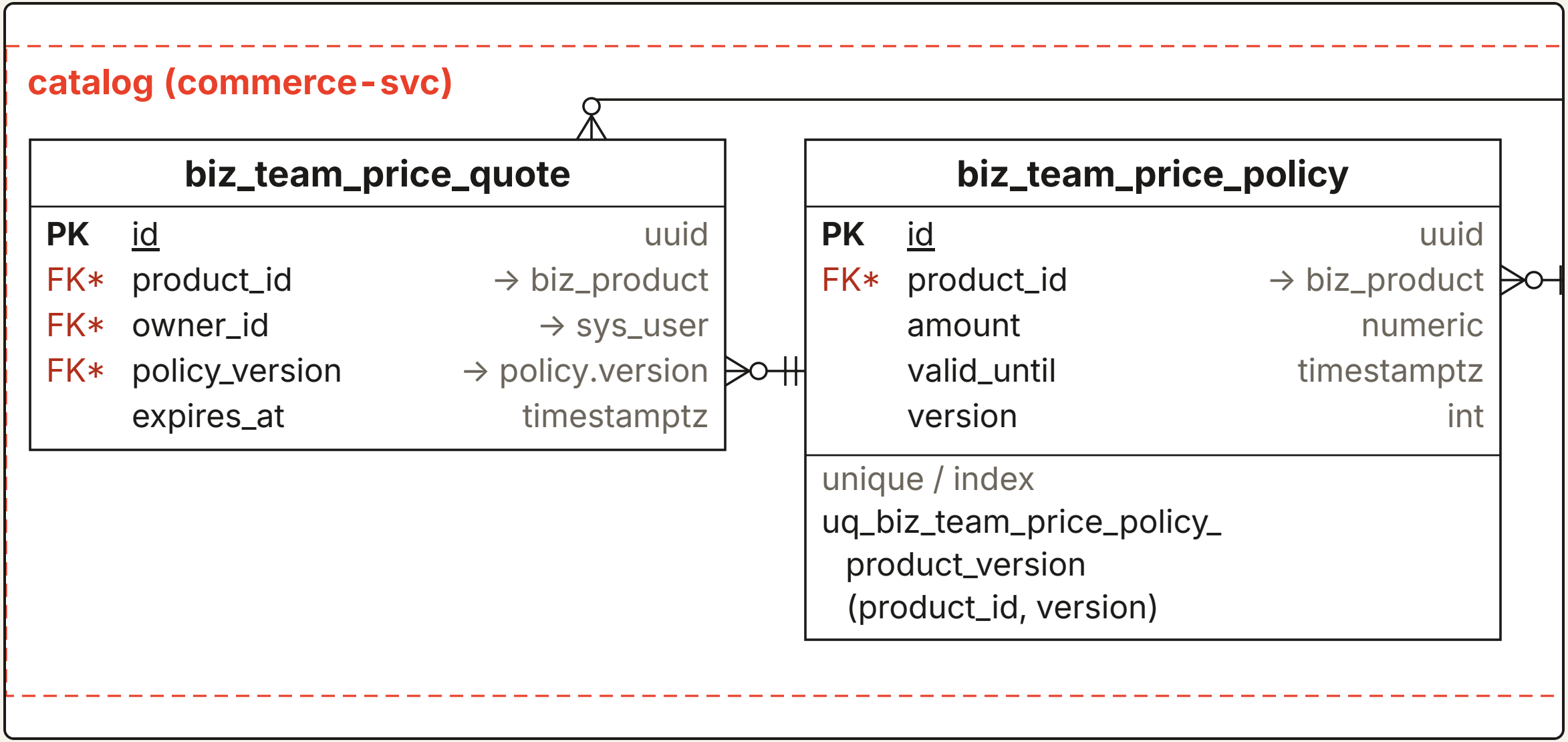
LJ Liu Jin

ERD 1:1 · identity (right, part 2/2) · 4/21



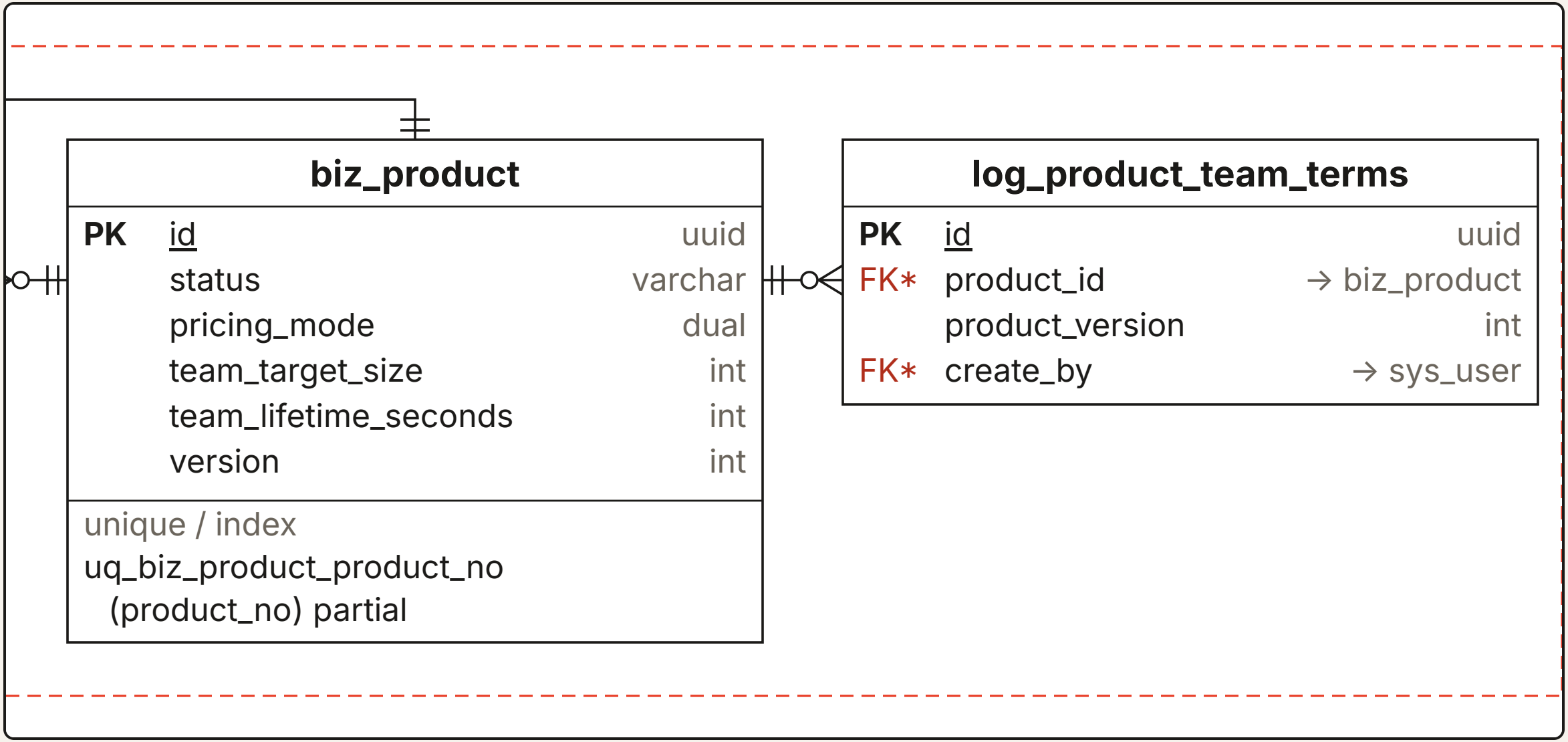
LJ Liu Jin

ERD 1:1 · catalog (left) · 5/21



LJ Liu Jin

ERD 1:1 · catalog (right) · 6/21



LJ Liu Jin

ERD 1:1 · groupbuy + share (left, part 1/2) · 7/21

groupbuy + share (commerce-svc)

biz_group_member

PK	id	uuid
FK*	deal_id	→ biz_group_deal
FK*	user_id	→ sys_user
FK*	payment_id	→ biz_payment
FK*	order_id	→ biz_order
FK*	sharer_user_id	→ sys_user

unique / index

uq_biz_group_member_deal_user
(deal_id, user_id) partial

uq_biz_group_member_payment_id
(payment_id)

biz_group_deal

PK	id	uuid
FK*	product_id	→ biz_product
FK*	leader_user_id	→ sys_user
	target_size, current_size	int
	status	varchar
	expire_time	timestamptz
	version	int

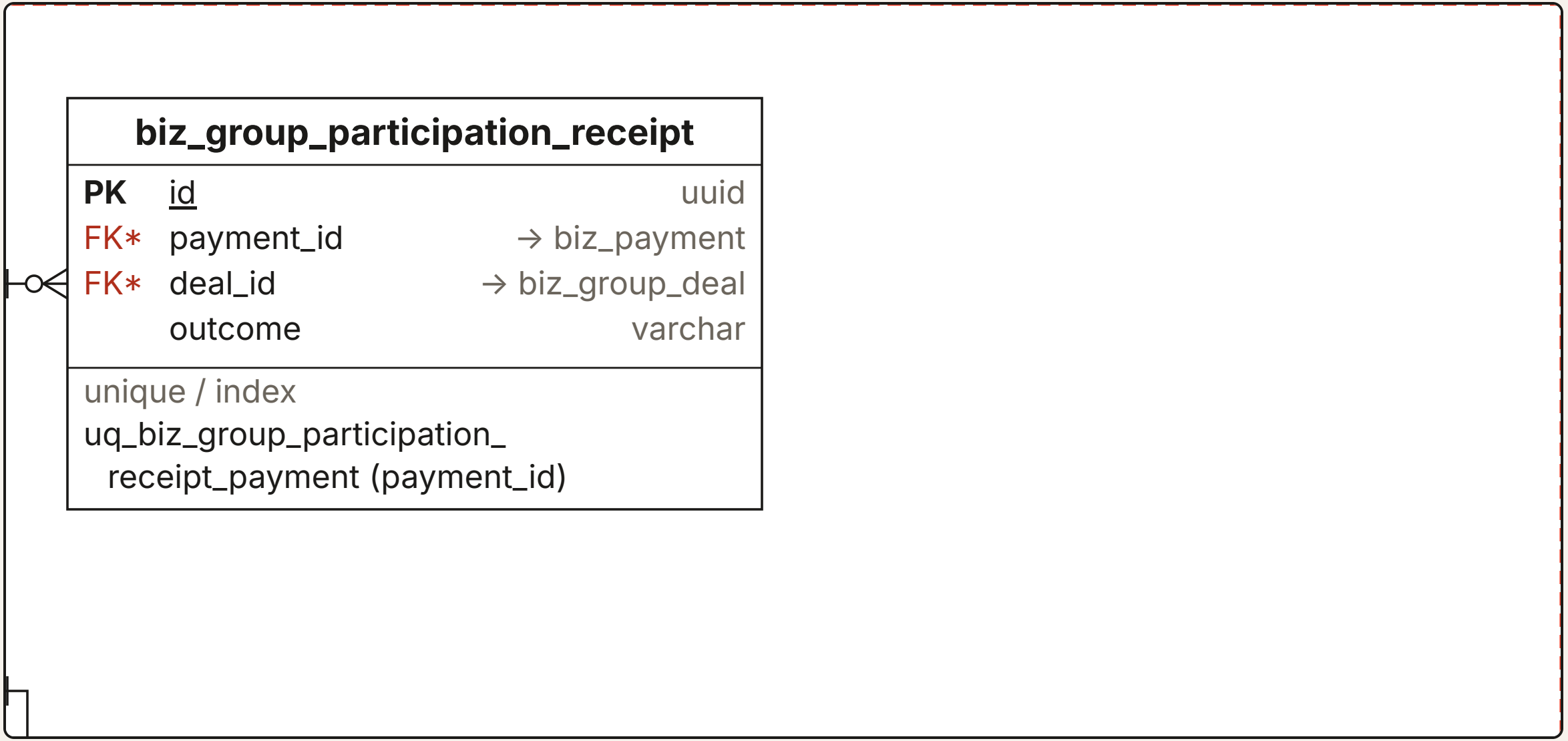
unique / index

uq_biz_group_deal_deal_no
(deal_no) partial

idx_biz_group_deal_status_expire
(status, expire_time) partial

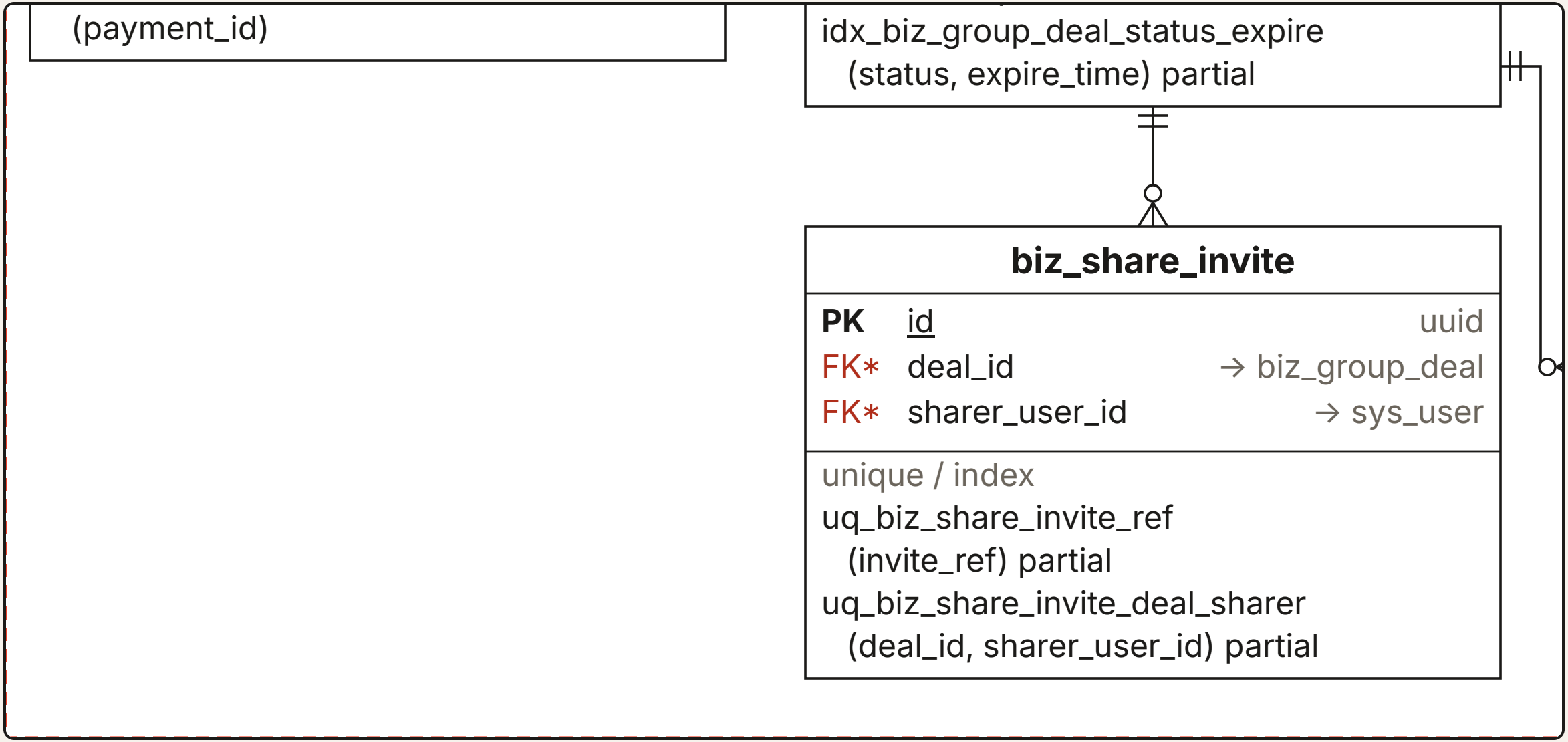
LJ Liu Jin

ERD 1:1 · groupbuy + share (right, part 1/2) · 8/21



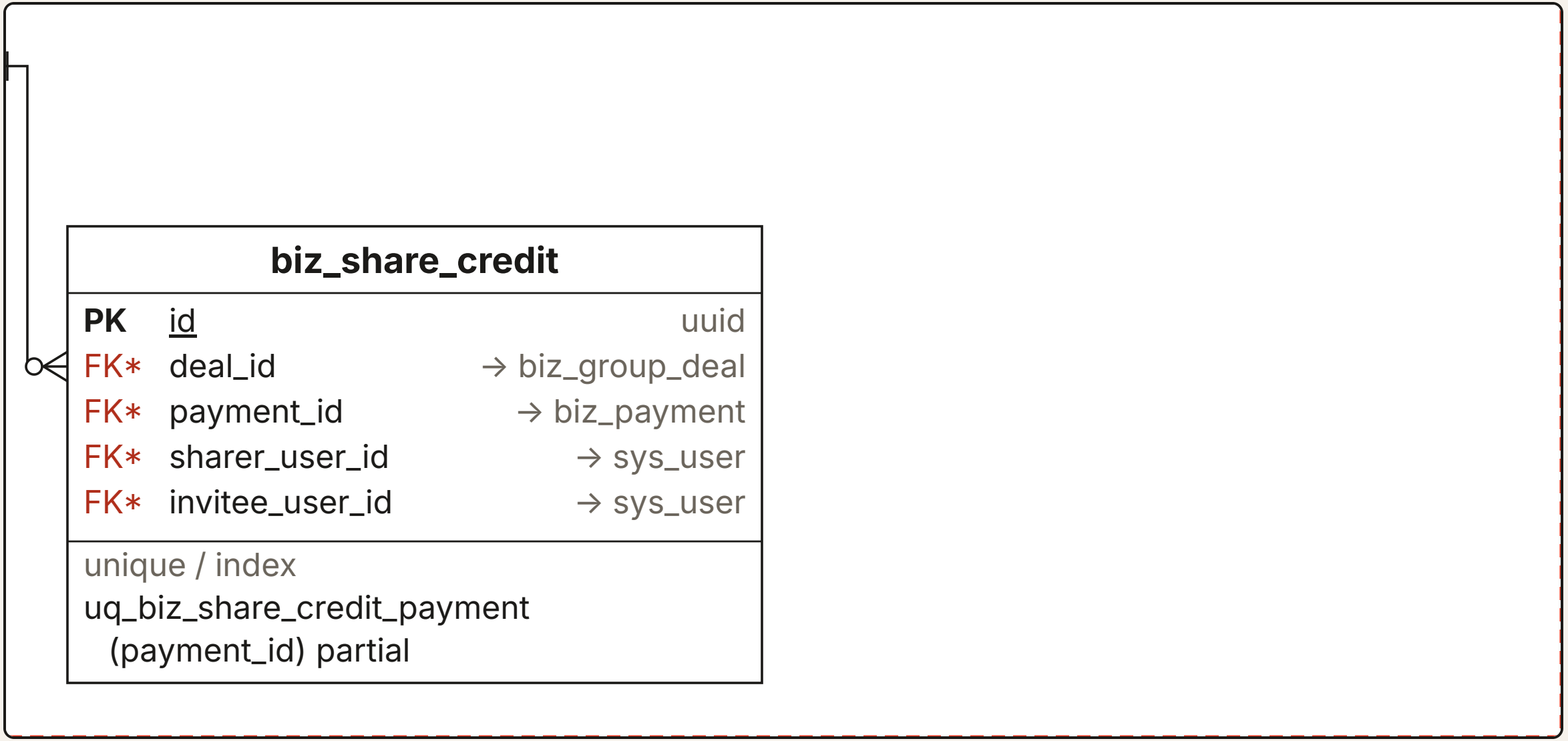
LJ Liu Jin

ERD 1:1 · groupbuy + share (left, part 2/2) · 9/21



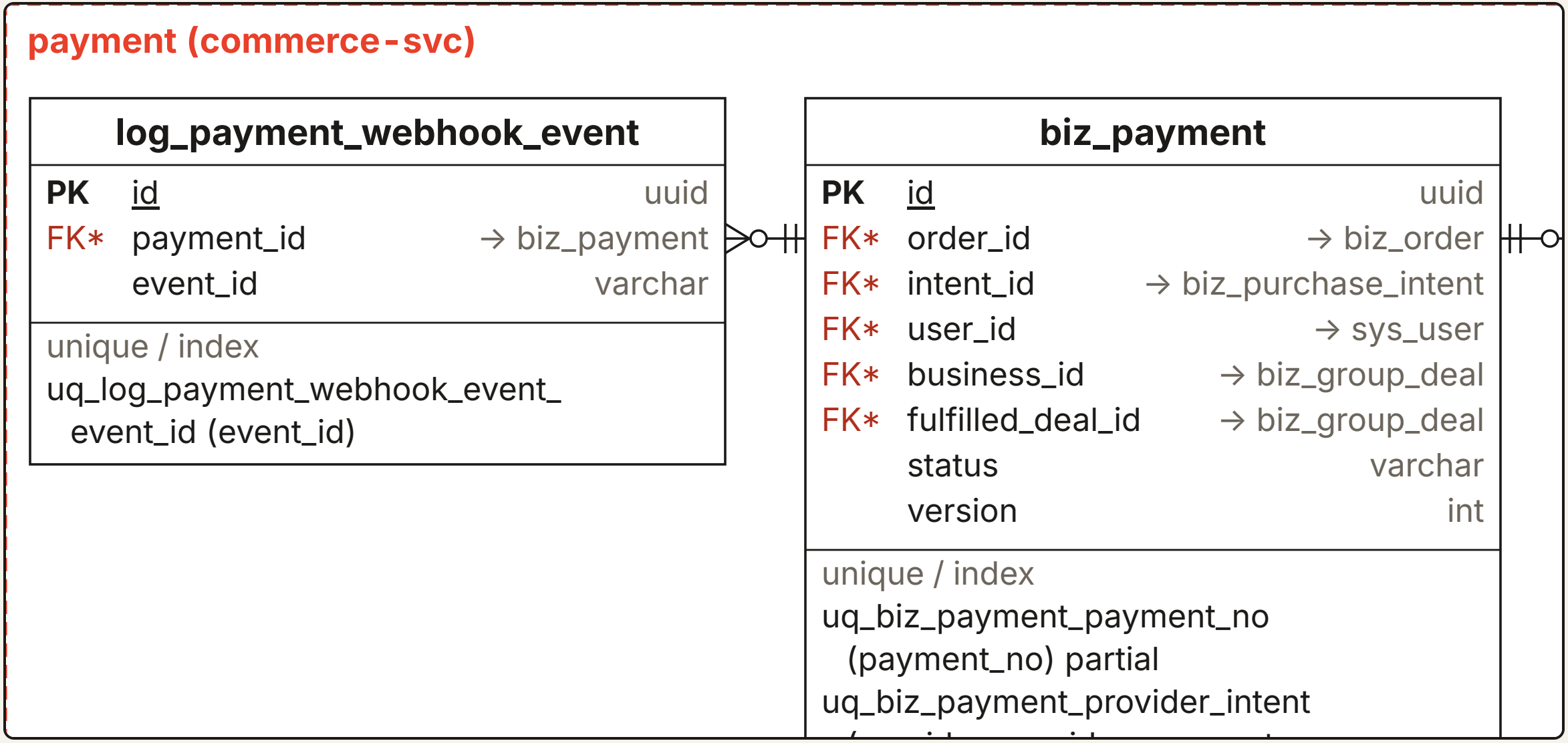
LJ Liu Jin

ERD 1:1 · groupbuy + share (right, part 2/2) · 10/21



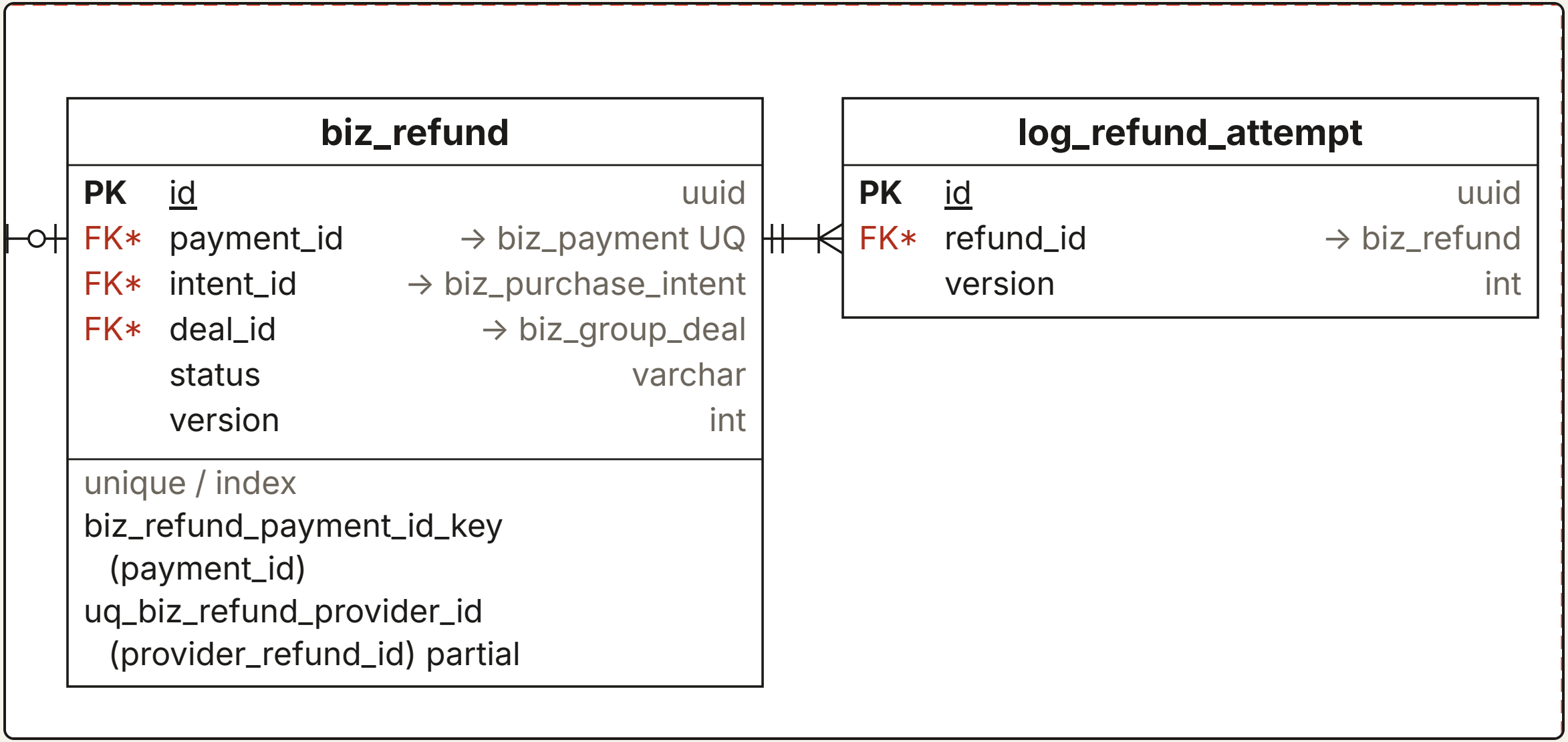
LJ Liu Jin

ERD 1:1 · payment (left, part 1/3) · 11/21



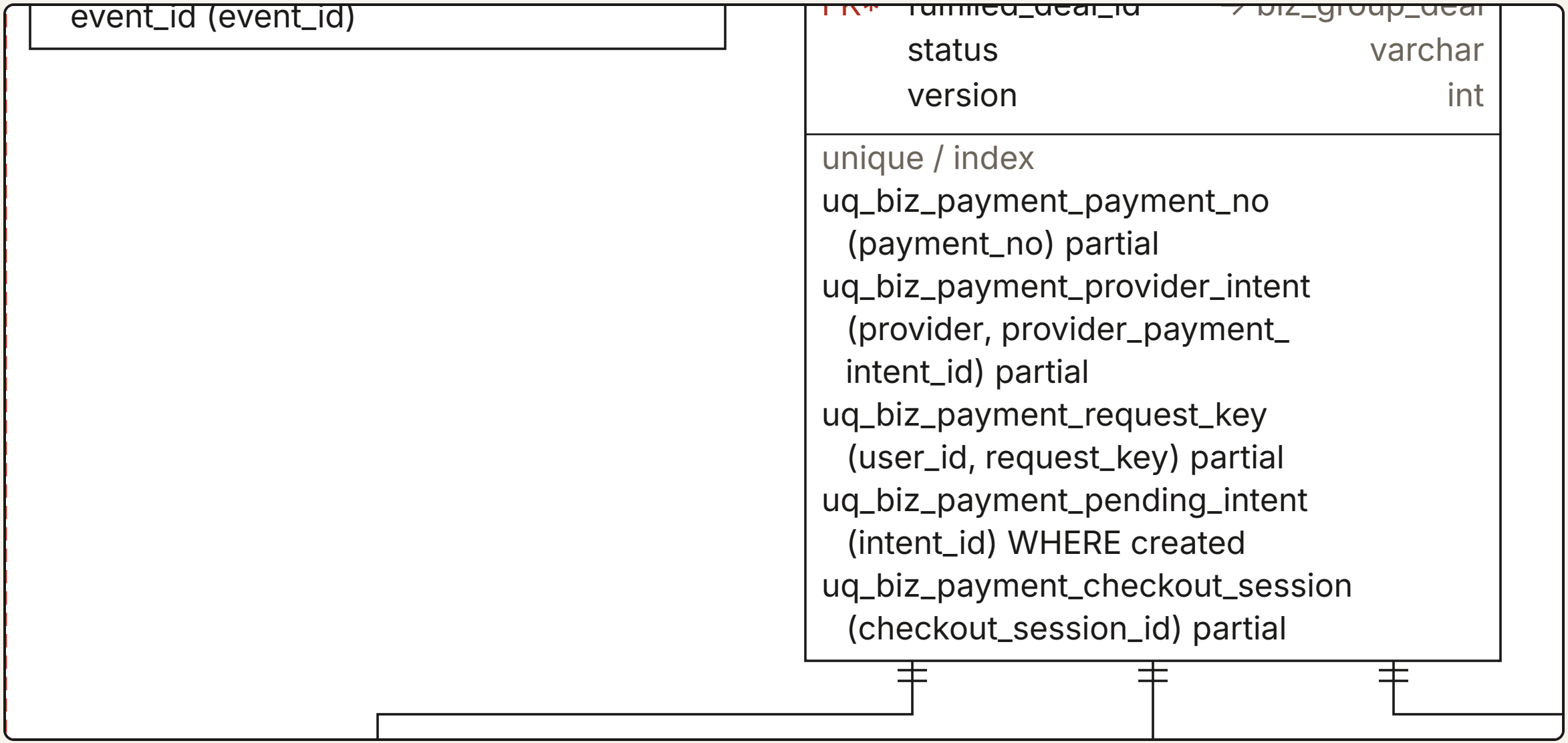
LJ Liu Jin

ERD 1:1 · payment (right, part 1/3) · 12/21



LJ Liu Jin

ERD 1:1 · payment (left, part 2/3) · 13/21



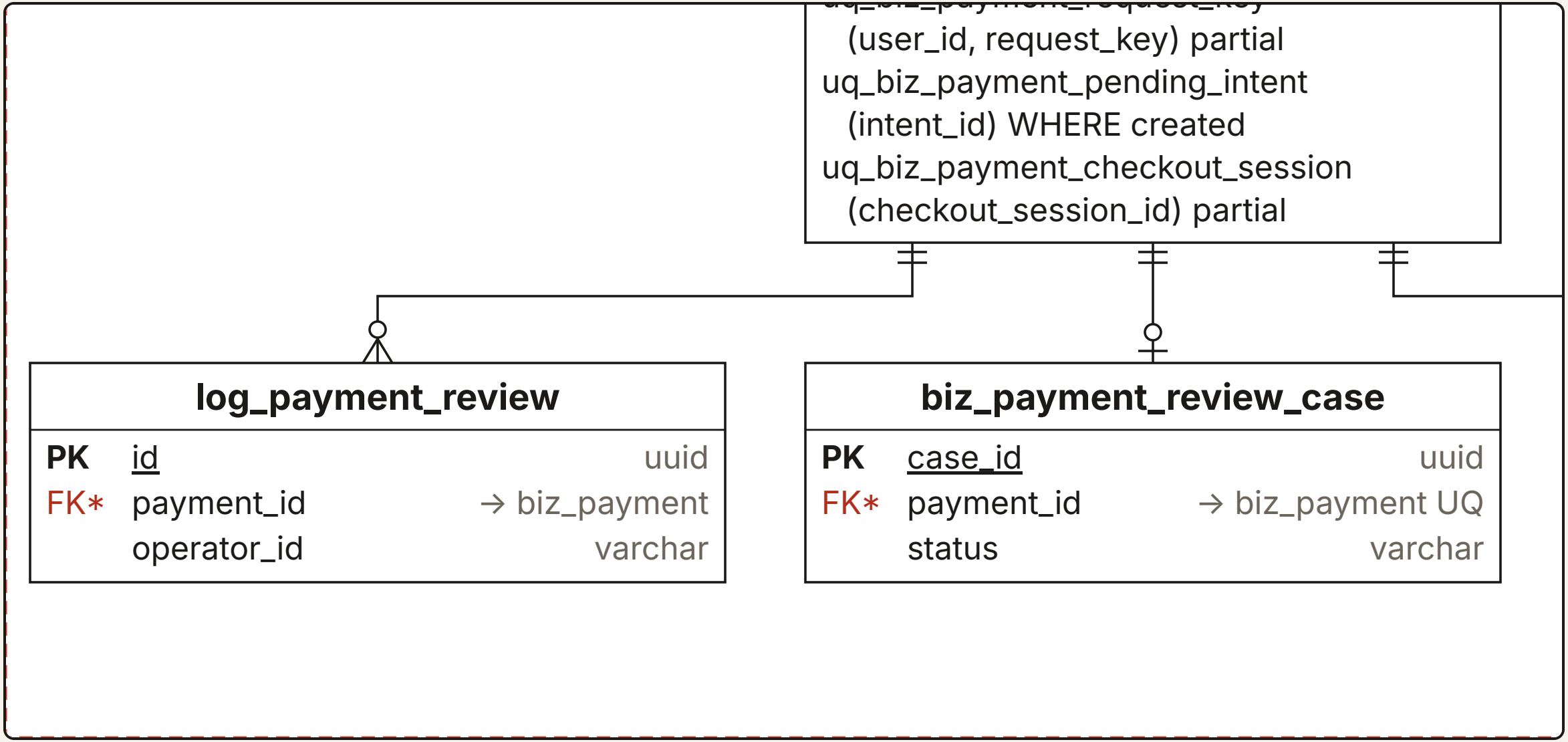
LJ Liu Jin

ERD 1:1 · payment (right, part 2/3) · 14/21

version	int
unique / index	
biz_refund_payment_id_key (payment_id)	
uq_biz_refund_provider_id (provider_refund_id) partial	

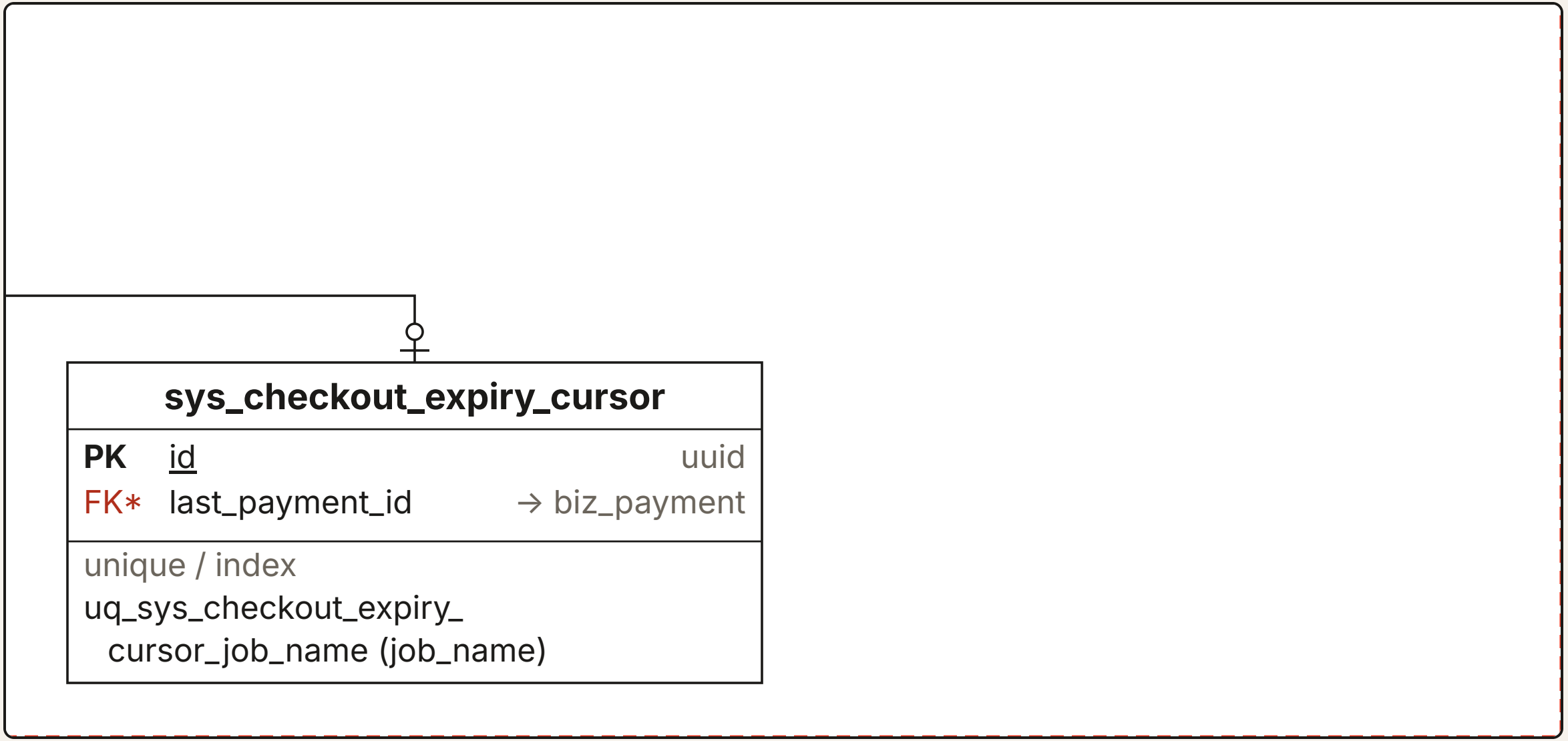
LJ Liu Jin

ERD 1:1 · payment (left, part 3/3) · 15/21



LJ Liu Jin

ERD 1:1 · payment (right, part 3/3) · 16/21



LJ Liu Jin

ERD 1:1 · order (part 1/2) · 17/21

order (commerce-svc)

biz_purchase_intent

PK	id	uuid
FK*	user_id	→ sys_user
FK*	deal_id	→ biz_group_deal
FK*	product_id	→ biz_product
FK*	fulfilled_payment_id	→ biz_payment
FK*	sharer_user_id	→ sys_user
	version	int

unique / index

uq_biz_purchase_intent_fulfilled_payment (fulfilled_payment_id) partial

biz_order

PK	id	uuid
FK*	payment_id	→ biz_payment
FK*	user_id	→ sys_user
FK*	deal_id	→ biz_group_deal
FK*	product_id	→ biz_product
	status	varchar
	version	int

unique / index

uq_biz_order_order_no (order_no) partial

uq_biz_order_payment_id (payment_id)

LJ Liu Jin

ERD 1:1 · order (part 2/2) · 18/21

biz_purchase_intent		
PK	id	uuid
FK*	user_id	→ sys_user
FK*	deal_id	→ biz_group_deal
FK*	product_id	→ biz_product
FK*	fulfilled_payment_id	→ biz_payment
FK*	sharer_user_id	→ sys_user
	version	int

unique / index

uq_biz_purchase_intent_fulfilled_payment (fulfilled_payment_id) partial

biz_order		
PK	id	uuid
FK*	payment_id	→ biz_payment
FK*	user_id	→ sys_user
FK*	deal_id	→ biz_group_deal
FK*	product_id	→ biz_product
	status	varchar
	version	int

unique / index

uq_biz_order_order_no (order_no) partial

uq_biz_order_payment_id (payment_id)

uq_biz_order_deal_user (deal_id, user_id) partial

LJ Liu Jin

ERD 1:1 · notification · 19/21

notification (notification-svc)

biz_notification

PK	id	uuid
FK*	event_id	→ event id
FK*	user_id	→ sys_user
FK*	reference_id	→ deal / payment

unique / index

uq_biz_notification_event_user
(event_id, user_id)

sys_consumed_message

PK	event_id	uuid
PK	consumer	varchar

LJ Liu Jin

ERD 1:1 · integration · scheduling (left) · 20/21

integration · scheduling (shared)

sys_outbox

PK	id	uuid = eventId
FK*	aggregate_id	→ any aggregate
	published_at	timestamptz
	attempts	int

unique / index

idx_sys_outbox_unpublished partial

idx_sys_outbox_published_retention partial

sys_scheduler_lock

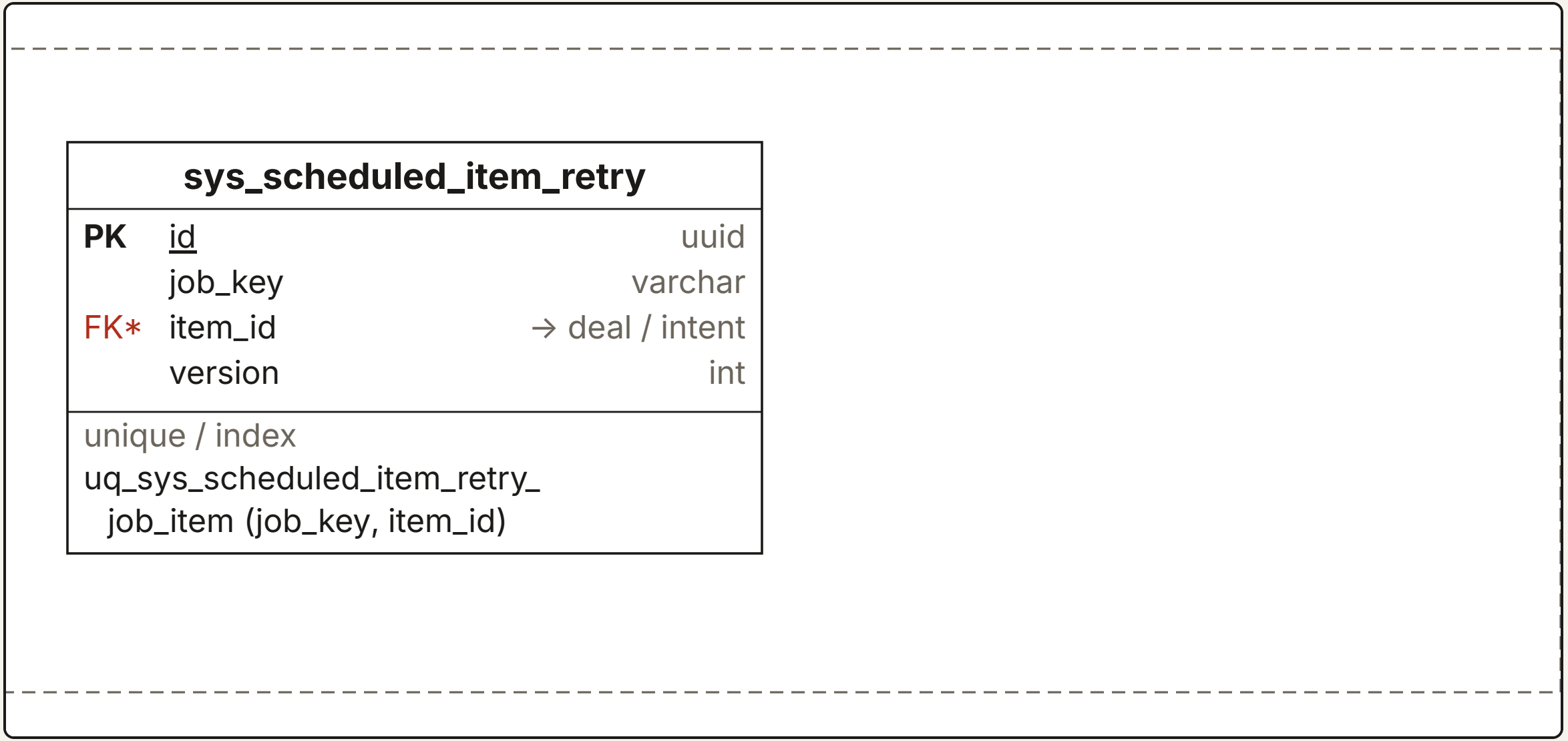
PK	id	uuid
	lock_key	varchar
	locked_until	timestamptz
	version	int

unique / index

uq_sys_scheduler_lock_key (lock_key) partial

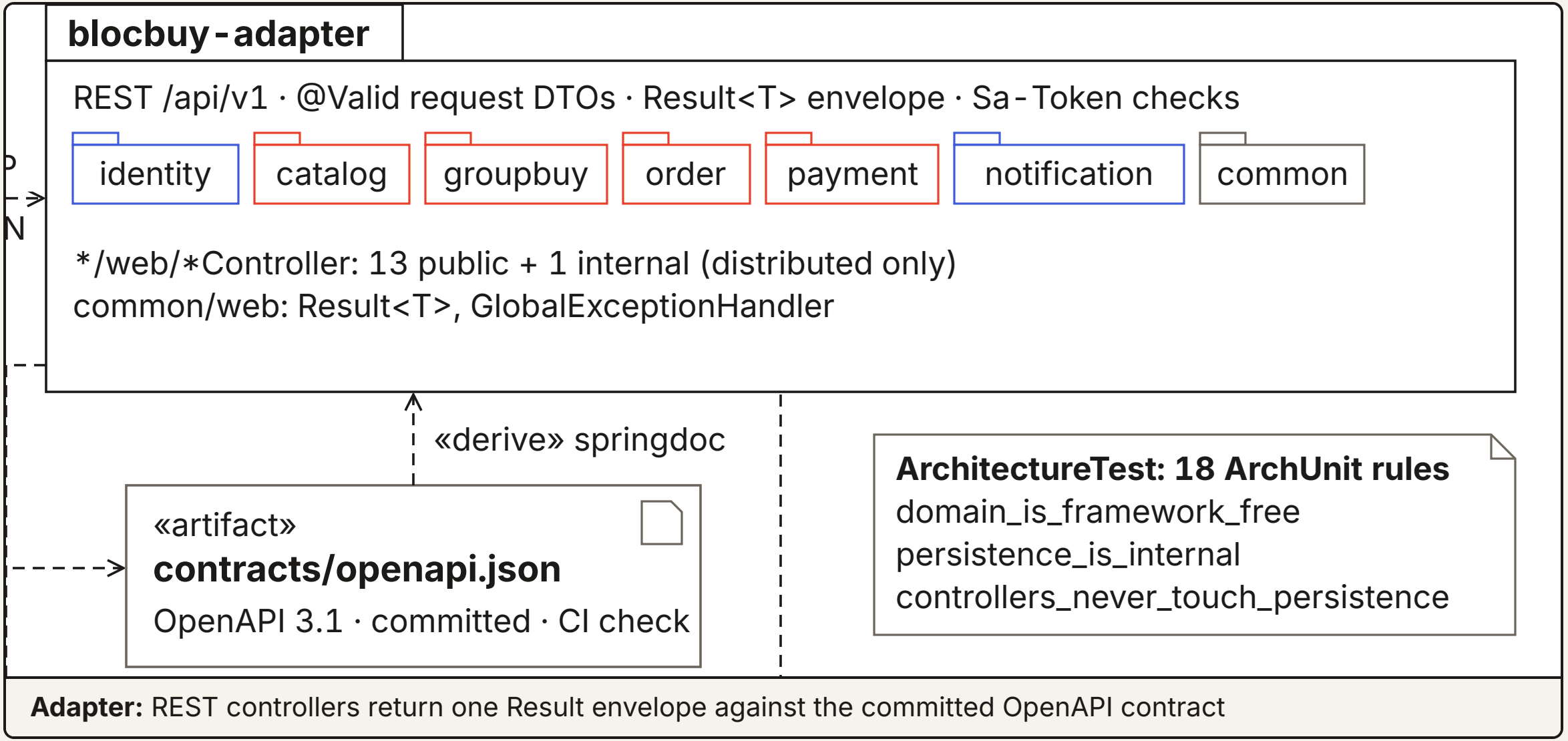
LJ Liu Jin

ERD 1:1 · integration · scheduling (right) · 21/21



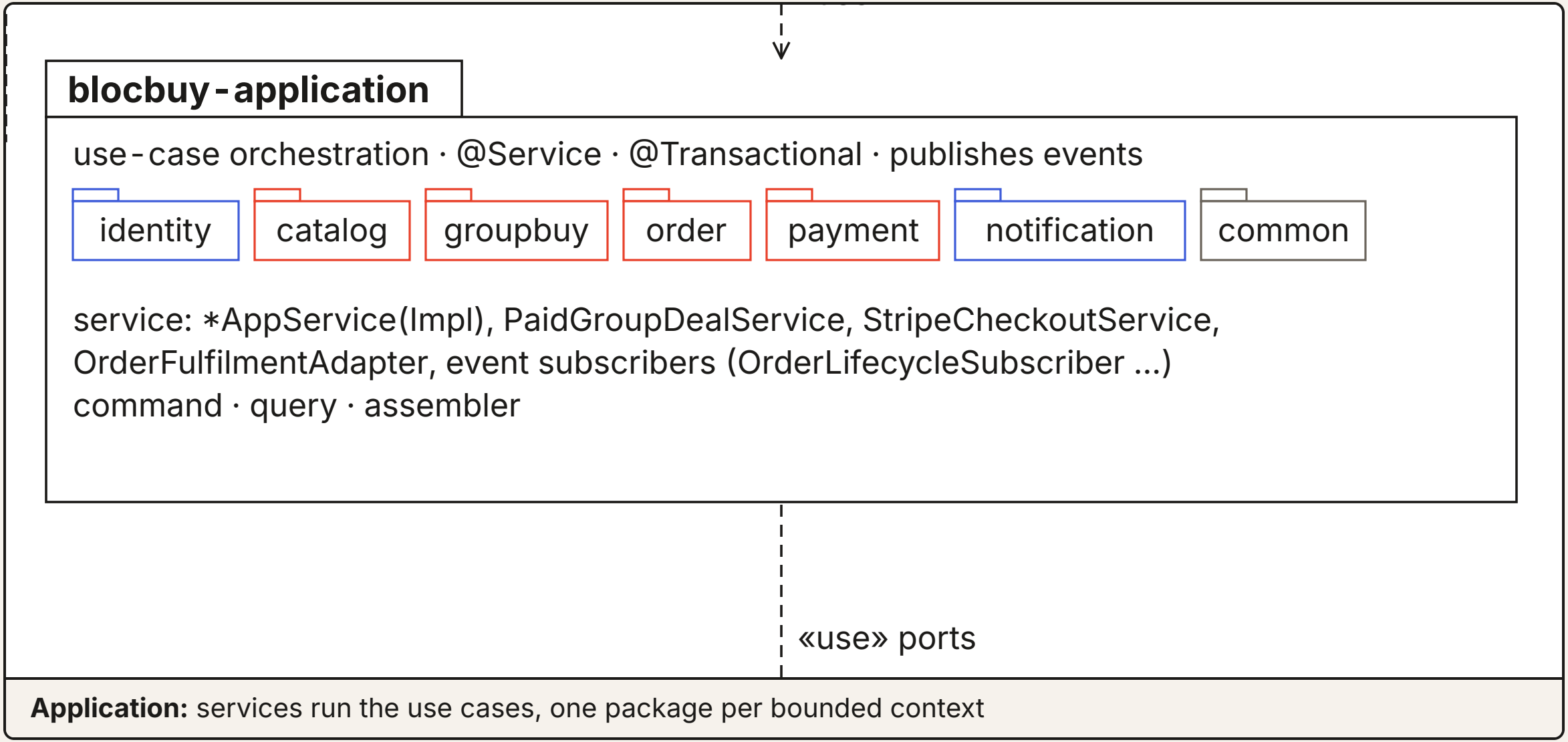
WZ Eric

Slide 9 · detail 1/3: adapter · contract



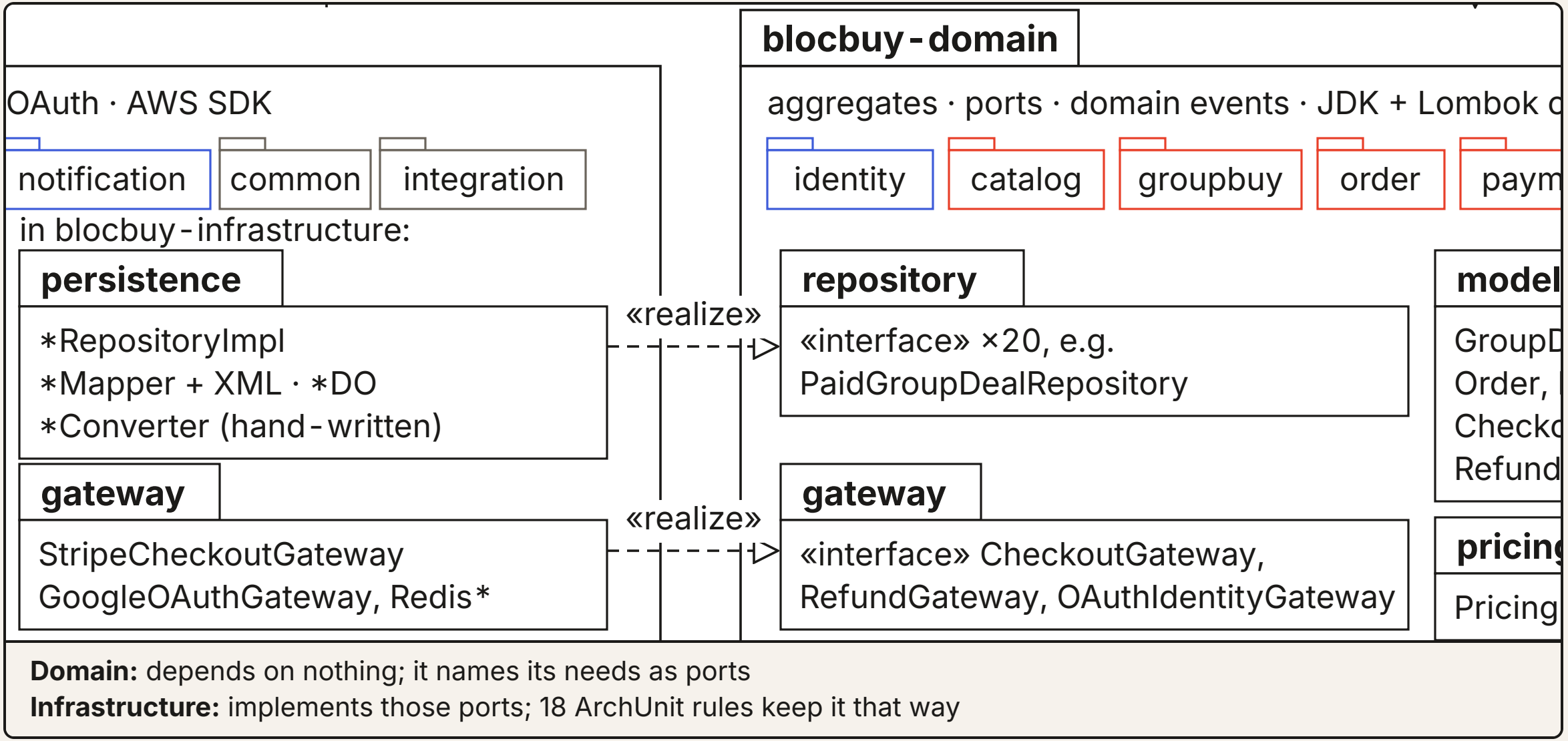
WZ Eric

Slide 9 · detail 2/3: application



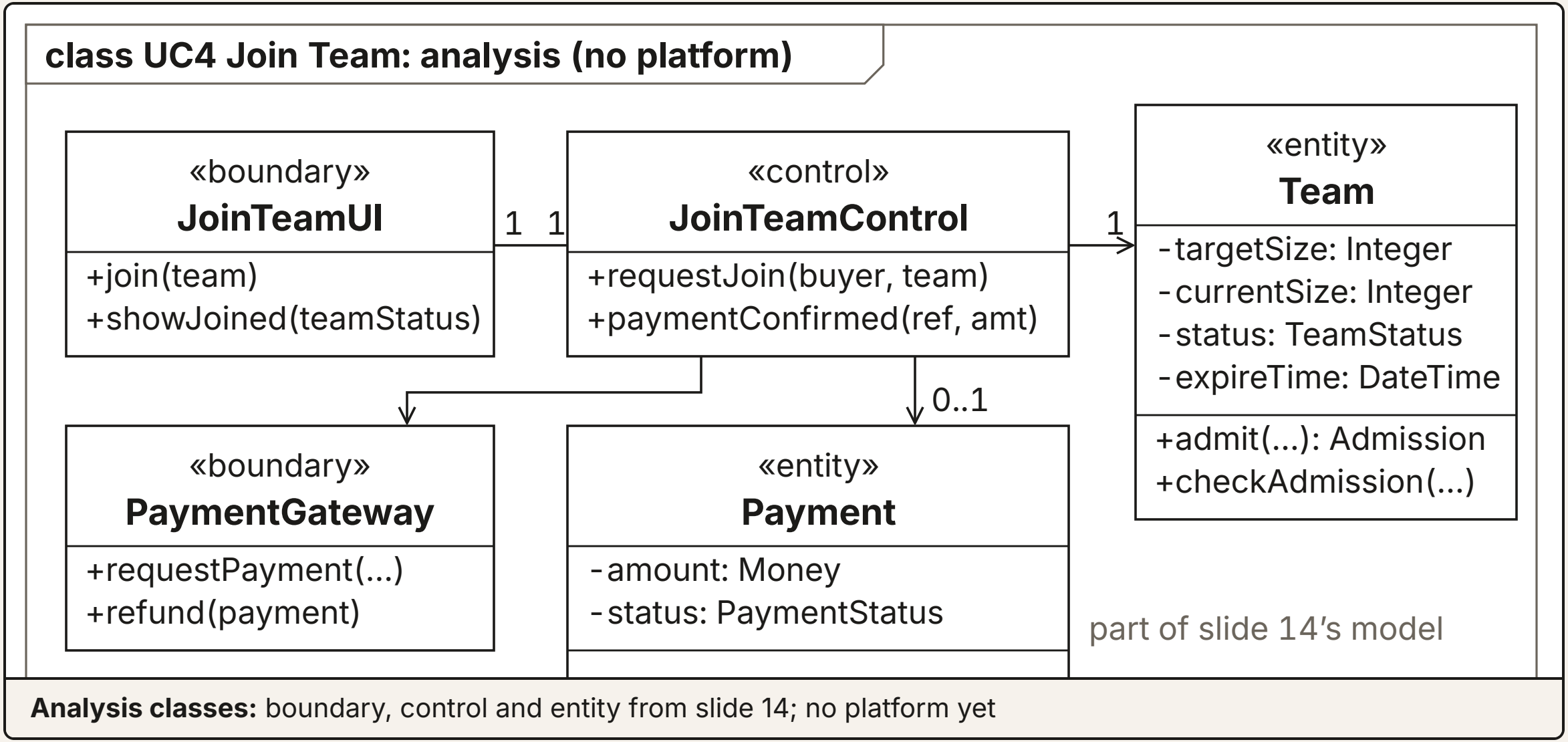
WZ Eric

Slide 9 · detail 3/3: infrastructure realises the domain ports



WZ Eric

Slide 12 · detail 1/3: analysis: UC4



Analysis classes: boundary, control and entity from slide 14; no platform yet

WZ Eric

Slide 12 · detail 2/3: strategy key

Transition strategies applied (tag = strategy)

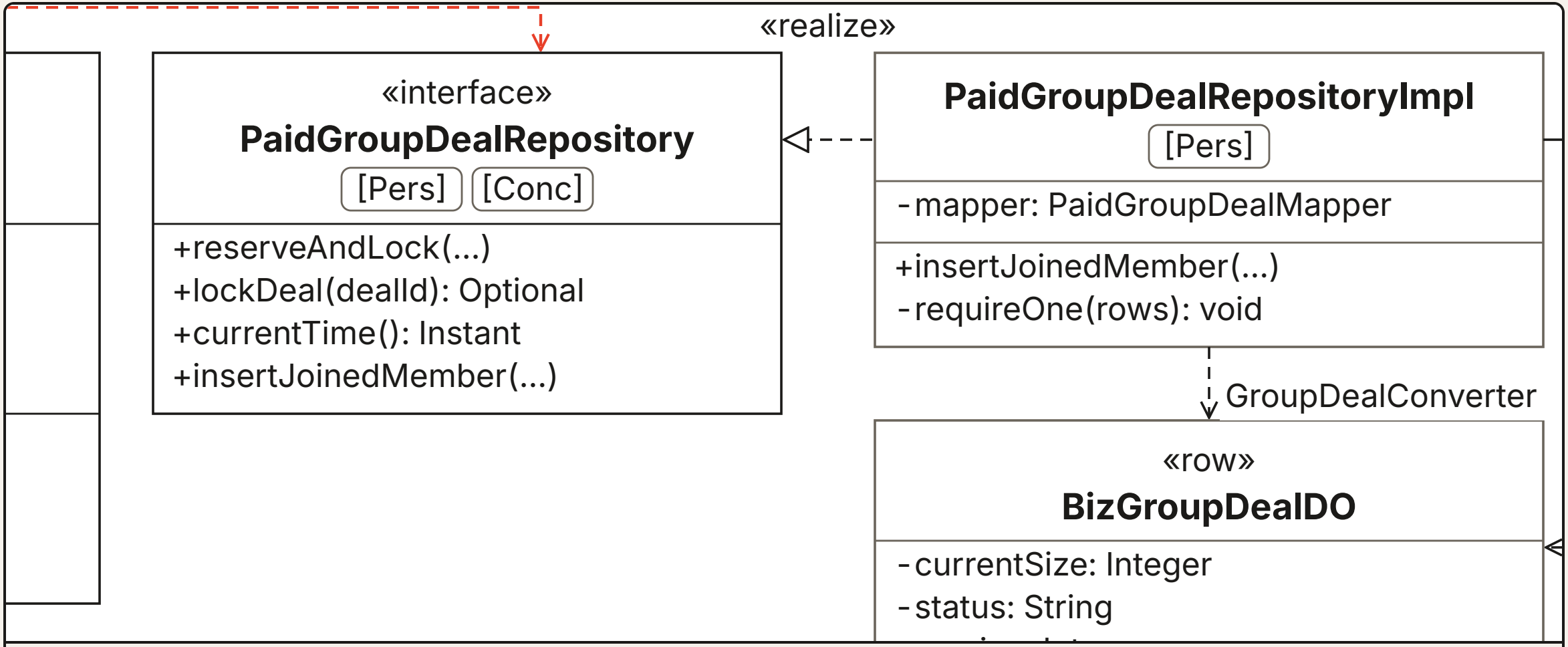
[Pres]	Presentation	JoinTeamUI → route + Query hook
[Dist]	Distribution	control interface → stub + @RestController
[Pers]	Persistence	Team → GroupDeal + port ← MyBatis impl
[Ext]	External interface	PaymentGateway → CheckoutGateway ← adapter
[Conc]	Concurrency	requestJoin → lock order + guarded UPDATE
[Evt]	Domain events	team full → TeamSucceeded published
[Sec]	Security	CSRF header, rate limit, session check
[Lang]	Language & framework	Spring beans, immutable Java aggregate

Payment entity → CheckoutPayment + CheckoutPaymentRepository
(payment context, same [Pers] shape; not drawn)

Strategy key: each tag names the transition strategy that produced a design class

WZ Eric

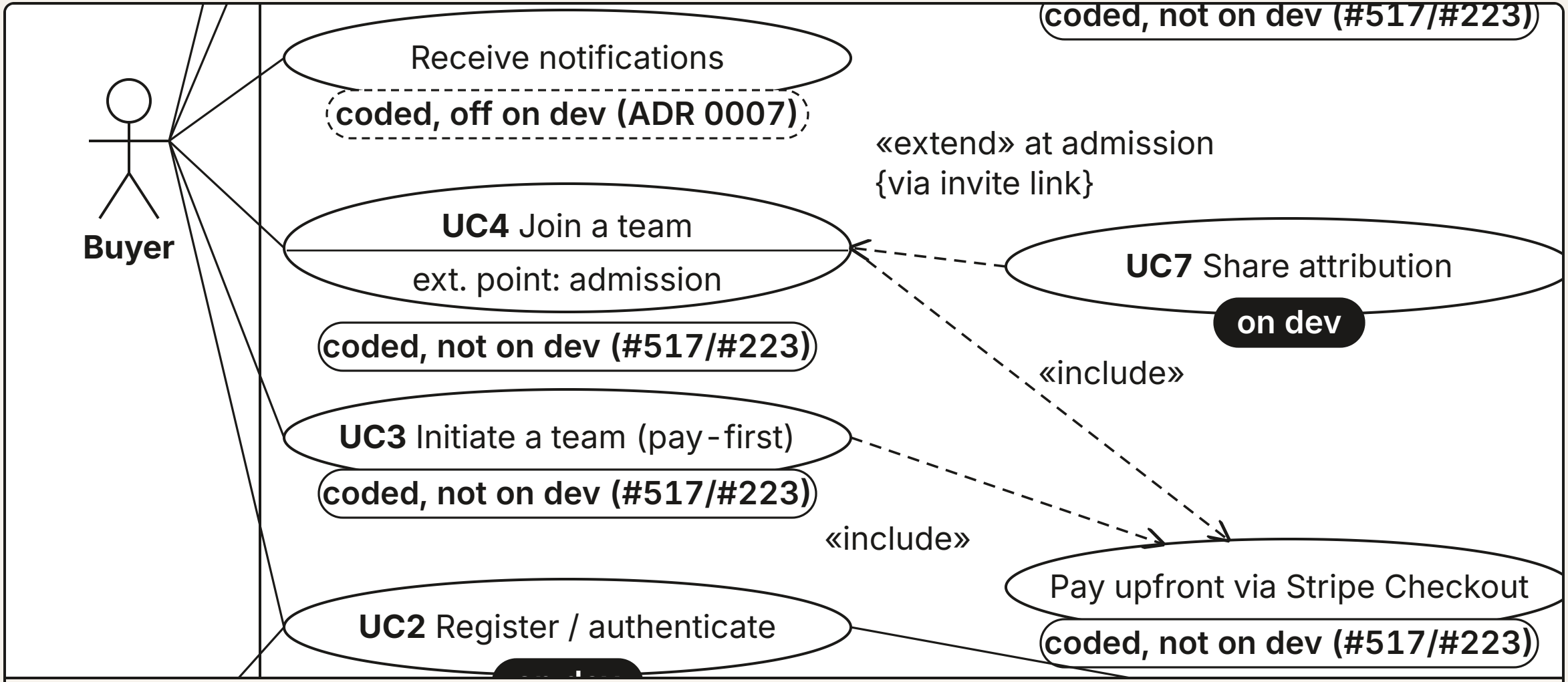
Slide 12 · detail 3/3: design classes · persistence



Design classes: the Team becomes the GroupDeal aggregate, reached through its repository port
◁ **means "realises":** MyBatis classes in infrastructure implement that port

WZ Eric

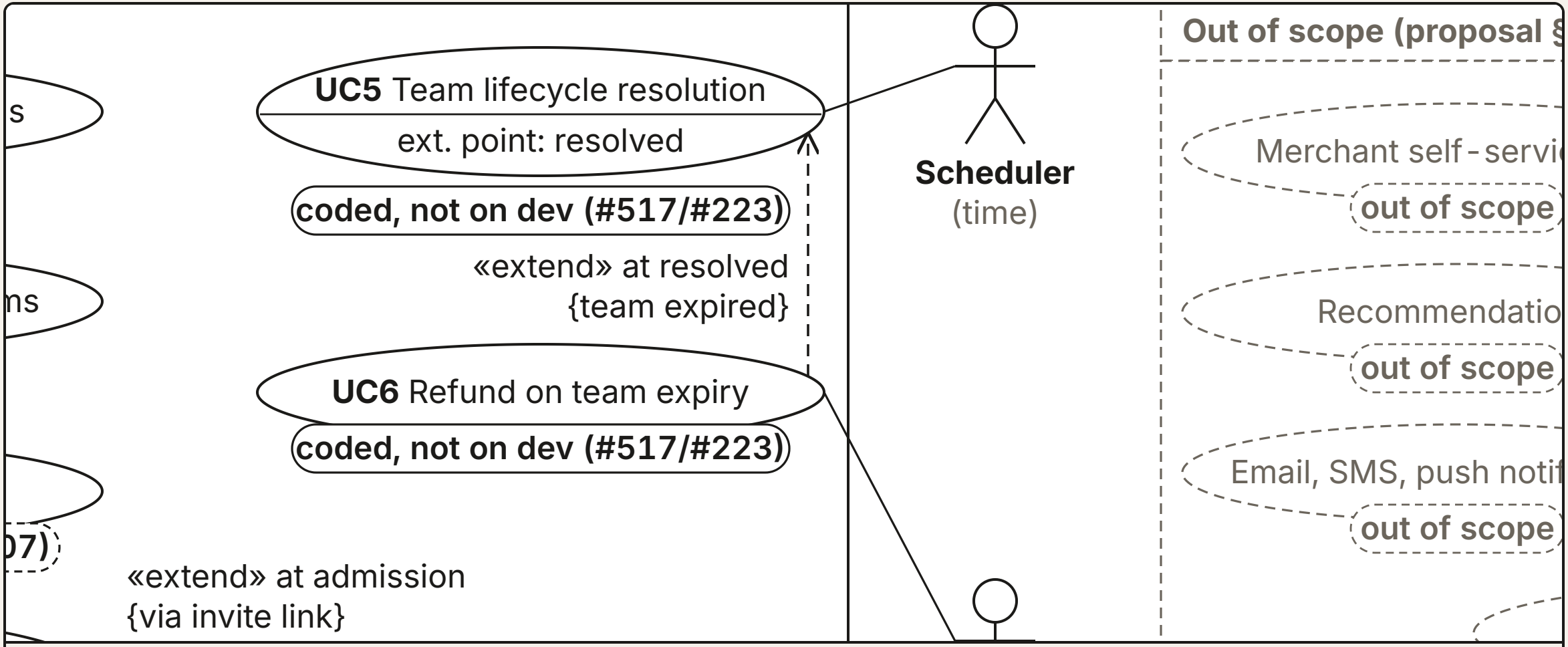
Slide 13 · detail 1/5: Buyer and the money path



In scope: start and join «include» paying upfront; invite attribution «extend»s join

WZ Eric

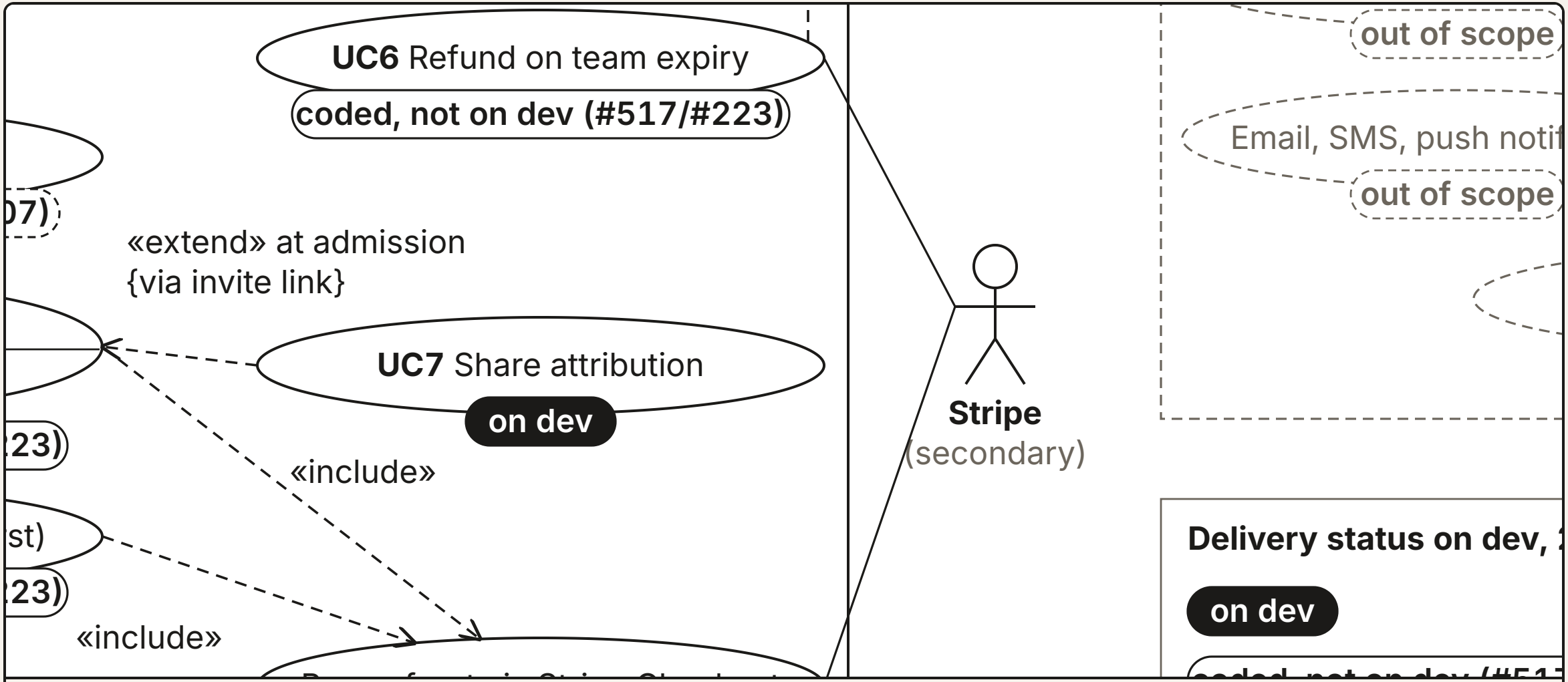
Slide 13 · detail 2/5: resolution: Scheduler



Scheduler: a time-driven actor resolves teams
«extend»: refund extends resolution when a team expires

WZ Eric

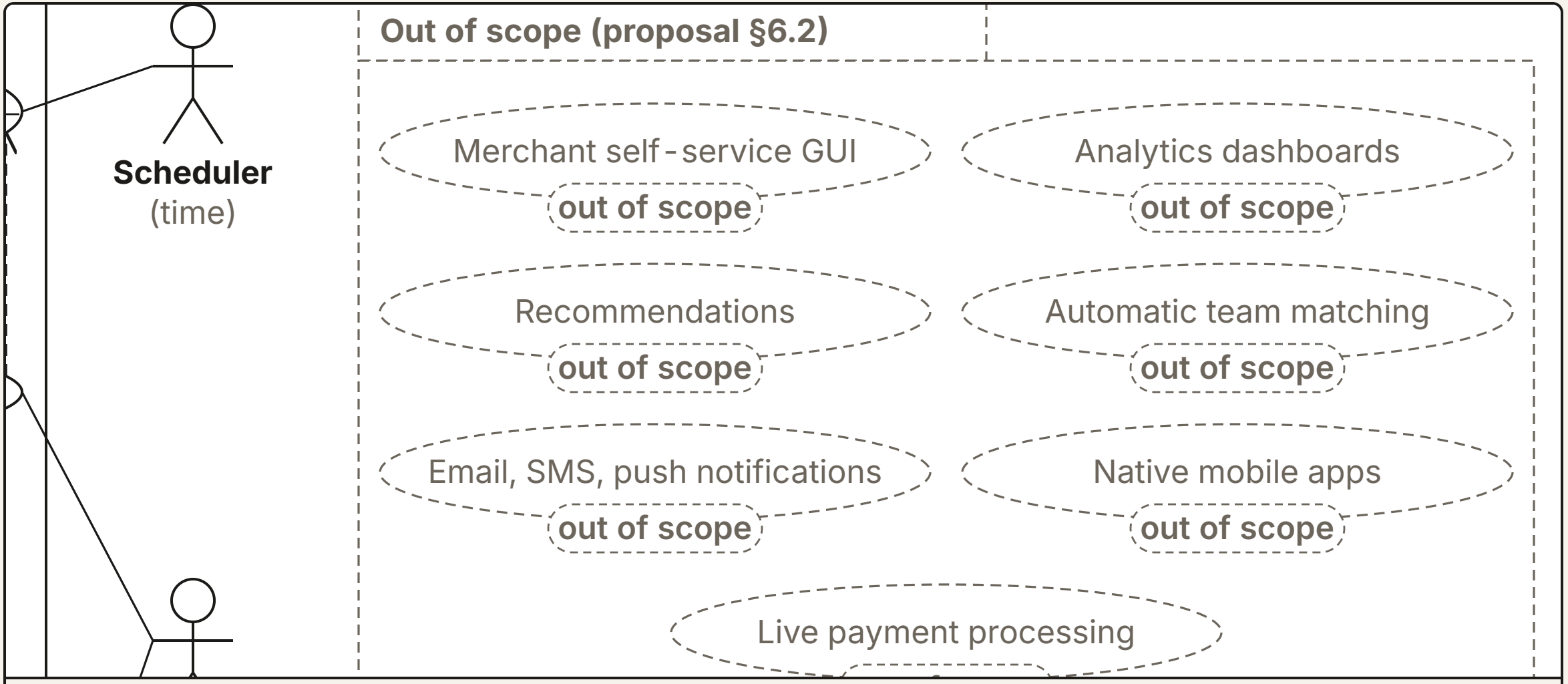
Slide 13 · detail 3/5: refund, payment: Stripe



Stripe: a secondary actor of refund and of paying upfront

WZ Eric

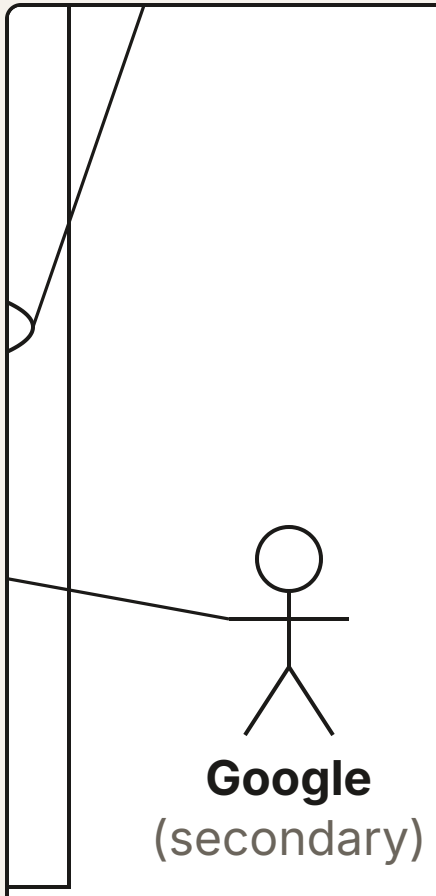
Slide 13 · detail 4/5: out of scope



Out of scope, greyed: as the proposal excluded

WZ Eric

Slide 13 · detail 5/5: delivery status



Delivery status on dev, 21 Oct (team ruling)

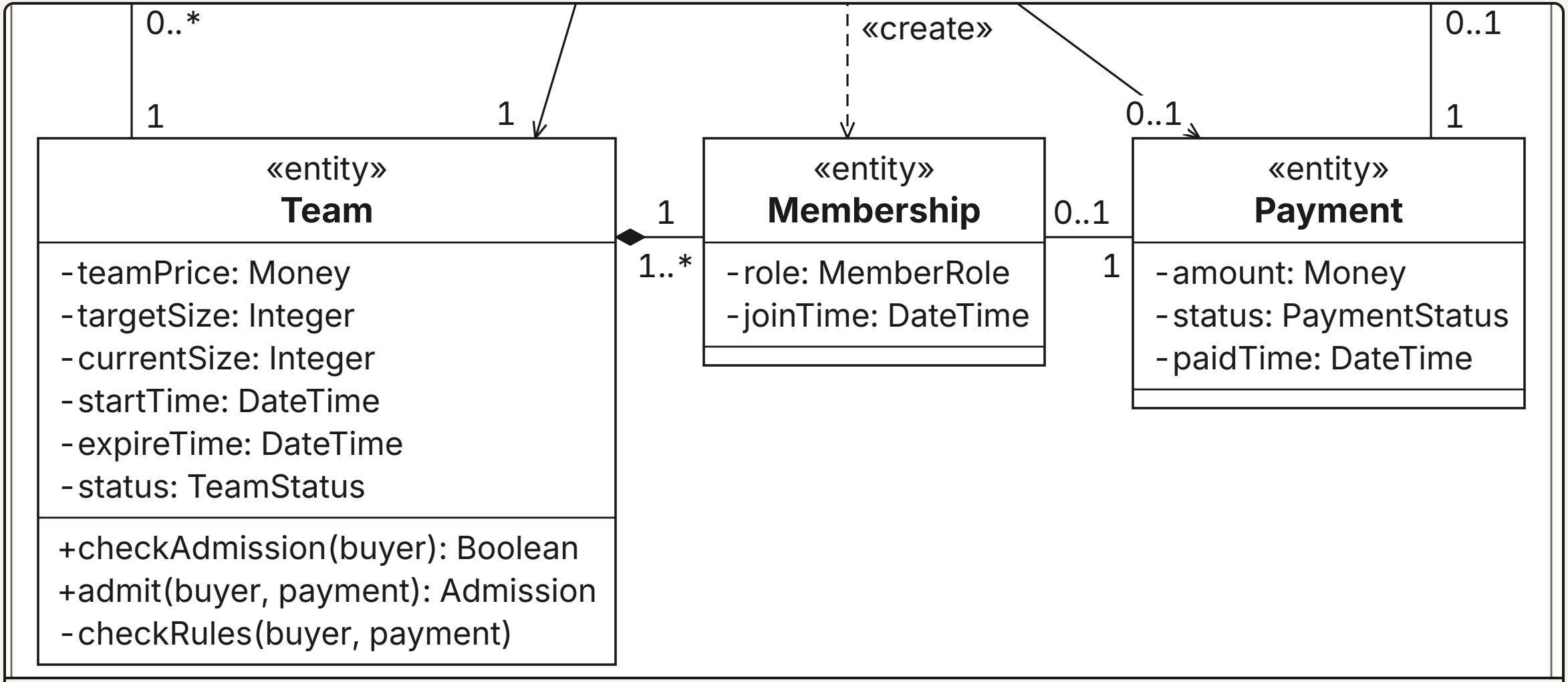
on dev	deployed and usable on dev
coded, not on dev (#517/#223)	built, tested; payments blocked
coded, off on dev (ADR 0007)	SQS consumer runs on SWE5001 only
out of scope	excluded by the proposal

- - - - - - → «include»: base → included
«extend»: extending → base

Status badges: the money path is coded, not yet working on dev

WZ Eric

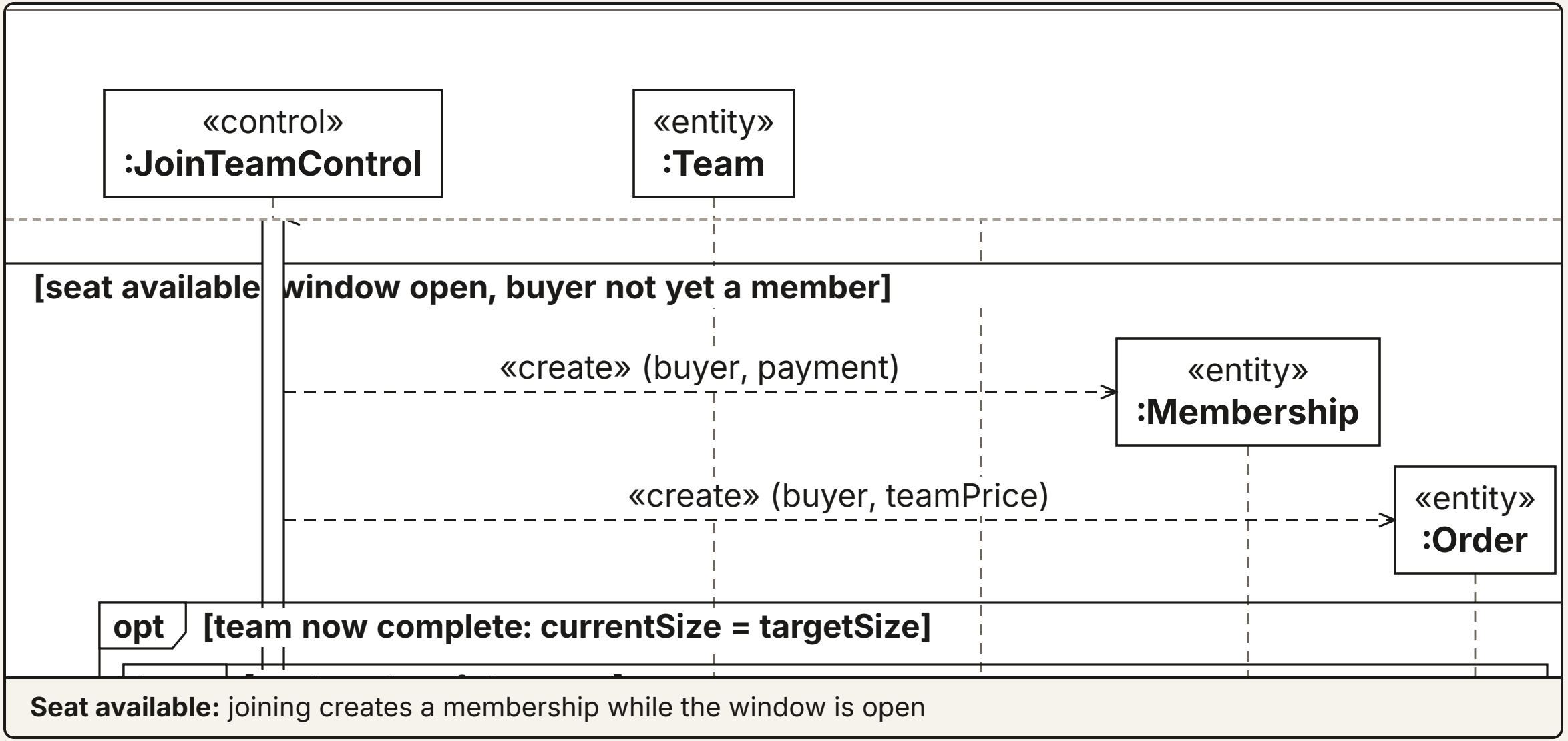
Slide 14 · detail 1/3: entities



Entities only: attributes sit on entities: the Team holds its size and deadline

WZ Eric

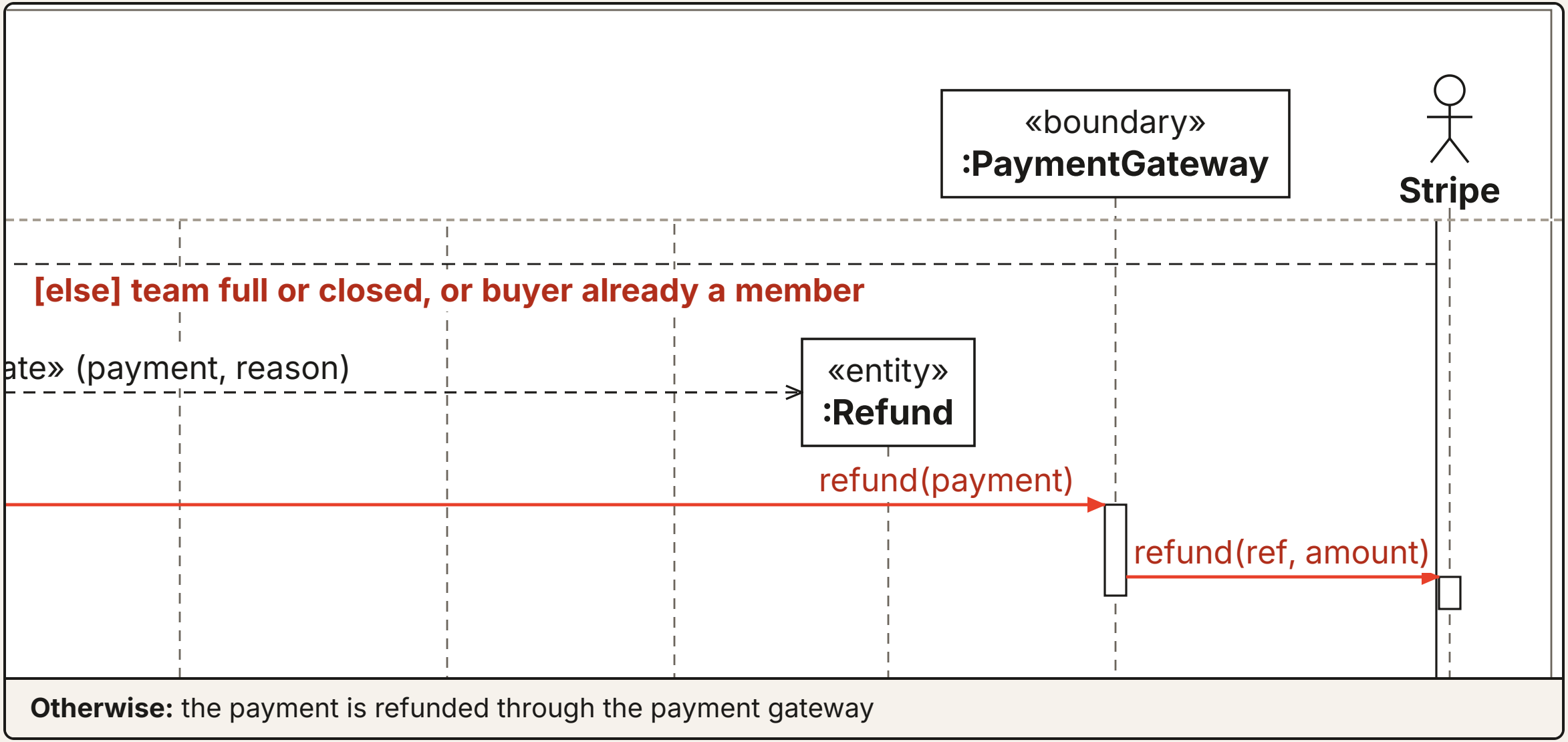
Slide 14 · detail 2/3: seat available: join



Seat available: joining creates a membership while the window is open

WZ Eric

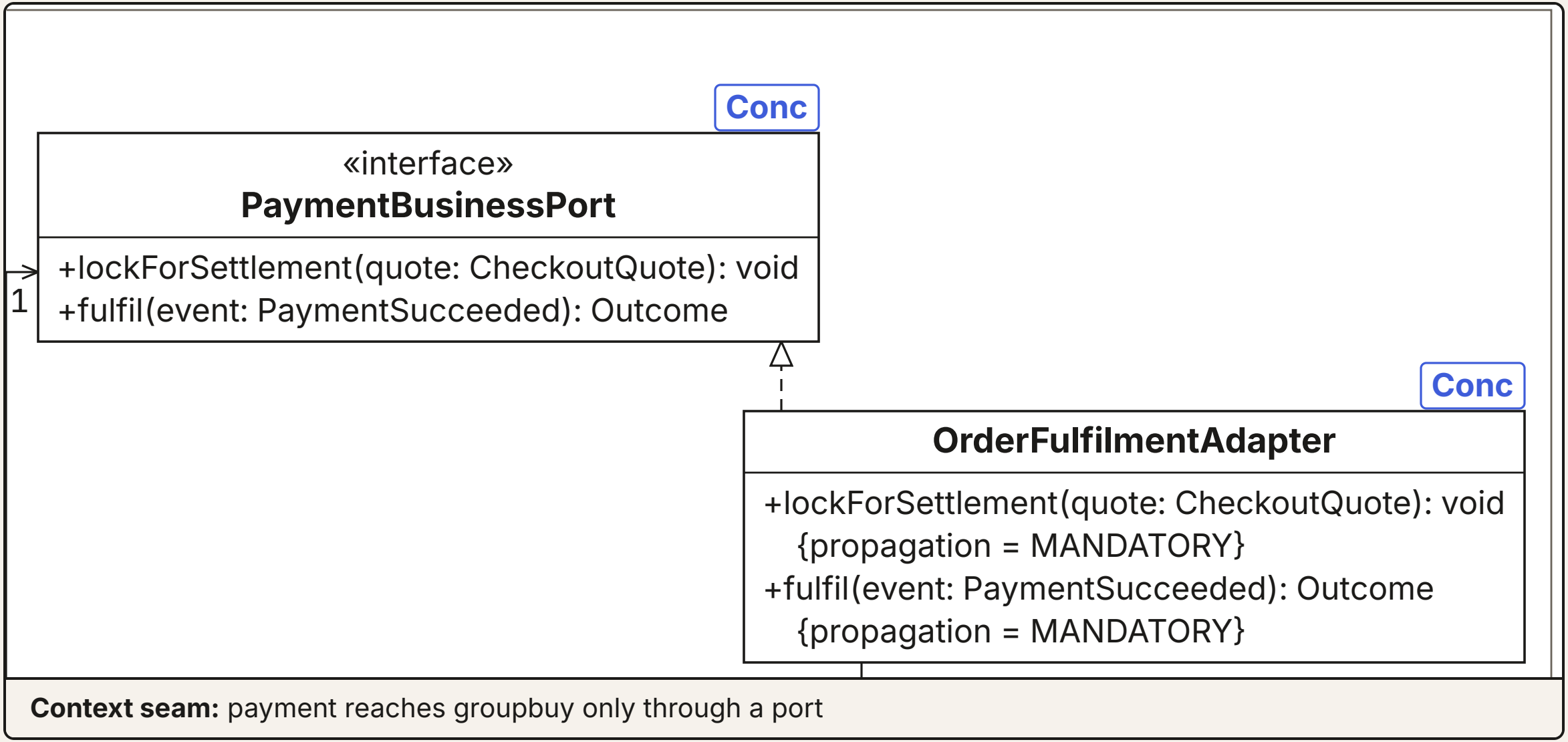
Slide 14 · detail 3/3: else: refund through the gateway



Otherwise: the payment is refunded through the payment gateway

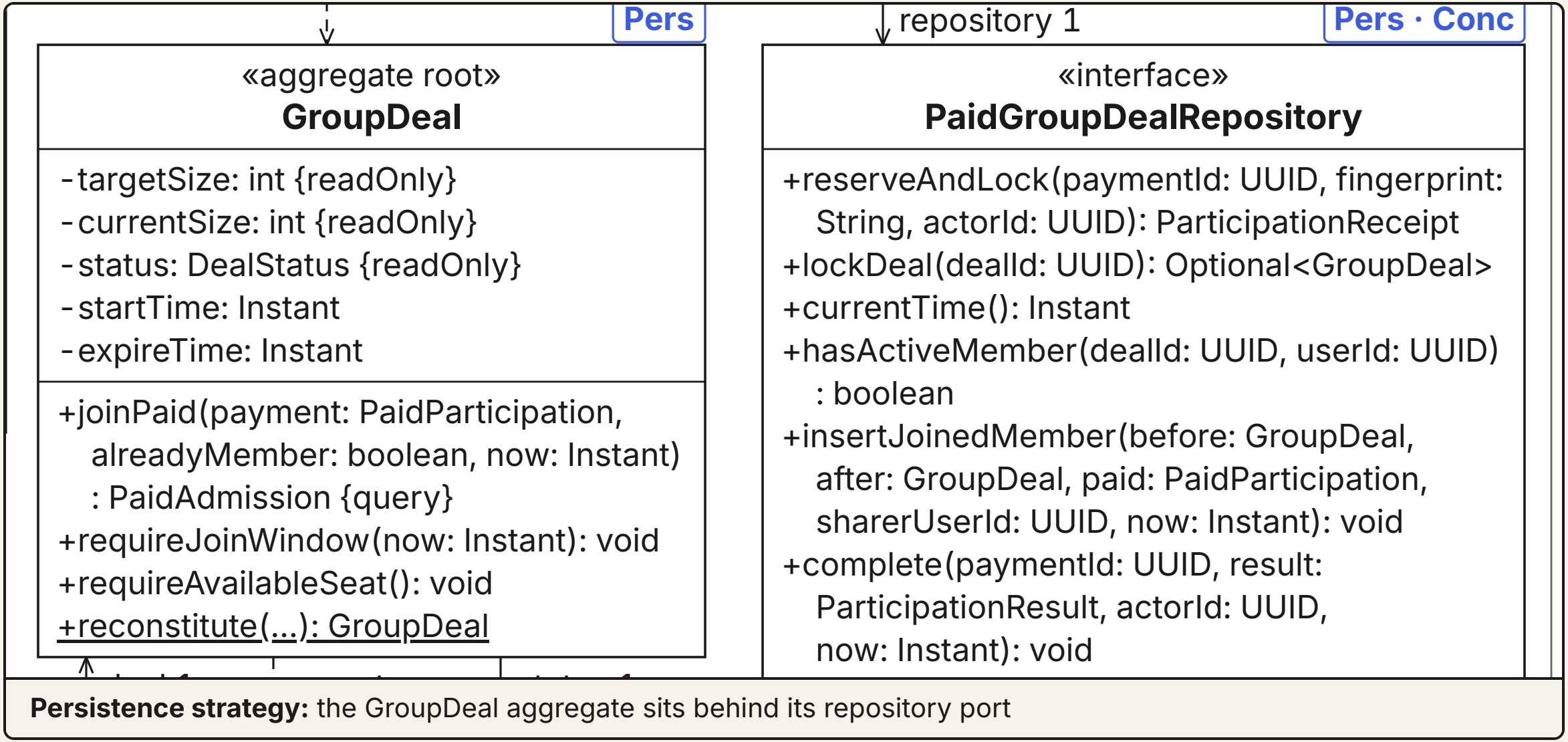
WZ Eric

Slide 15 · detail 1/3: payment meets groupbuy through a port



WZ Eric

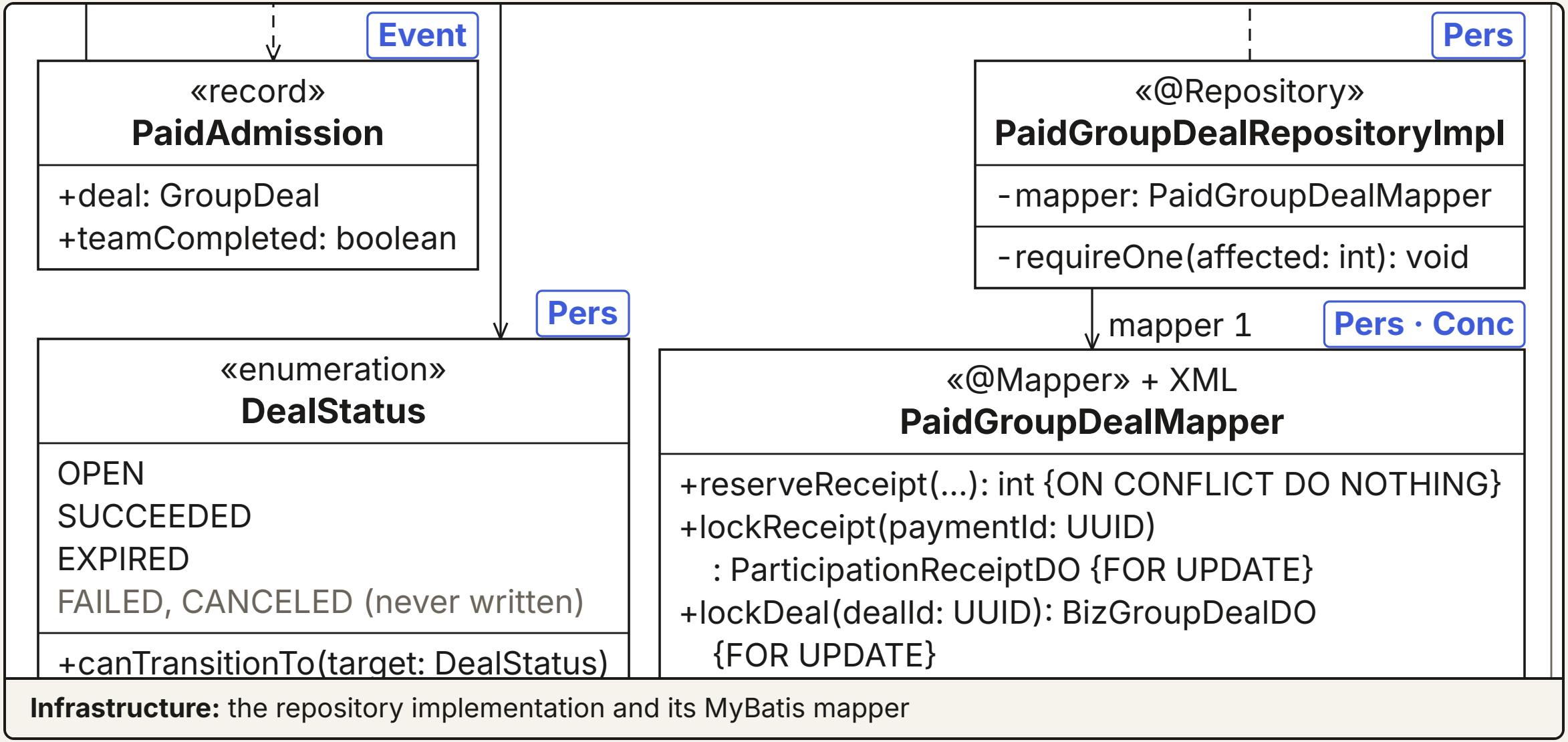
Slide 15 · detail 2/3: aggregate behind its repository port



Persistence strategy: the GroupDeal aggregate sits behind its repository port

WZ Eric

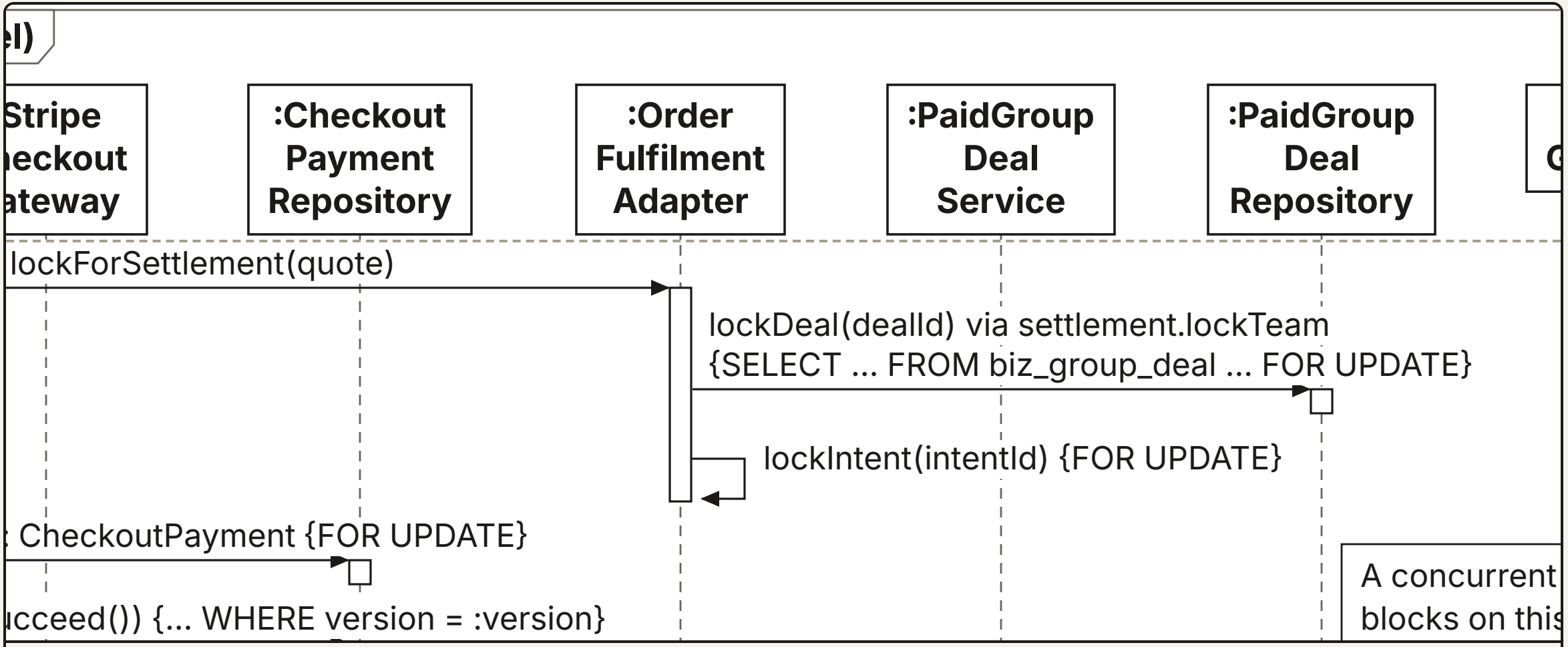
Slide 15 · detail 3/3: MyBatis implementation of the port



Infrastructure: the repository implementation and its MyBatis mapper

WZ Eric

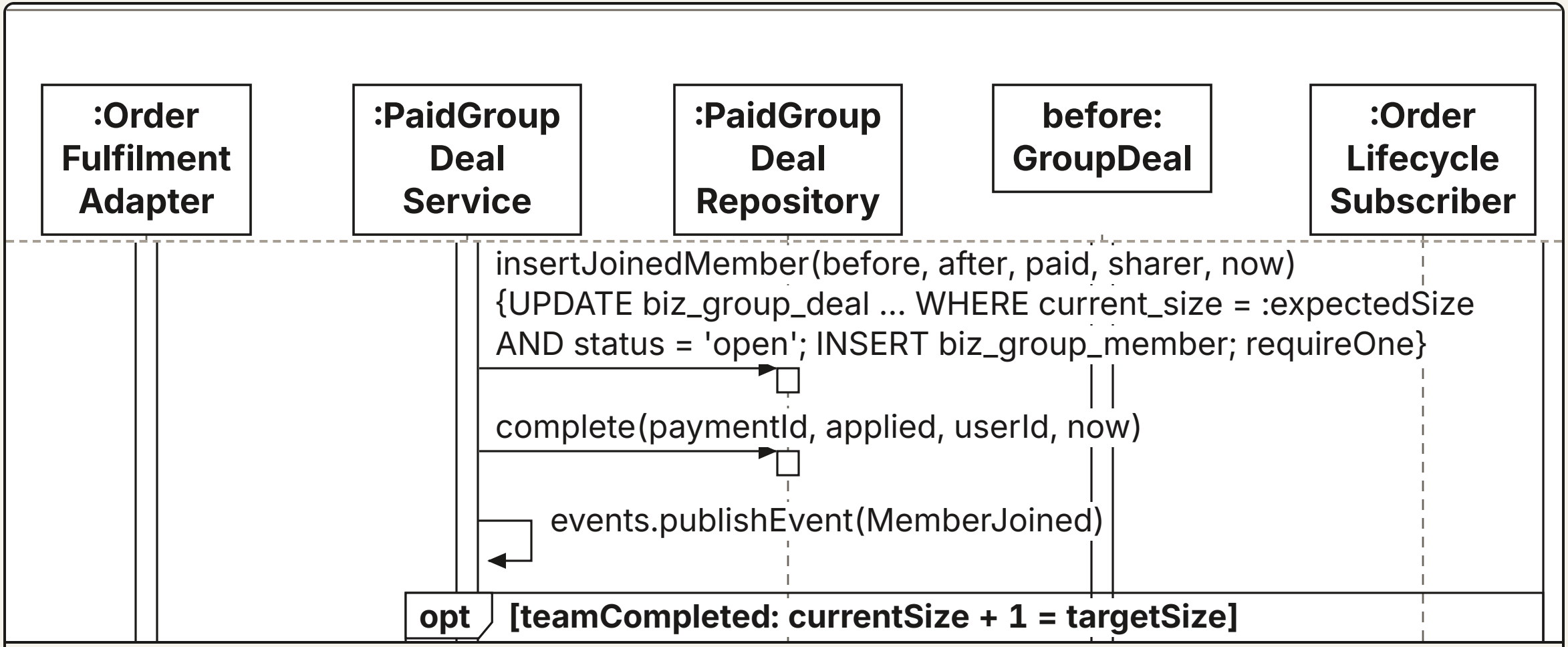
Slide 16 · detail 1/3: team row locked first



One transaction: after the webhook's signature check; drawn as critical, as UML has no transaction fragment
Lock order: team row, intent, then payment; a racing join waits on the team row

WZ Eric

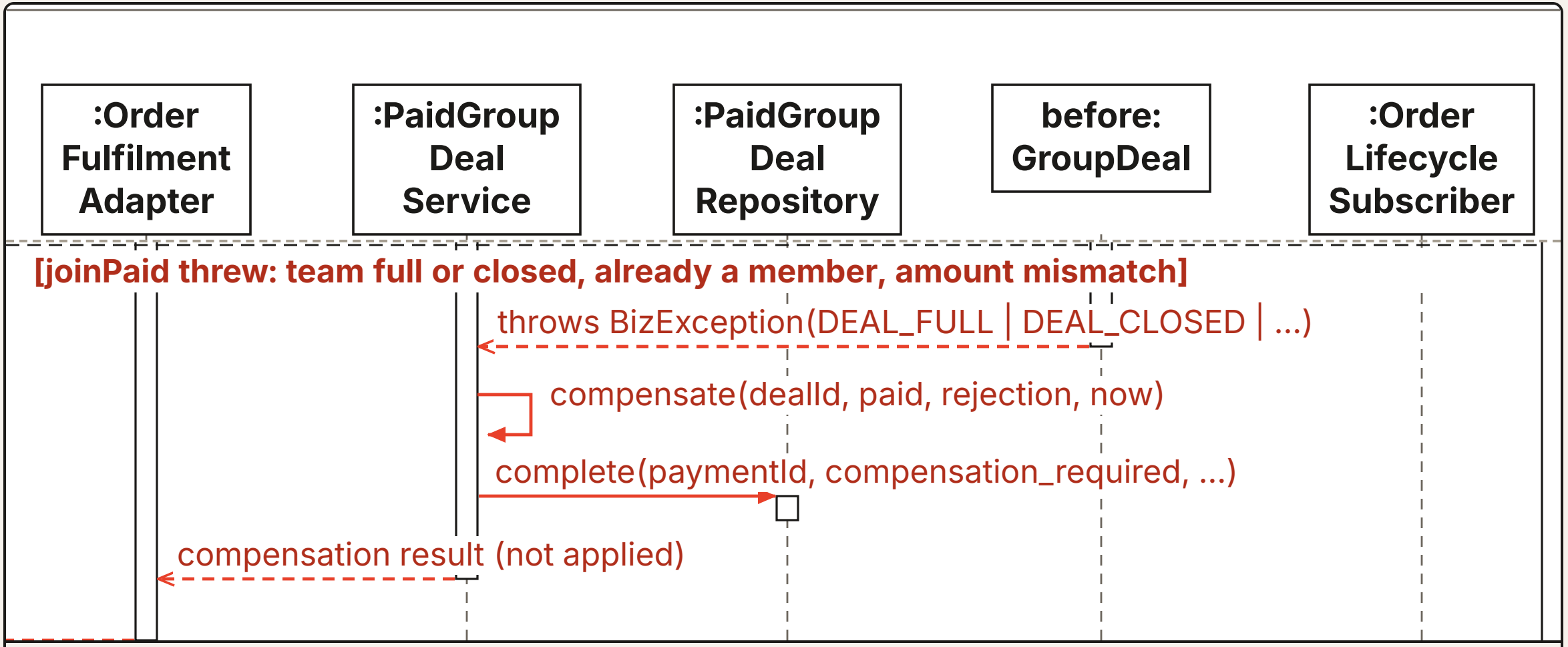
Slide 16 · detail 2/3: guarded update · last seat



Guarded update: `UPDATE … WHERE current_size = expected AND status = open`
Last seat: the opt [teamCompleted] branch publishes `TeamSucceeded`

WZ Eric

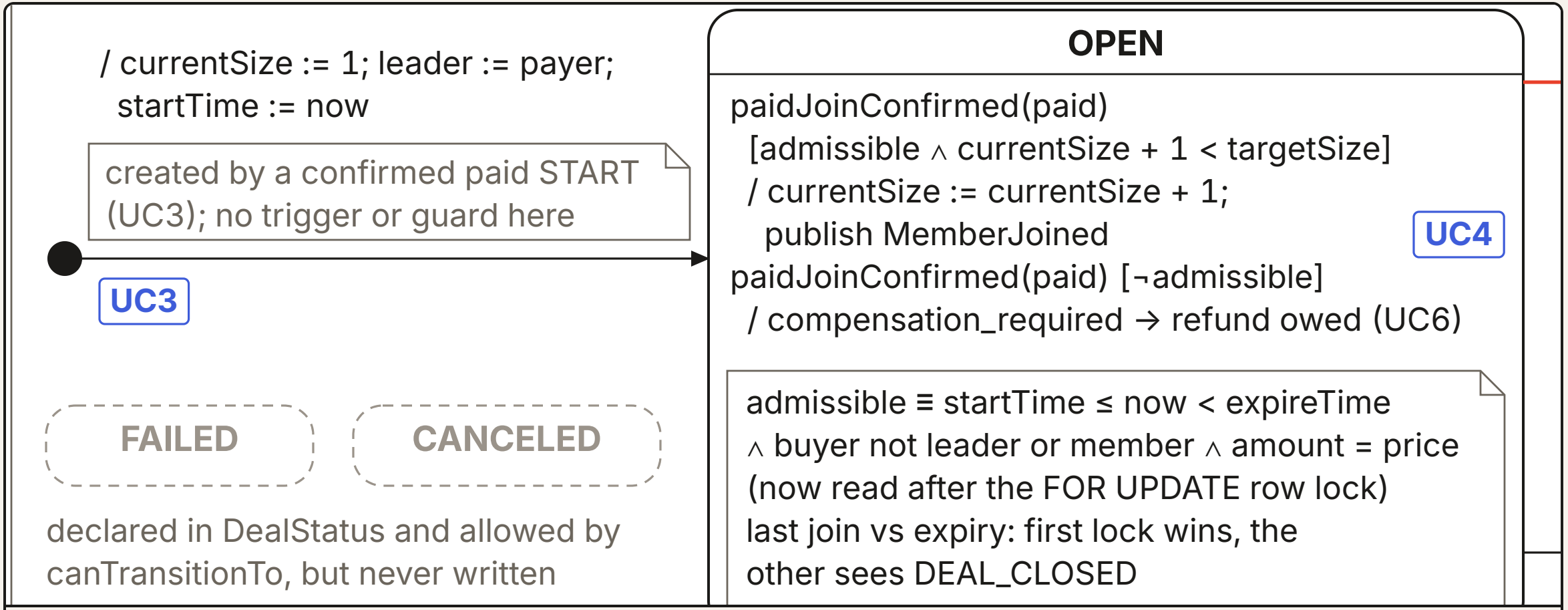
Slide 16 · detail 3/3: loser: compensation, refund owed



Seat gone: DEAL_FULL or DEAL_CLOSED: the receipt owes a refund, never an extra member
Evidence: 20 paid buyers race for 1 seat: 19 refunds owed (local integration test)

WZ Eric

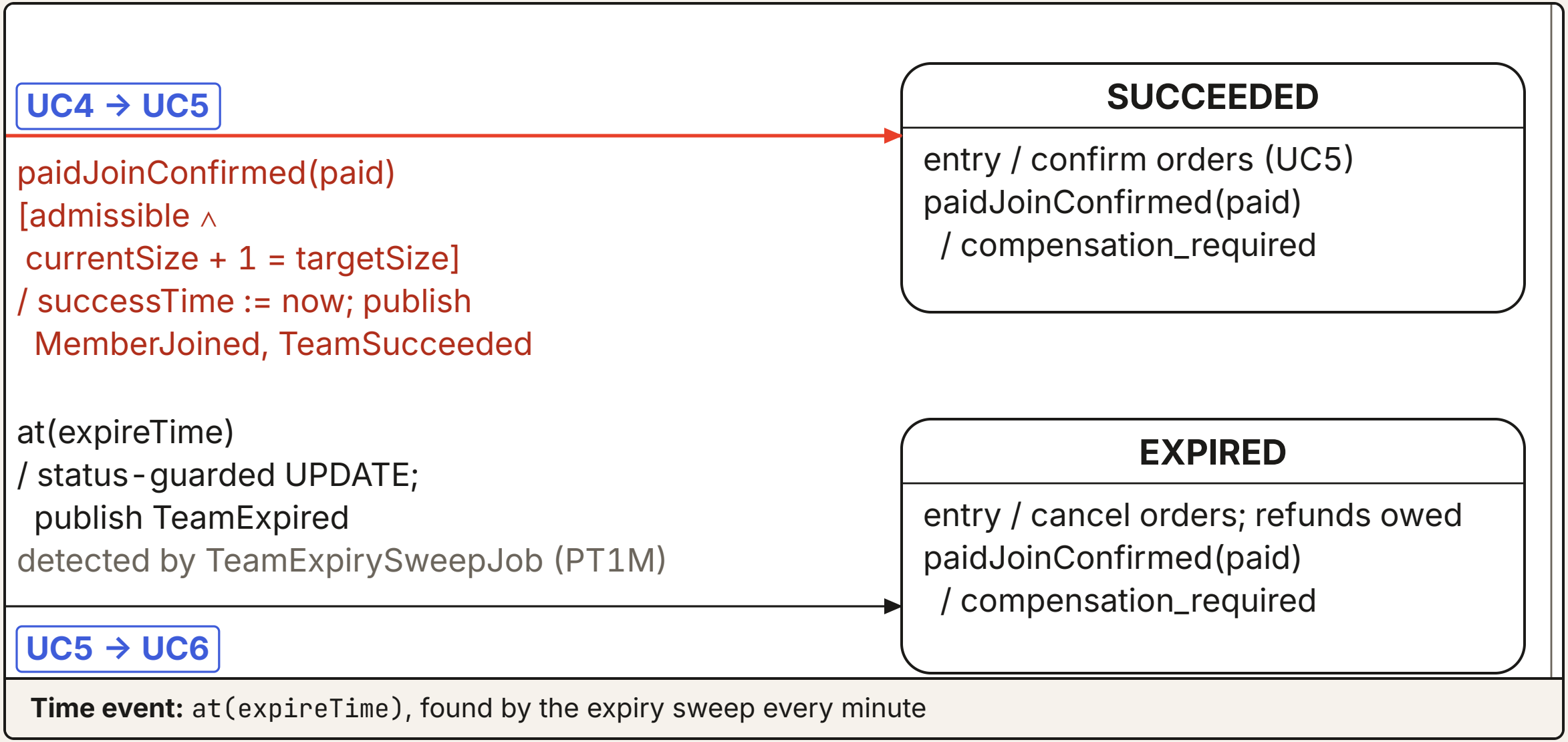
Slide 17 · detail 1/2: OPEN and its joins



Guards: price → new member → window → seat (`GroupDeal.joinPaid`)
Code it matches: `DealStatus.canTransitionTo`: only OPEN moves, never to itself
Greyed: FAILED and CANCELED are declared but never written

WZ Eric

Slide 17 · detail 2/2: succeeded or expired

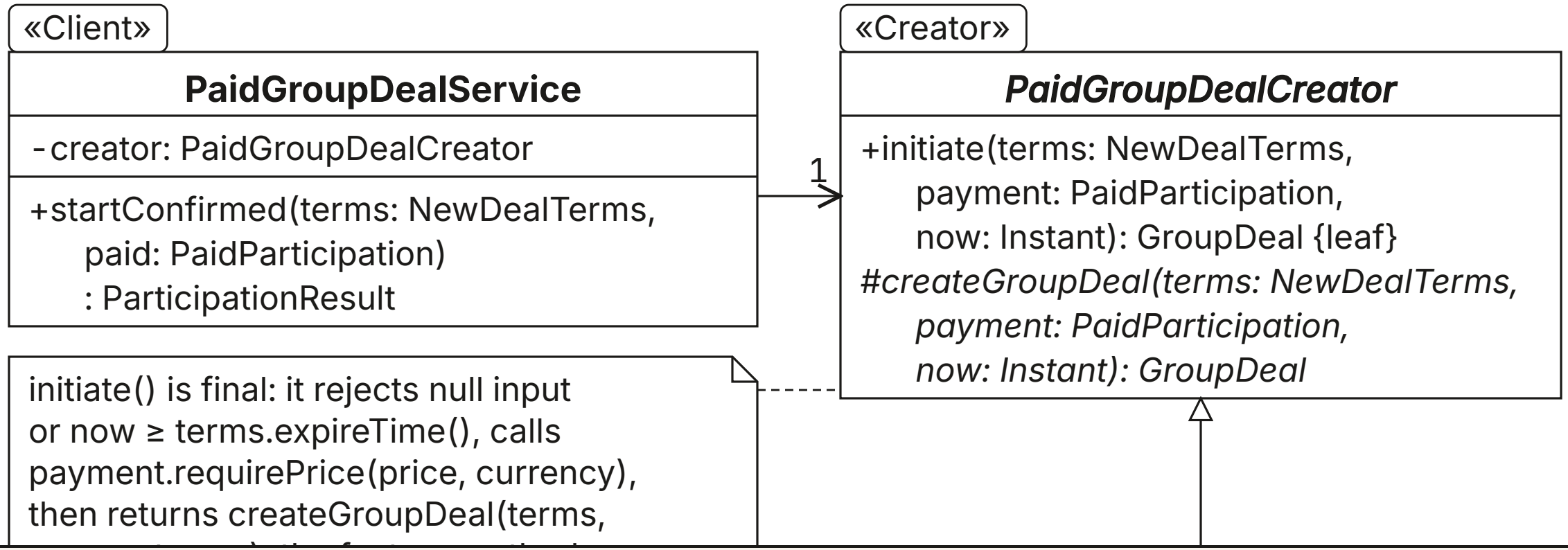


Time event: `at(expireTime)`, found by the expiry sweep every minute

WZ Eric

Slide 18 · detail 1/3: Client · Creator

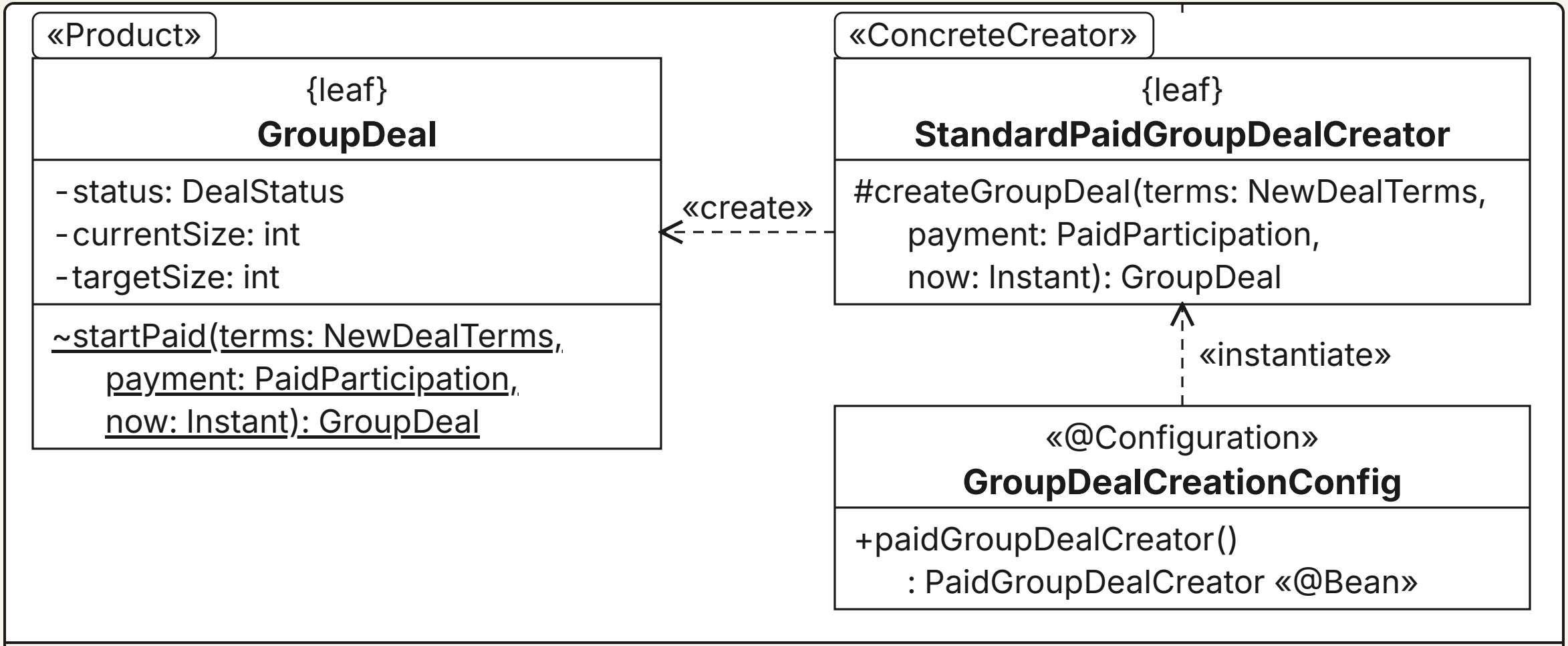
Factory Method: UC3 start creates the team (GroupDeal)



Client: the service depends only on the abstract creator
Needed because: the proposal makes new kinds of team, without editing the service, a design driver

WZ Eric

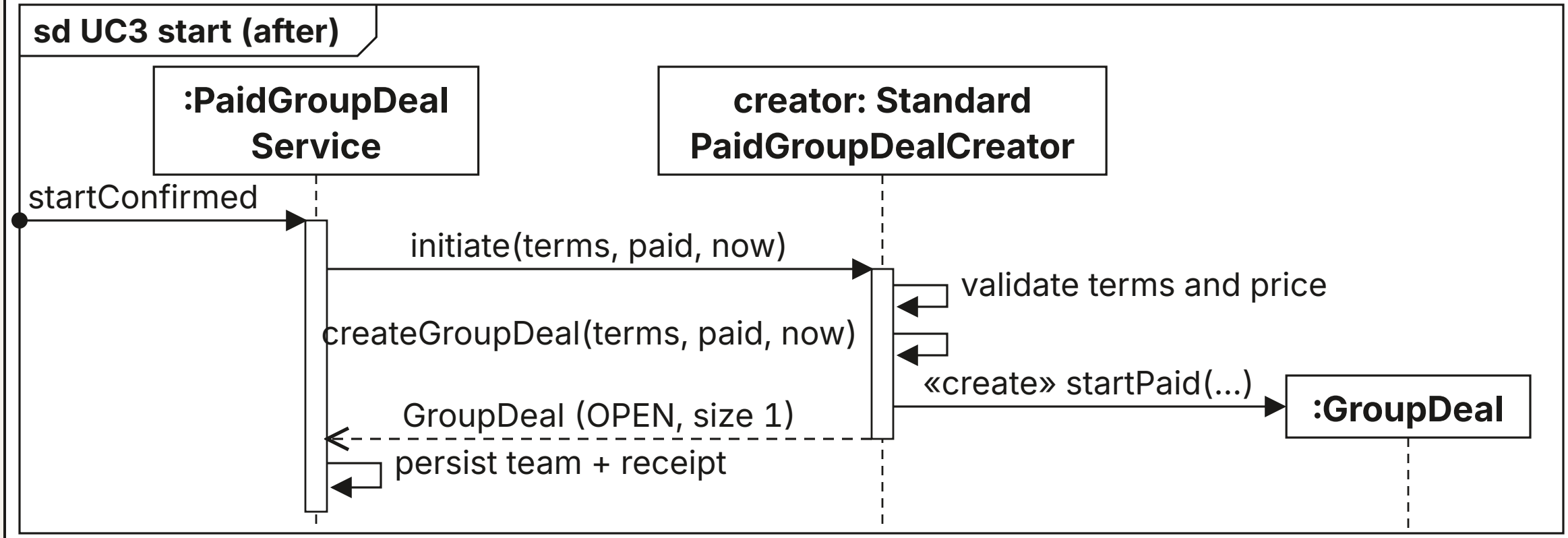
Slide 18 · detail 2/3: ConcreteCreator · Product



Participants: Creator `PaidGroupDealCreator`, ConcreteCreator `StandardPaidGroupDealCreator`, Product `GroupDeal`
Candidates: Factory Method, static factory, Builder, Abstract Factory

WZ Eric

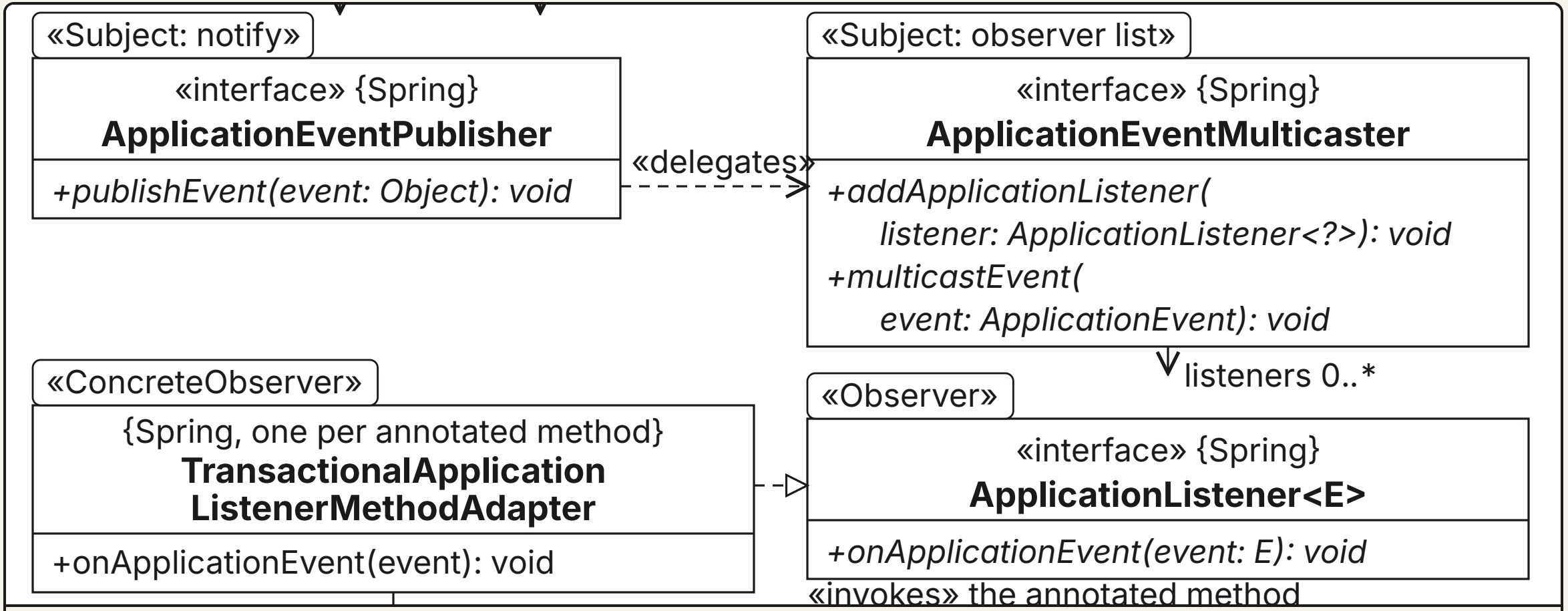
Slide 18 · detail 3/3: initiate → createGroupDeal



Why: the final `initiate` validates, then calls `createGroupDeal`: every creator keeps the rules
Liability: one creator and one team class today
Tested by: `PaidGroupDealServiceTest.startUsesTheInjectedCreator`

YQ Yorik

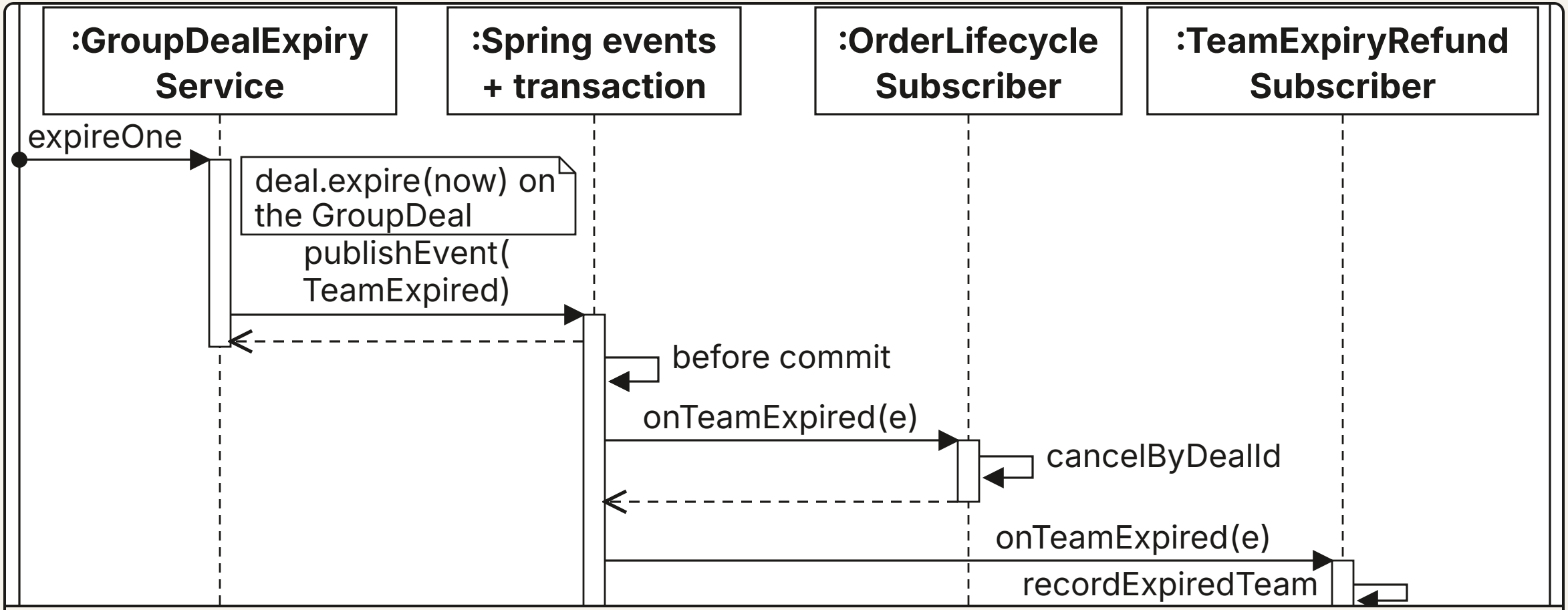
Slide 19 · detail 1/2: Subject · Observer · ConcreteObserver adapter



Subject: Spring's `ApplicationEventPublisher` and its multicaster
ConcreteObserver: Spring's adapter, one per subscriber method
Candidates: Observer, Mediator, direct calls, a message broker

YQ Yorik

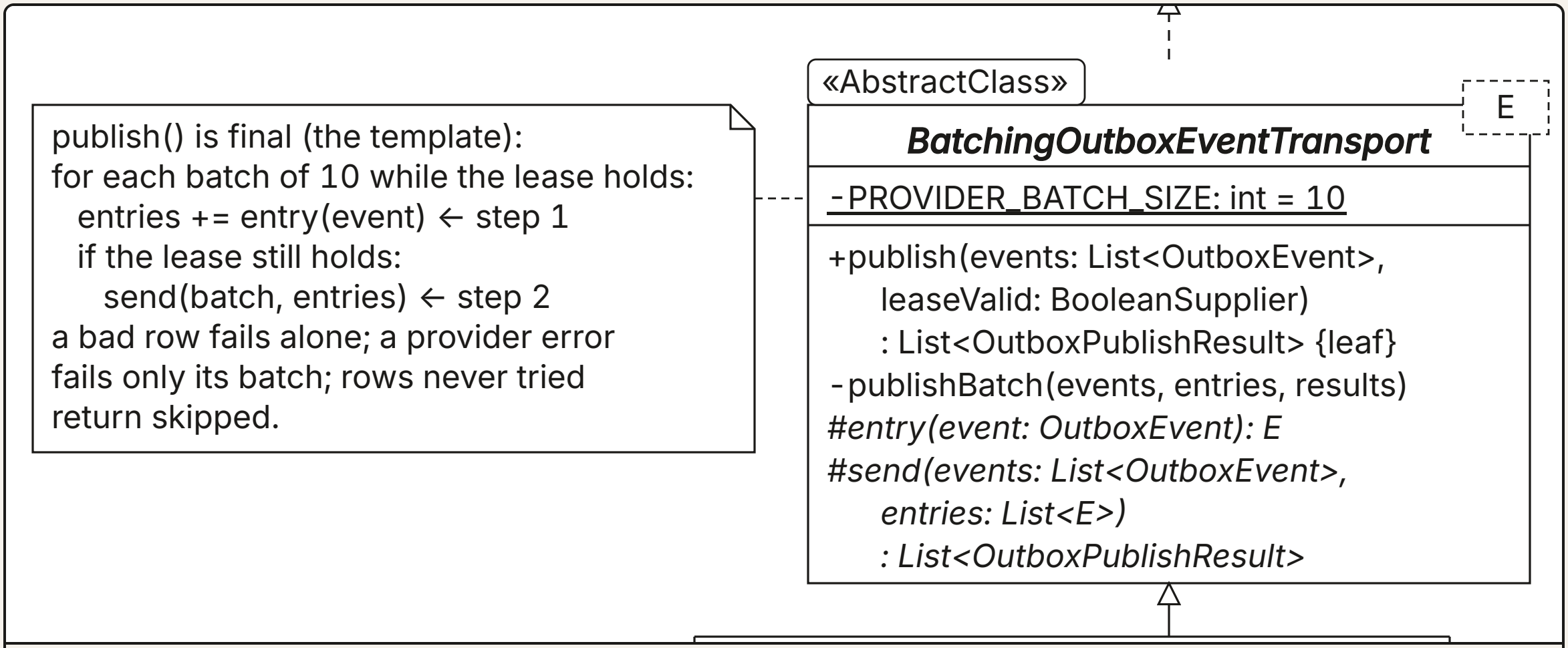
Slide 19 · detail 2/2: publish → before commit, in order



Before commit, in order: `OrderLifecycleSubscriber` (10), then `TeamExpiryRefundSubscriber` (20)
Liability: implicit control flow; a wrong phase is a silent bug
Tested by: `TeamExpiryRefundSubscriberTest`: a failure rolls the expiry back

PS Pan

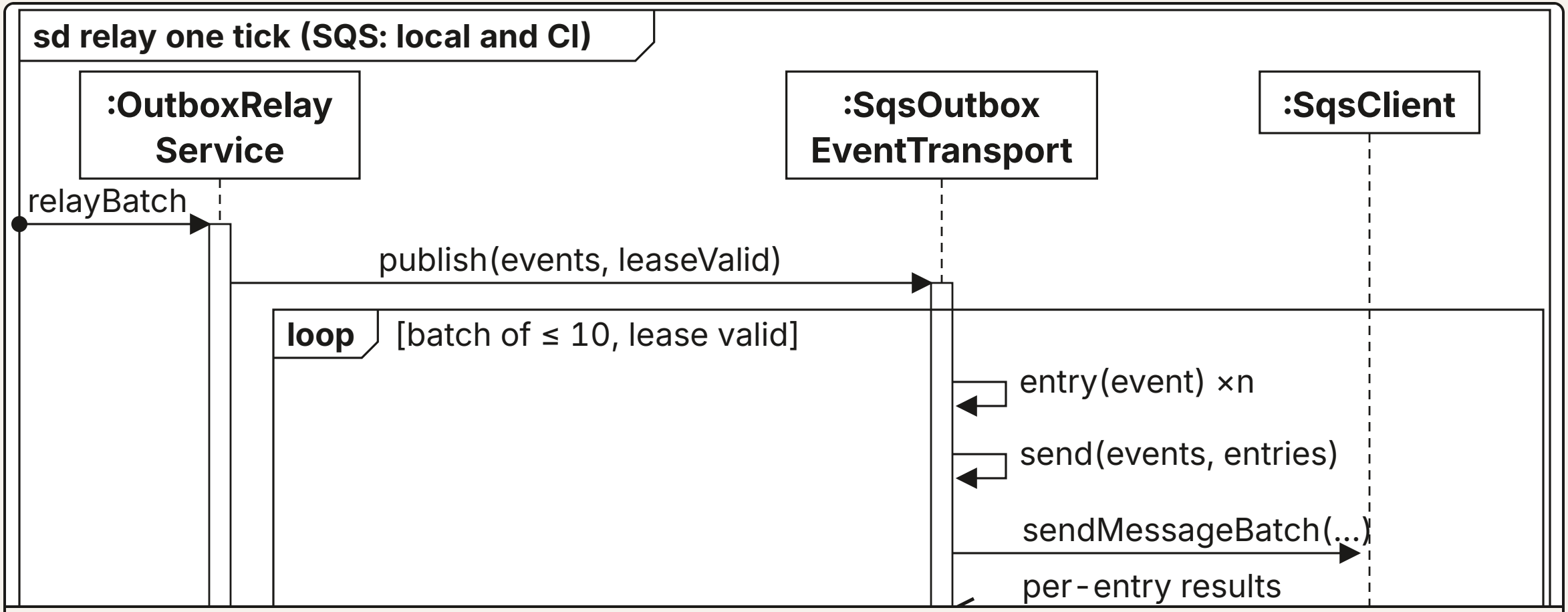
Slide 20 · detail 1/2: template method



AbstractClass: `BatchingOutboxEventTransport<E>`: the final `publish` runs the loop
Candidates: Template Method; Strategy (the whole algorithm does not vary); an adapter per SDK

PS Pan

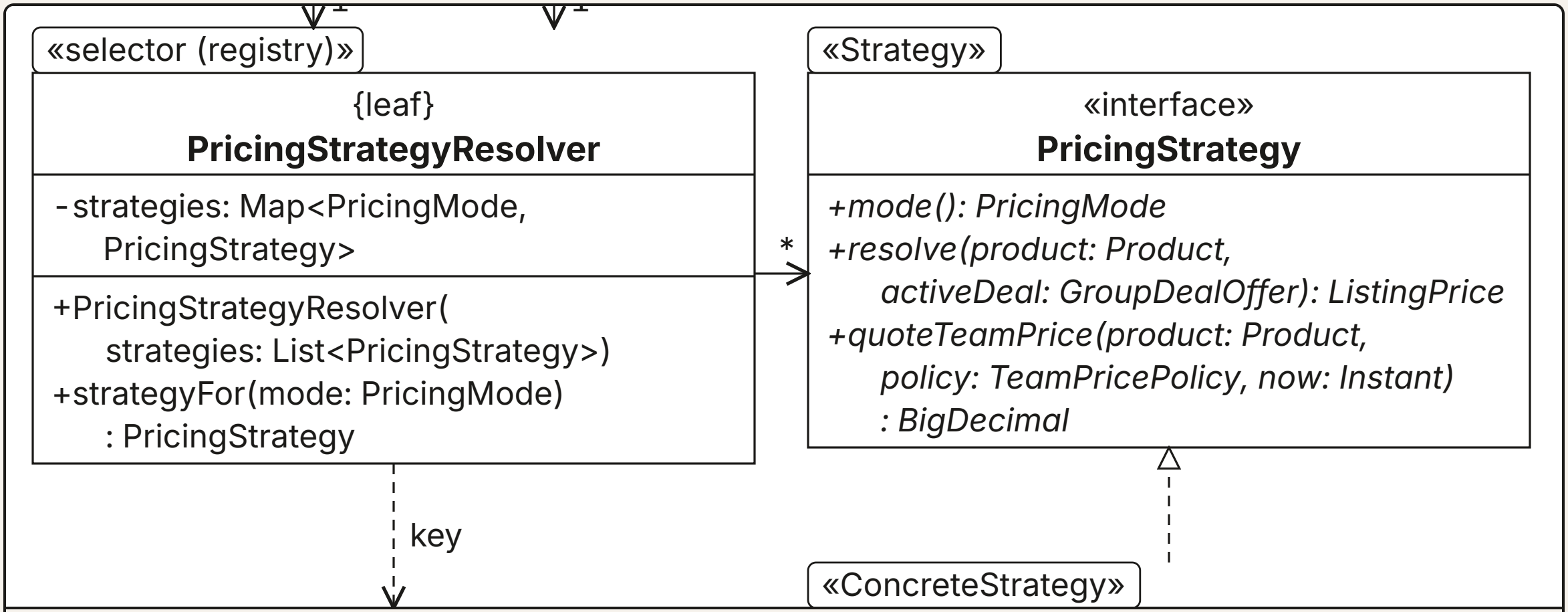
Slide 20 · detail 2/2: publish calls entry, send



Primitive operations: `entry` and `send`: each transport implements only these
Liability: a subclass cannot change the loop
Tested by: `OutboxBatchFailureTest`, both providers

JC Lim

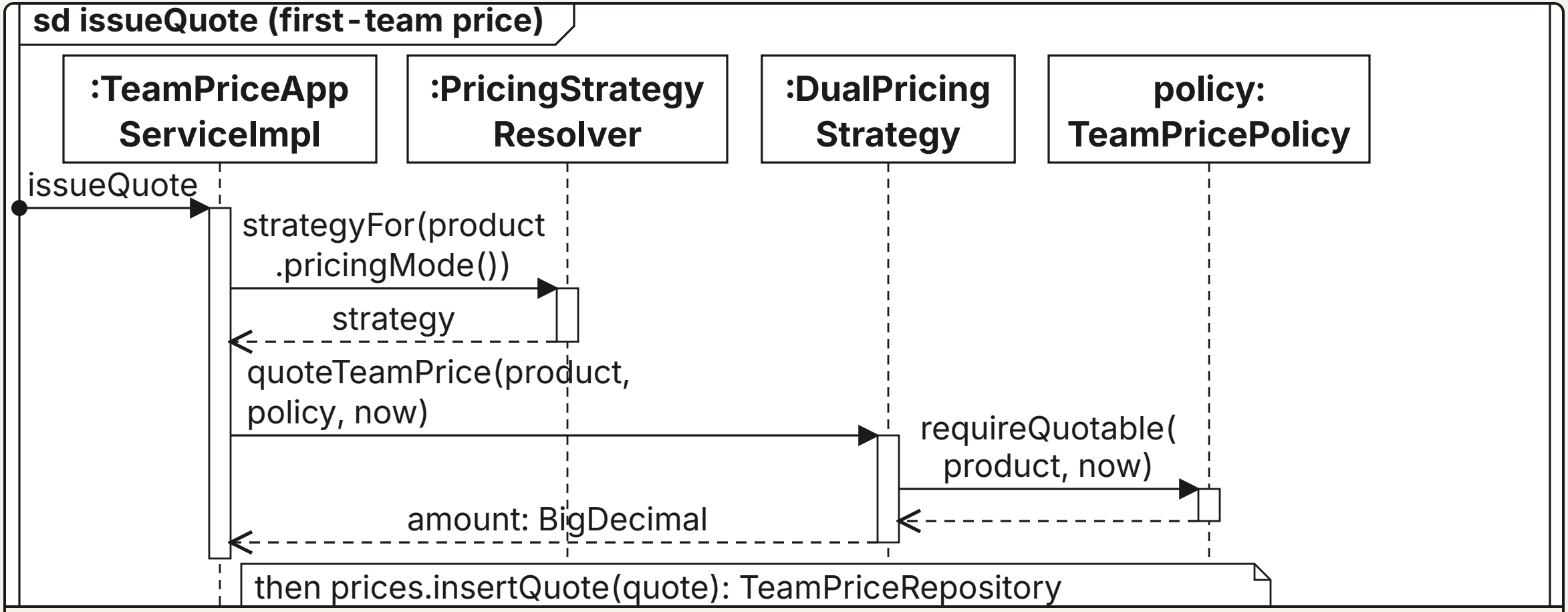
Slide 21 · detail 1/2: resolver · Strategy



Participants: Strategy `PricingStrategy`, picked by a resolver
Contexts: the product and team-price services
Candidates: Strategy, Template Method (no shared skeleton), Decorator

JC Lim

Slide 21 · detail 2/2: quote → resolve → strategy

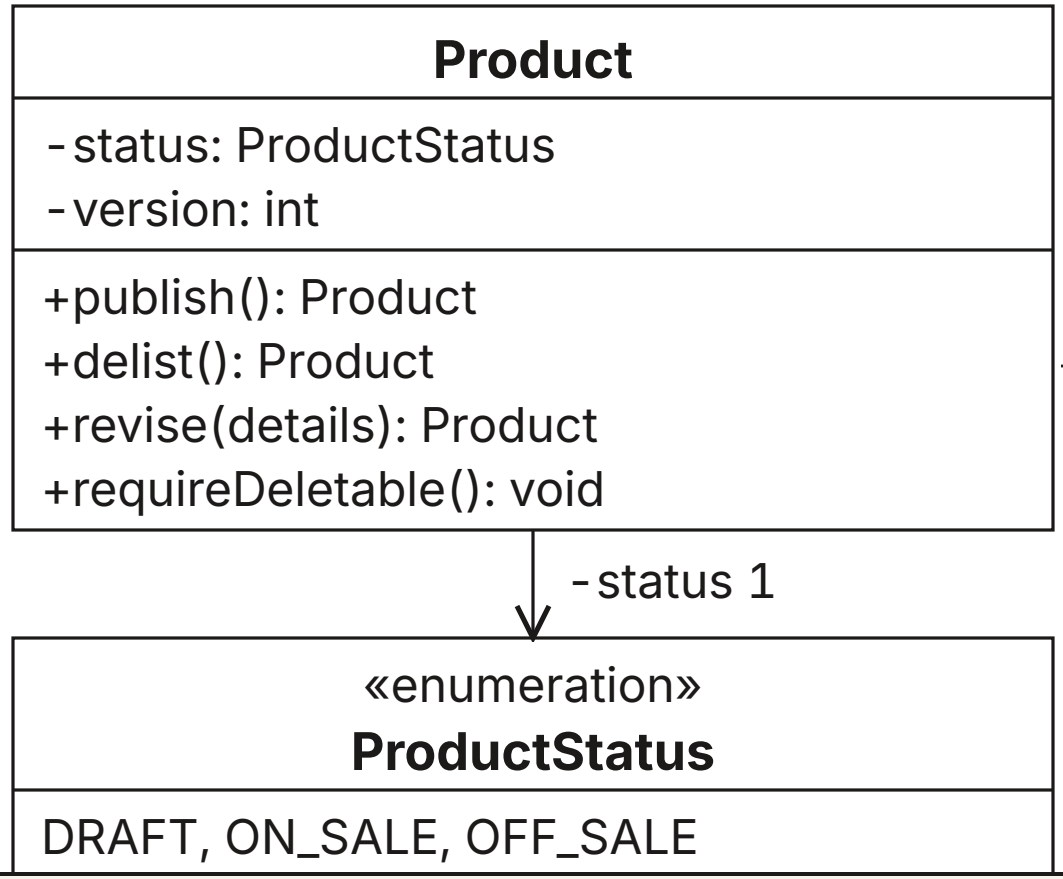


Delegation: each context hands the arithmetic to the ConcreteStrategy `DualPricingStrategy`
Liability: one strategy today; new pricing schemes are a design driver
Tested by: `PricingStrategyResolverTest`, `DualPricingStrategyTest`

LJ Liu Jin

Slide 22 · detail 1/2: brute force: class

Before (brute force): a status switch in every operation



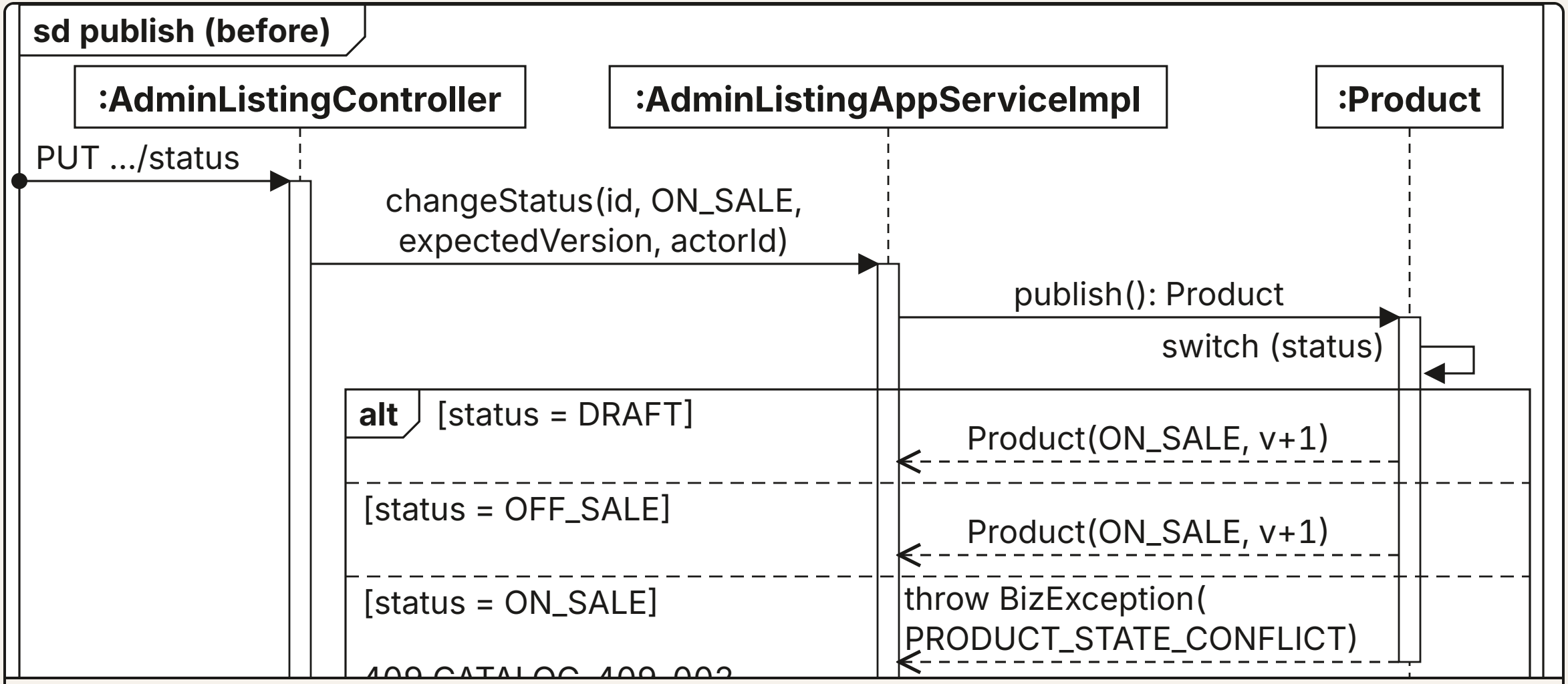
Same switch, repeated 4 times

```
publish(): switch (status)
  DRAFT, OFF_SALE → ON_SALE
  ON_SALE → throw 409
delist(): switch (status)
  ON_SALE → OFF_SALE
  DRAFT, OFF_SALE → throw 409
requireDeletable(): switch
  DRAFT → ok, else throw 409
revise(d): switch (status)
  DRAFT → every field editable
  else 409 if price/stock/currency
```

Brute force: the same `switch (status)` in four operations of `Product`, in red

LJ Liu Jin

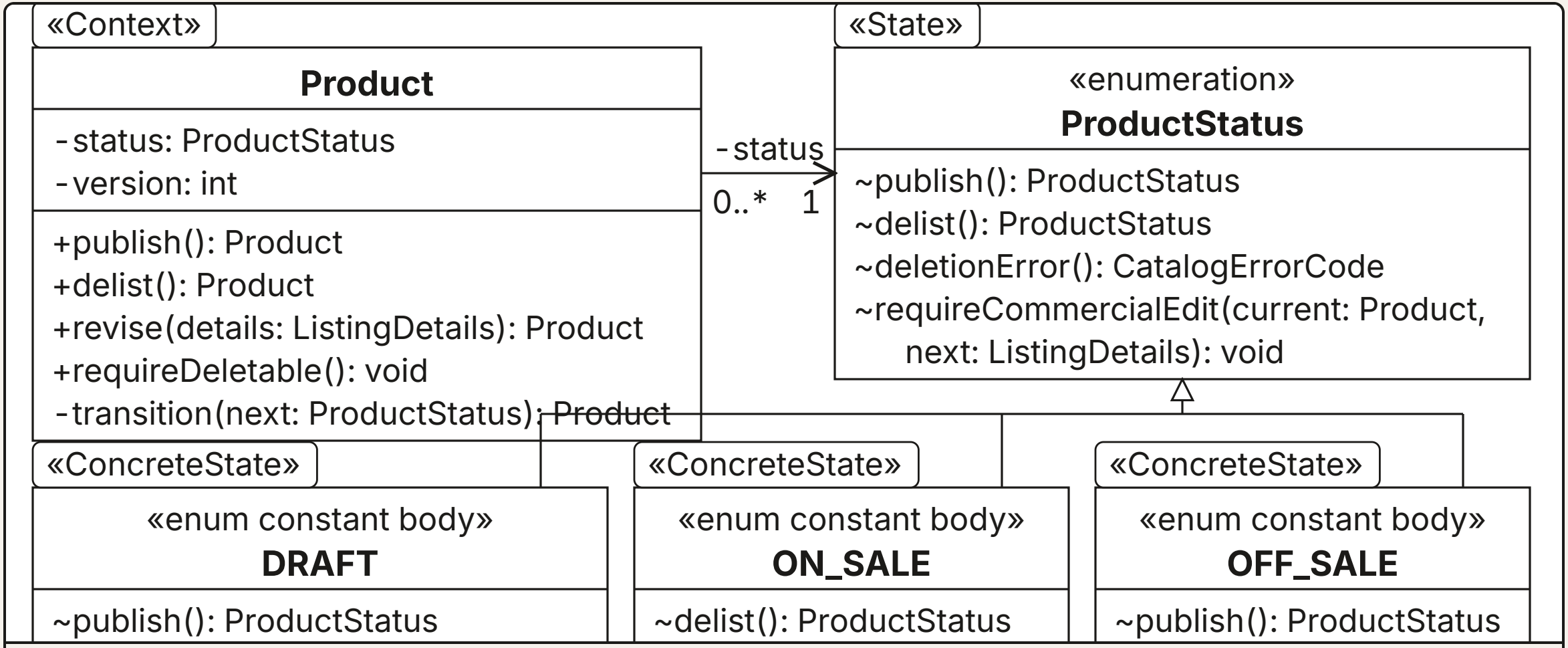
Slide 22 · detail 2/2: brute force: sequence



In the sequence: `publish()` branches on the listing's status

LJ Liu Jin

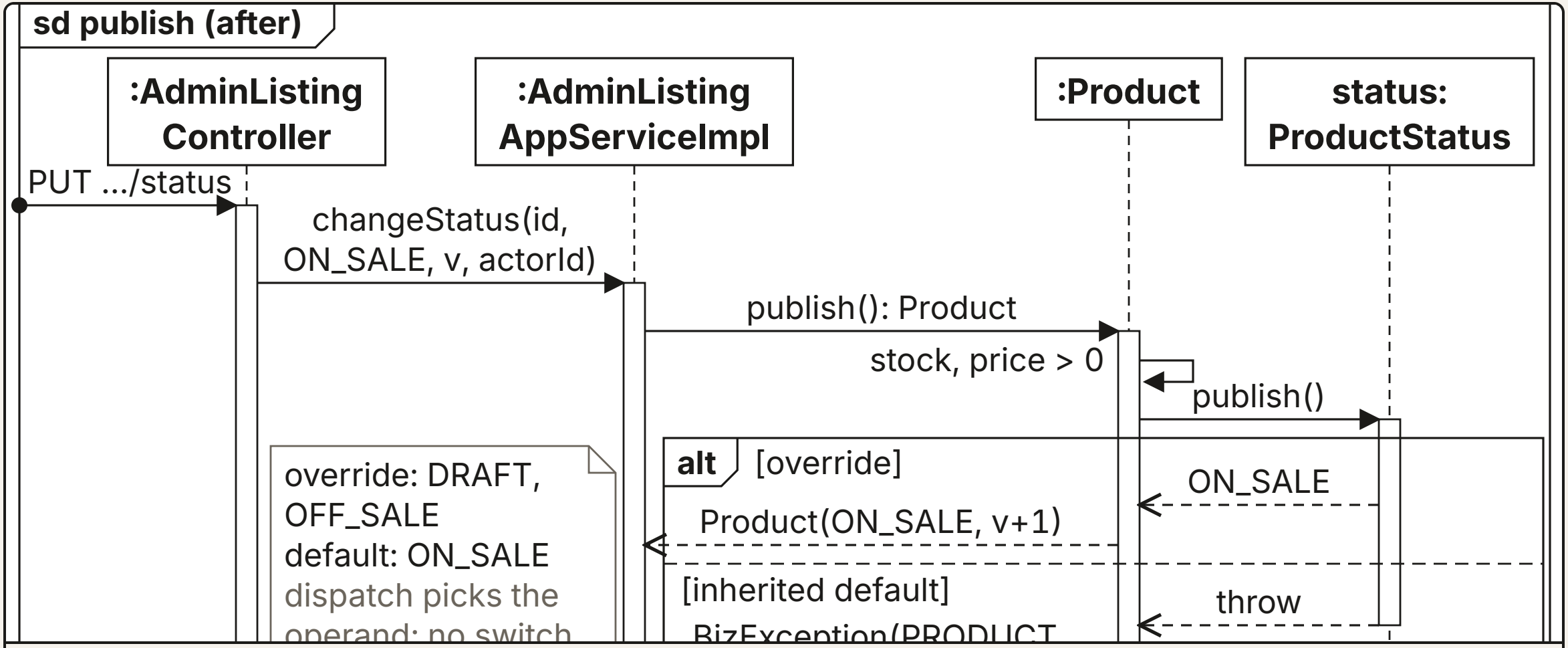
Slide 23 · detail 1/2: State · ConcreteStates



Participants: Context `Product`, State `ProductStatus`
ConcreteStates: DRAFT, ON_SALE, OFF_SALE: enum constants with bodies, one subclass each

LJ Liu Jin

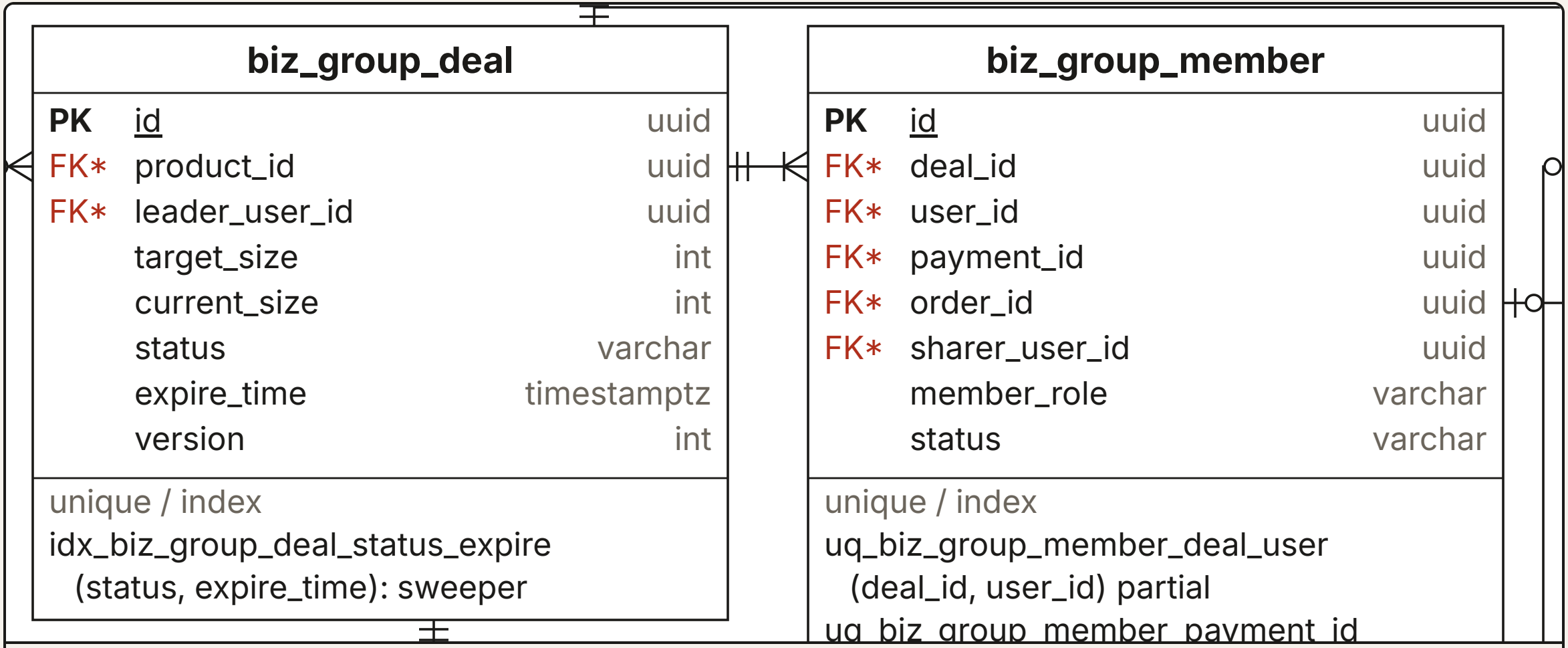
Slide 23 · detail 2/2: after: publish delegates



No switch: each state answers publish its own way
Liability: a new state still needs a constant, its overrides and tests

LJ Liu Jin

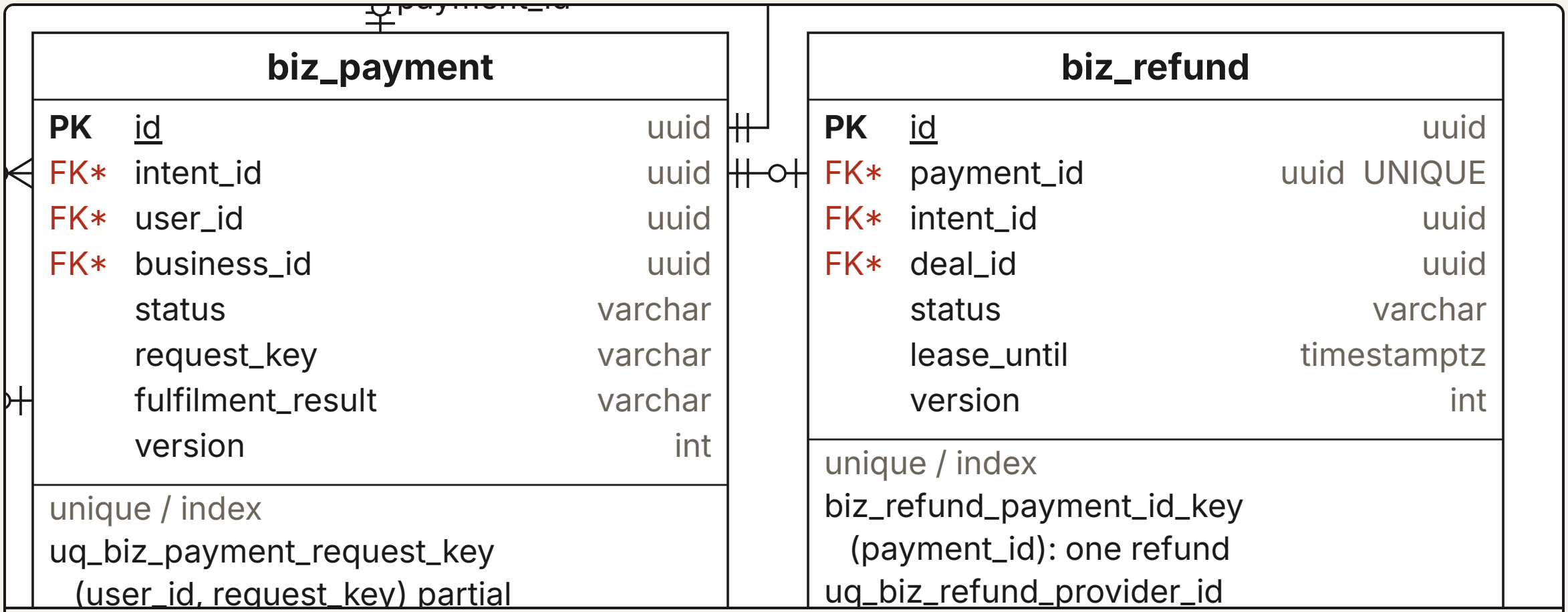
Slide 24 · detail 1/2: deal · membership



Keys: UUIDv7 surrogate primary keys from PostgreSQL 18
Race backstop: one membership per buyer per team, by a unique index

LJ Liu Jin

Slide 24 · detail 2/2: payment · refund



Race backstop: one refund per payment, by a unique key
Associations: logical FK columns checked by the application: our DDL convention, not a course rule
Aggregates: one table per aggregate root; nothing inherited, so no inheritance mapping

LJ Liu Jin

Slide 25 · detail 1/4: Q1 · technology-facing · supporting the team

Q1 technology-facing · supporting the team — **automated**

Unit, component `unit`

JUnit: 162 of 185 test classes
Vitest: 77 files

Integration `integration`

Testcontainers: 45 of 49 classes, on real PostgreSQL, Redis and ElasticMQ

Contract, ArchUnit `integration`

openapi-diff on the committed contract
18 architecture rules

Runs in `build-quality`, `frontend-check` (required), `contract-check`

Caught: two red tests reproduced the join-quote defect before its fix

Complex classes under test

Team state machine	`DealStatusTest`, `PaidGroupDealTest`
Last-seat capacity	`StripeLocalIntegrationTest.twentyBuyersRaceForLastSeat…` (integration)
Refund idempotency	`RefundServiceTest.lostResponseRetriesSameDurableKey`

LJ Liu Jin

Slide 25 · detail 2/4: Q2 · business-facing · supporting the team

Q2 business-facing · supporting the team — **automated**

Story acceptance (system)

`StripeRolloutAcceptanceIT`: real HTTP, scheduler, signed webhooks

End-to-end (system)

Playwright: 8 specs, on the preview build and on dev after each deploy

Runs in `build-quality`, `e2e-preview`, `e2e-dev`

Caught: a 502 on e2e-dev during a deploy (3–4 Oct): one host

LJ Liu Jin

Slide 25 · detail 3/4: Q3 · business-facing · critiquing the product

Q3 business-facing · critiquing the product — **manual**

Exploratory, acceptance system

By hand, logged
Stripe Sandbox START and JOIN in a browser (20 Sep, local)
Sign-in cookie Secure and HttpOnly on dev (8 Oct)

Caught: no defect logged from these runs

Not done: acceptance testing with outside users

LJ Liu Jin

Slide 25 · detail 4/4: Q4 · technology-facing · critiquing the product

Q4 technology-facing · critiquing the product **tools**

Performance, load system

k6 on the SWE5001 ECS environment, by hand (next slide)

Security scanning types

Static analysis (SAST)
Dependencies (SCA)
Image and IaC configuration
Secrets

Runs in `security.yml`, `build-quality`, `docker`: the scans gate the pipeline (slide 30)

Caught: full-history gitleaks: 24 matches; 4 badge tokens still open (slide 32)

PS Pan

Slide 29 · detail 1/4: PR · backend

1 · On every pull request into dev

trigger: pull_request → dev (ci-backend also on label

ci-backend.yml

backend/**, contracts/**

build-quality — required
Spotless + Checkstyle
mvnw verify: unit tests, ArchUnit, integration tests, SpotBugs, JaCoCo
OpenAPI candidate from JAR
verified boot JAR

contract-check
openapi.json drift
openapi-diff: breaking

SonarCloud Backend — required
gate: ≥ 80 % new code

docker — required
image ci-<sha>
boot with PG 18 + Redis 8.10;
/api/actuator/health UP within 60 s;
Trivy image scan

Other PR workflows, path-filtered

typecsfe, smoke (live search contract), perf, tooling (k6 tooling)

On a backend pull request: build-quality runs every unit, architecture and integration test
Also: contract-check fails on drift; docker boots the image and Trivy scans it

PS Pan

Slide 29 · detail 2/4: PR · frontend, security

ci-frontend.yml

frontend/**

frontend-check — required
pnpm check: gen:api, tsc, oxlint, oxfmt, Vitest, knip; build dist/

↓

SonarCloud Frontend — required
≥ 80 % new code

e2e-preview
Playwright on the PR's dist/

security.yml

gitleaks — required
PR commits

workflow-lint
actionlint

trivy-deps
Maven + pnpm

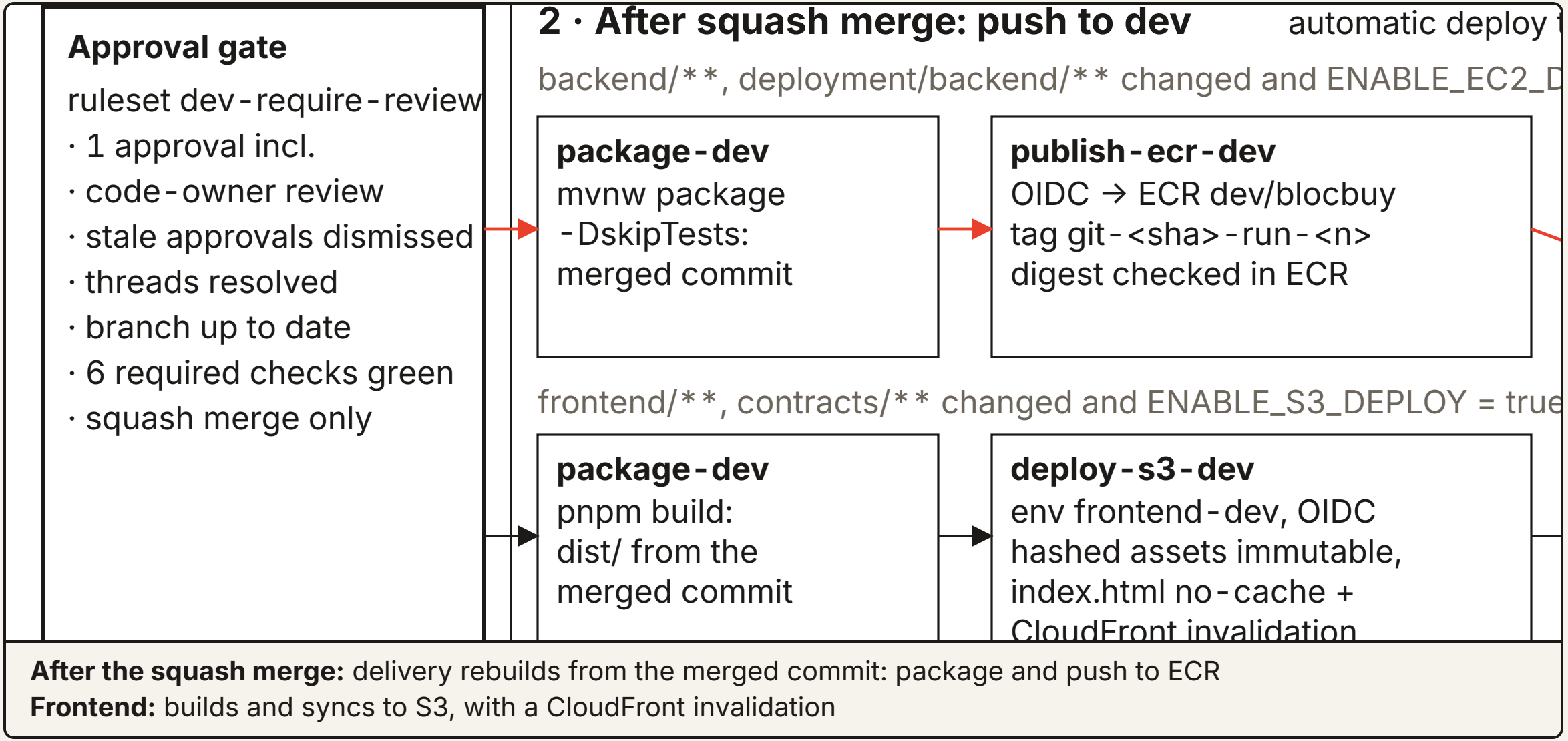
trivy-config
IaC, Docker

deployment-bundle
ShellCheck; Compose must hold only the backend service; SSM bundle file allowlist and size

Frontend: its full check and Playwright on its own build
Security: gitleaks on every pull request; Trivy on dependencies and configuration
Required: the six blue boxes

PS Pan

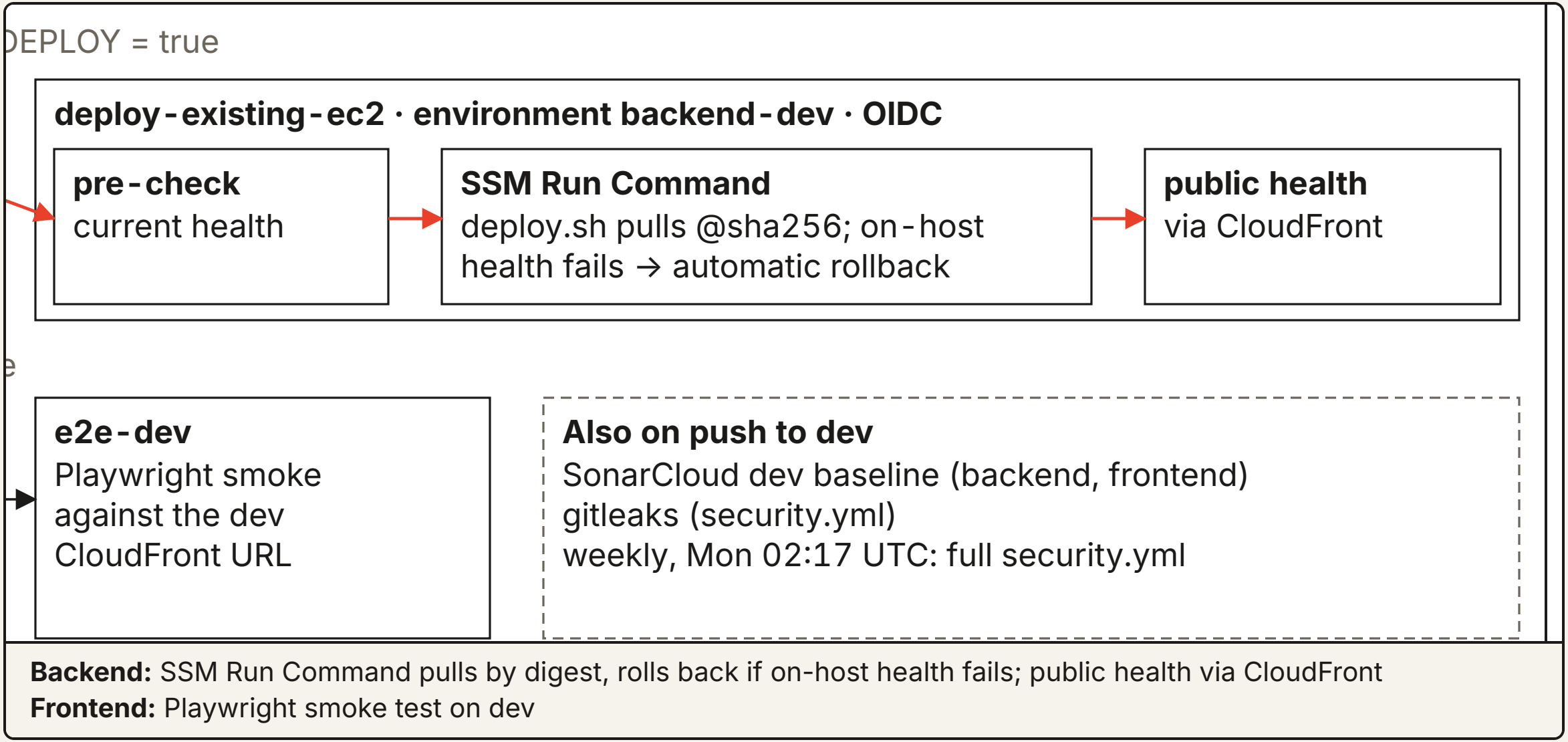
Slide 29 · detail 3/4: gate · delivery



After the squash merge: delivery rebuilds from the merged commit: package and push to ECR
Frontend: builds and syncs to S3, with a CloudFront invalidation

PS Pan

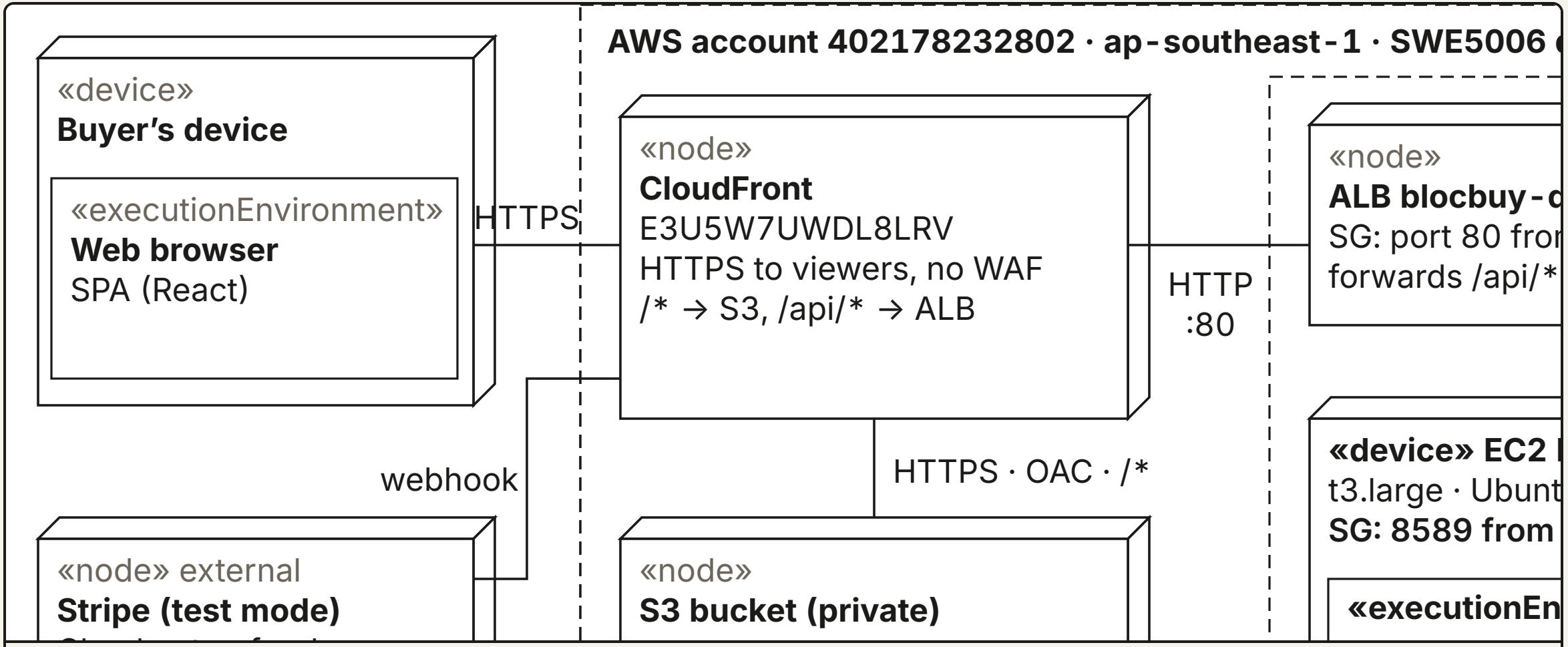
Slide 29 · detail 4/4: deploy · verification



Backend: SSM Run Command pulls by digest, rolls back if on-host health fails; public health via CloudFront
Frontend: Playwright smoke test on dev

PS Pan

Slide 31 · detail 1/3: request path

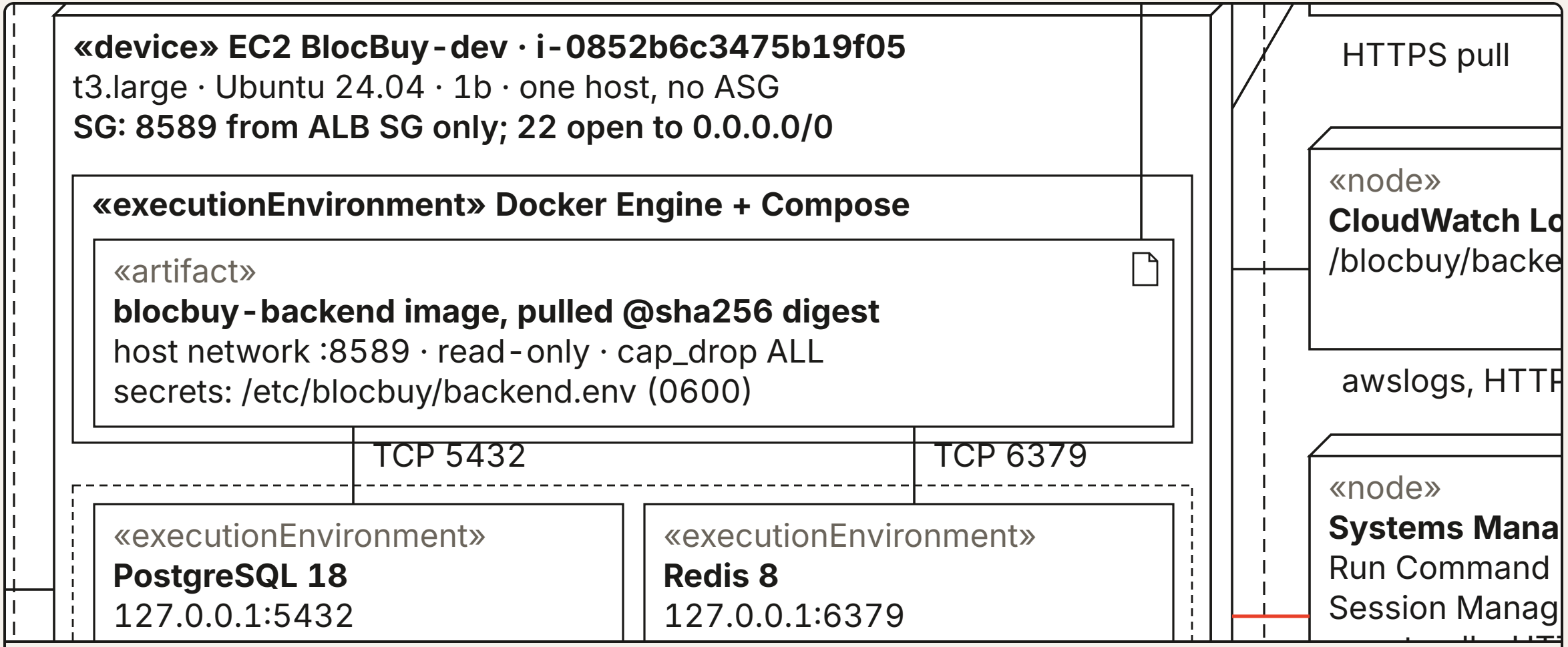


One environment: dev; the SWE5006 production stack was planned, never applied

Request path: the browser talks only to CloudFront: files from a private bucket, API calls to the load balancer

PS Pan

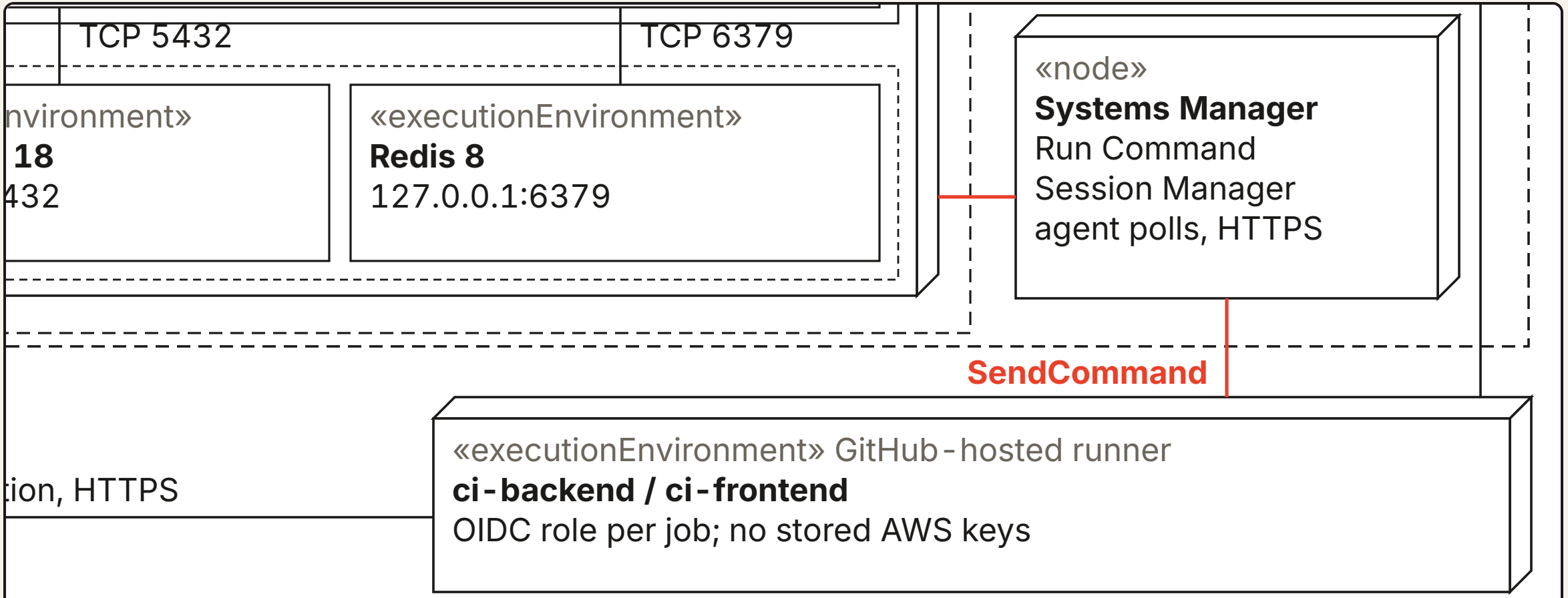
Slide 31 · detail 2/3: one host



One host: the container runs by digest, beside PostgreSQL and Redis on loopback
Trade-off: cheap and always on, but one point of failure

PS Pan

Slide 31 · detail 3/3: deploy path



Approver: the code owner on the pull request; the merge releases
Practice: every merge deploys automatically to dev, not production; SWE5001 production is Continuous Delivery
Verified by: on-host health check with automatic rollback, public health, Playwright smoke